

5.1.2 搜索策略分类

(1) 从初始状态出发的正向搜索,也称为数据驱动

正向搜索是从问题给出的条件——一个用于状态转换的操作算子集合出发的。搜索的过程为应用操作算子从给定的条件中产生新条件,再用操作算子从新条件产生更多的新条件,这个过程一直持续到有一条满足目的要求的路径产生为止。数据驱动就是用问题给定数据中的约束知识指导搜索,使其沿着那些已知正确的线路前进。

(2) 从目的状态出发的逆向搜索,也称为目的驱动

逆向搜索是先从想达到的目的入手,看哪些操作算子能产生该目的以及应用这些操作算子产生目的时需要哪些条件,这些条件就成为我们要达到的新目的,即子目的。逆向搜索就通过反向的连续的子目的不断进行,直至找到问题给定的条件为止。这样就找到了一条从数据到目的的操作算子所组成的链。

根据搜索过程中是否运用与问题有关的信息,可以将搜索方法分为启发式搜索和盲目搜索。

盲目搜索(blind search)是指在对特定问题不具有任何有关信息的条件下,按固定的步骤(依次或随机调用操作算子)进行的搜索,它能快速地调用一个操作算子。

启发式搜索(heuristic search)则是考虑特定问题领域可应用的知识,动态地确定调用操作算子的步骤,优先选择较适合的操作算子,尽量减少不必要的搜索,以求尽快地到达结束状态,提高搜索效率。

盲目搜索中,由于没有可参考的信息,只要能匹配的操作算子都须运用,这会搜索出更多的状态,生成较大的状态空间显示图;而启发式搜索中,运用一些启发信息,只采用少量的操作算子,生成较小的状态空间显示图,就能搜索到一个解答,但是每使用一个操作算子便需做更多的计算与判断。启发式搜索一般要优于盲目搜索,但不可过于追求更多的甚至完整的启发信息。

5.2 状态空间的搜索策略

5.2.1 状态空间表示法

状态空间(state space)表达法是知识表示的一种基本方法。

状态是用来表示系统状态、事实等叙述型知识的一组变量或数组:

$$Q = [q_1, q_2, \dots, q_n]^T \quad (5.1)$$

操作是用来表示引起状态变化的过程型知识的一组关系或函数:

$$F = \{f_1, f_2, \dots, f_m\} \quad (5.2)$$

状态空间是利用状态变量和操作符号,表示系统或问题的有关知识的符号体系。状态空

间可以用一个四元组表示:

$$(S, O, S_0, G)$$

其中, S 是状态集合。 S 中每一元素表示一个状态, 状态是某种结构的符号或数据。 O 是操作算子的集合。利用算子可将一个状态转换为另一个状态。 S_0 是包含问题的初始状态, 是 S 的非空子集, $S_0 \subset S$ 。 G 是包含问题的目的状态, 是 S 的非空子集, $G \subset S$ 。 G 可以是若干具体状态, 也可以是满足某些性质的路径信息描述。

从 S_0 结点到 G 结点的路径称为求解路径。求解路径上的操作算子序列为状态空间的一个解。例如, 操作算子序列 $O_1, \dots, O_k$ 使初始状态转换为目标状态, 如图 5.1 所示。

$$S_0 \xrightarrow{O_1} S_1 \xrightarrow{O_2} S_2 \xrightarrow{O_3} \dots \xrightarrow{O_k} G$$

图 5.1 状态空间的解

则 $O_1, \dots, O_k$ 即为状态空间的一个解。当然, 解往往不是唯一的。

任何类型的数据结构都可以用来描述状态, 如符号、字符串、向量、多维数组、树和表格等。所选用的数据结构形式要与状态所蕴含的某些特性具有相似性。例如对八数码问题, 一个 3×3 的阵列便是一个合适的状态描述方式。

例 5.1 八数码问题的状态空间表示。

八数码问题(重排九宫问题)是在一个 3×3 的方格盘上, 放有 1~8 的数码, 余下一格为空。空格四周上下左右的数码可移到空格。需要找到一个数码移动序列使初始的无序数码转变为一些特殊的排列。例如, 图 5.2(a) 所示的八数码问题的初始状态为问题的一个布局, 需要找到一个数码移动序列使其转变为图 5.2(b) 所示的目标状态。

2	3	1
5		8
4	6	7

(a) 初始状态

1	2	3
8		4
7	6	5

(b) 目标状态

图 5.2 八数码问题

该问题可以用状态空间来表示。此时八数码的任何一种摆法就是一个状态, 所有的摆法即为状态集 S , 它们构成了一个状态空间, 其数目为 $9!$ 。而 G 是指定的某个或某些状态。对于操作算子设计, 如果着眼在数码上, 相应的操作算子就是数码的移动, 其操作算子共有 $4(\text{方向}) \times 8(\text{数码}) = 32$ 个。如着眼在空格上, 即空格在方格盘上的每个可能位置的上下左右移动, 其操作算子可简化成 4 个:

如果空格上边有数字, 则将空格向上移 Up;

如果空格左边有数字, 则将空格向左移 Left;

如果空格下边有数字,则将空格向下移 Down;

如果空格右边有数字,则将空格向右移 Right。

移动时要确保空格不会移出方格盘之外,因此并不是在任何状态下都能运用这4个操作算子。如空格在方格盘的右上角时,只能运用两个操作算子——向左移 Left 和向下移 Down。

5.2.2 状态空间的图描述

状态空间可用有向图来描述,图的结点表示问题的状态,图的弧表示状态之间的关系,就是求解问题的步骤。初始状态对应于实际问题的已知信息,是图中的根结点。问题的状态空间描述中,寻找从一种状态转换为另一种状态的某个操作算子序列就等价于在一个图中寻找某一路径。

图 5.3 所示为用有向图描述的状态空间。该图表示对状态 S_0 允许使用操作算子 O_1, O_2 及 O_3 , 并分别使 S_0 转换为 S_1, S_2 及 S_3 。这样一步步利用操作算子转换下去,如 $S_{10} \in G$, 则 O_2, O_6, O_{10} 就是一个解。

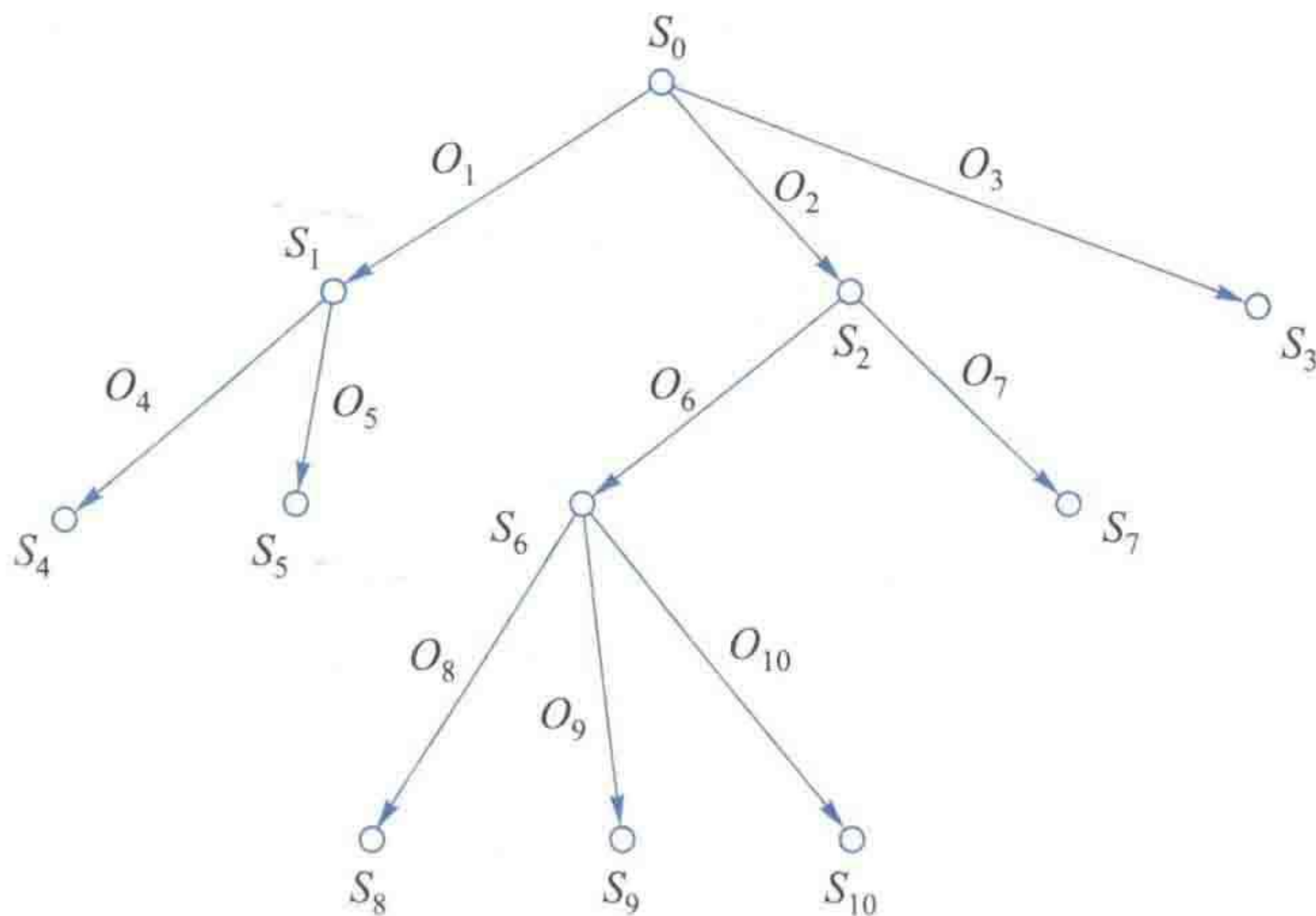


图 5.3 状态空间的有向图描述

上面是较为形式化的说明,下面再以八数码问题为例,讨论具体问题的状态空间的有向图描述。

例 5.2 对于八数码问题,如果给出问题的初始状态,就可以用图来描述其状态空间。其中的弧可用表明空格的4种可能移动的4个操作算子来标注,即空格向上移 Up、向左移 Left、向下移 Down、向右移 Right。该图的部分描述如图 5.4 所示。

在某些问题中,各种操作算子的执行是有不同费用的。如在旅行商问题中,两两城市之间的距离通常是不相等的,那么,在图中只需要给各弧线标注距离或费用即可。下面以旅行商问题为例说明这类状态空间的图描述,其终止条件则是用求解路径本身的特点来描述,即经过图中所有城市的最短路径被找到时搜索便结束。

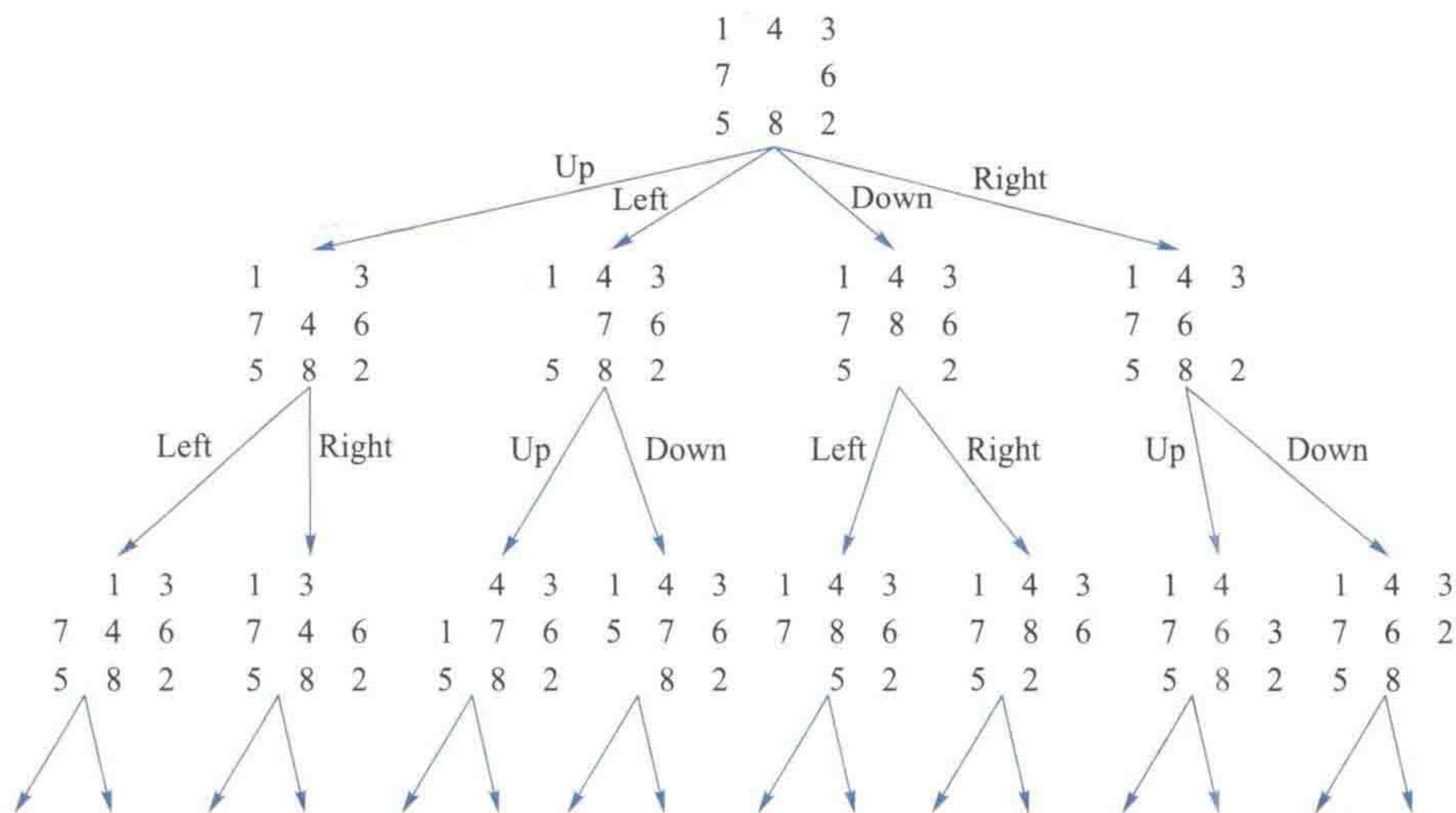


图 5.4 八数码状态空间图(部分)

例 5.3 旅行商问题(traveling salesman problem, TSP)或旅行推销员问题:假设一个推销员从出发地到若干个城市去推销产品,然后回到出发地,要求每个城市必须去一次,而且只能去一次。问题是要找到一条最好的路径,使得推销员所经过的路径最短或者费用最少。

图 5.5 是这个问题的一个实例,图中结点代表城市,弧上标注的数值表示经过该路径的费用(或距离)。我们假定推销员从 A 城出发。

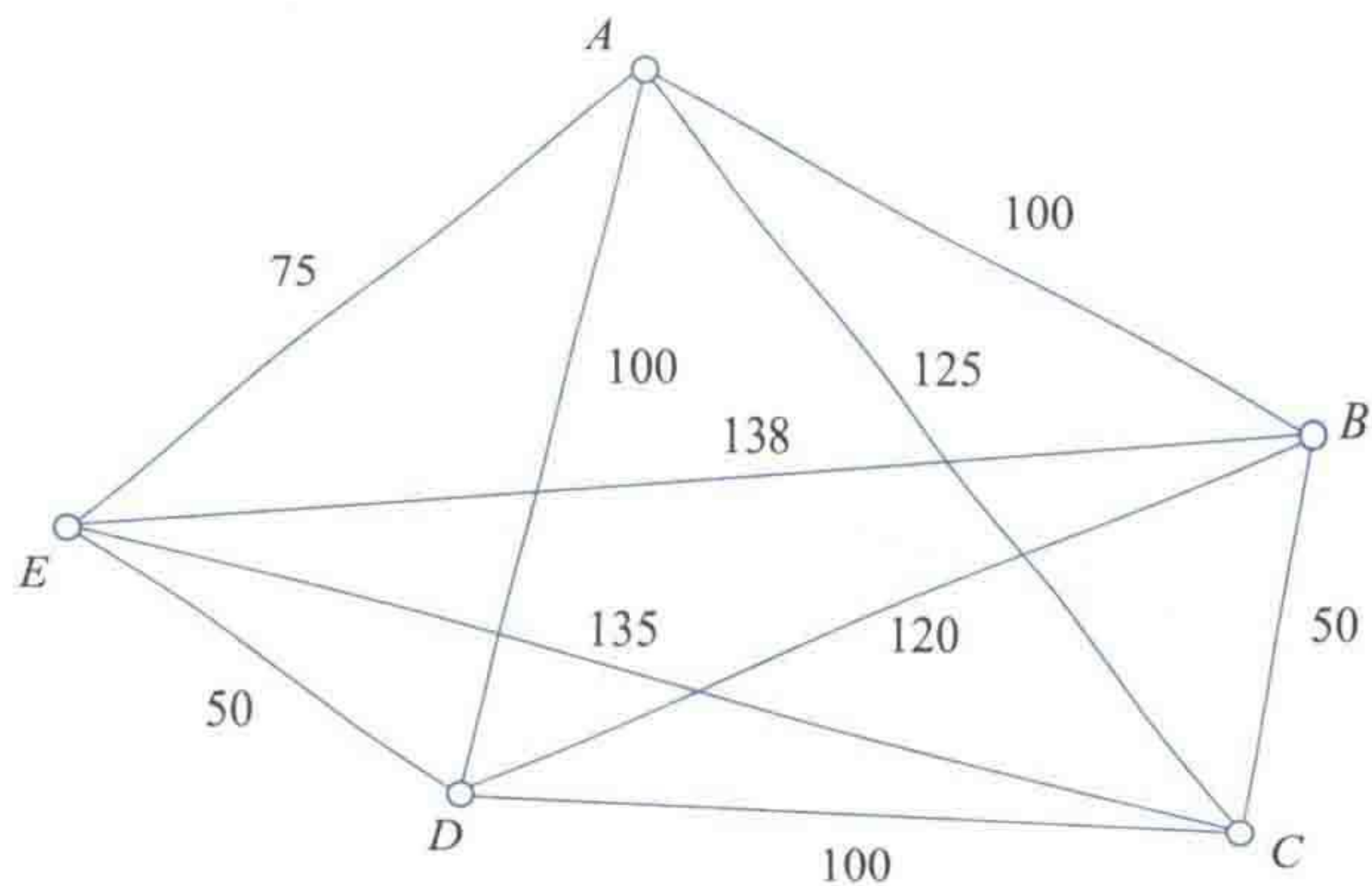


图 5.5 旅行商问题的一个实例

图 5.6 是该问题的部分状态空间表示。可能的路径有很多,例如,费用为 375 的路径(A, B, C, D, E, A)就是一个可能的旅行路径,但目的是要找具有最小费用的旅行路径。

上面两个例子中,只绘出了问题的部分状态空间图,当然,完全可以绘出问题的全部状态空间图,但对许多实际问题,要在有限的时间内绘出问题的全部状态图是不可能的。例如旅行

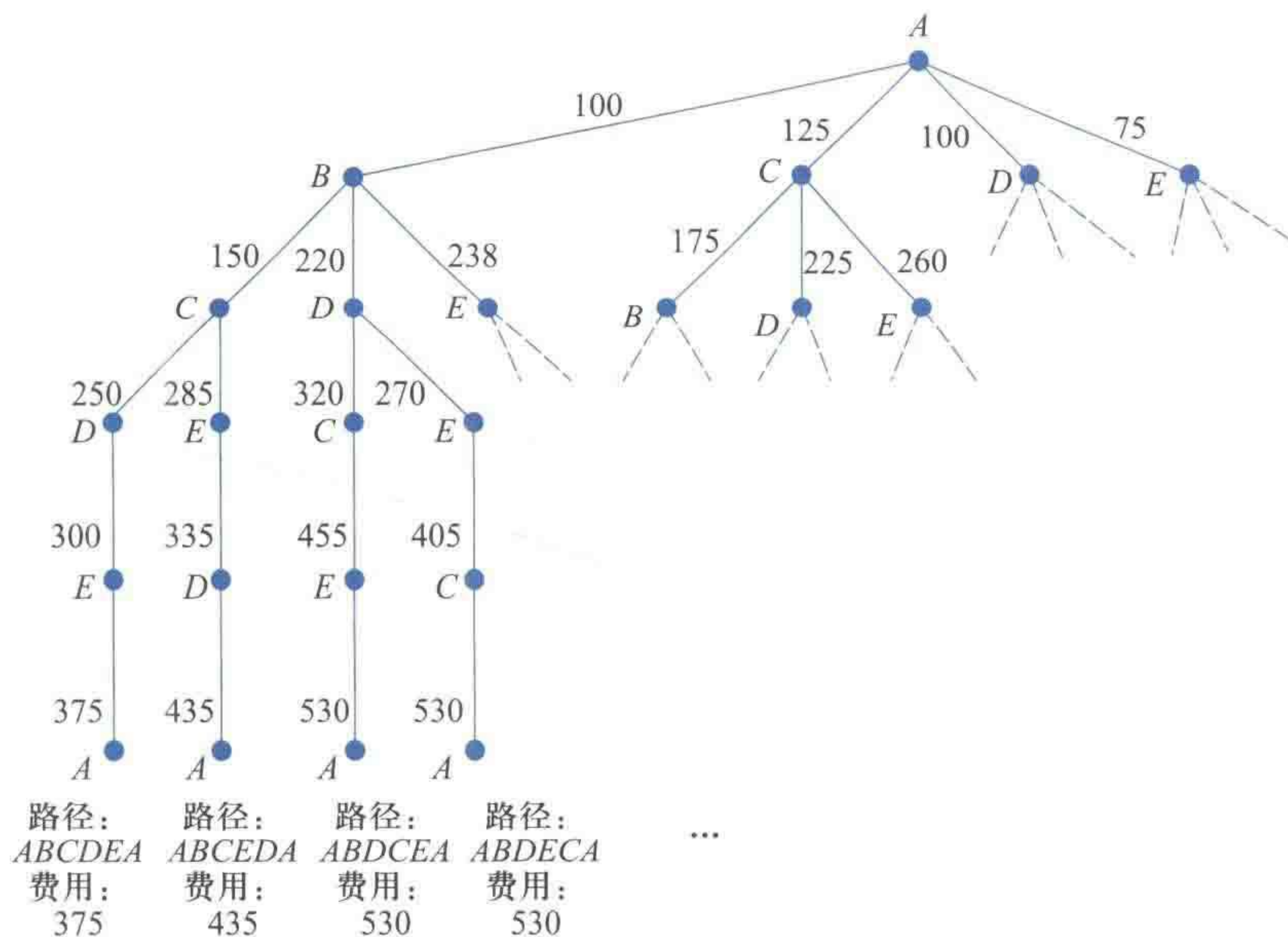


图 5.6 旅行商问题的状态空间图(部分)

商问题, n 个城市存在 $\frac{1}{2}(n-1)!$ 条路径。如果用 10^8 次/秒的计算机进行穷举,则当 $n=7$ 时,搜索时间为 $t=2.5\times 10^{-5}$ s;当 $n=15$ 时, $t=1.8$ h;当 $n=20$ 时, $t=350$ 年;当 $n=50$ 时, $t=5\times 10^{48}$ 年;当 $n=100$ 时, $t=5\times 10^{142}$ 年;当 $n=200$ 时, $t=5\times 10^{358}$ 年。因此,这类显式表示对于大型问题的描述是不切实际的,而对于具有无限结点集合的问题则是不可能的。因此要研究能够在有限时间内搜索到较好解的搜索算法。

状态空间搜索是搜索某个状态空间以求得操作算子序列的一个解答的过程。这种搜索是状态空间问题求解的基础。

搜索策略的主要任务是确定选取操作算子的方式。它有两种基本方式:盲目搜索和启发式搜索。

5.3 盲目的图搜索策略

5.3.1 回溯策略

不管是正向搜索还是逆向搜索求解问题,都是在状态空间图中找到从初始状态到目的状态的路径。路径上弧的序列对应于解题的步骤。若在选择操作算子求解问题时,能给出绝对可靠的预测或者绝对正确的选择策略,一次性成功穿过状态空间达到目的,构造出一条解题路

径,那就不需要所谓的搜索了。但事实上,不是总能给出正确的选择,求解实际问题时必须尝试多条路径才能找到目的状态。一个搜索算法的策略就是要决定树或图中状态的搜索次序。回溯策略是一种系统地尝试状态空间中各种不同路径的技术。

回溯策略的搜索是从初始状态出发,不停地试探性地寻找路径,直到到达目的或“不可解结点”,即“死胡同”为止。回溯策略是当遇到不可解结点时就回溯到路径中最近的父结点上,查看该结点是否还有其他的子结点未被扩展。若有,则沿这些子结点继续搜索;如果找到目标,就成功退出搜索,返回解题路径。

可以看出,回溯策略的搜索过程呈现出递归过程的性质。搜索过程在每个结点上的检查遵循着递归方式。下面给出递归过程。

Step Track (DataList):	
Data := First(DataList);	* 当前状态为状态序列表中第一个状态
if Member(Data, Tail(DataList)) then return FAIL;	* 回老路,退回
if Goal(Data) then return NIL;	* 达到目的状态,成功返回
if DeadEnd(Data) then return FAIL;	* 达到不合理状态,退回
if Length(DataList) > Bound then return FAIL;	* 已到深度限制,退回
Rules := AppRules(Data);	* 得出可应用的规则集
Loop: if Null(Rules) then return FAIL;	* 进入死胡同,退回
R := First(Rules);	* 取出第一条可用规则
Rules := Tail(Rules);	
Newdata := Gen(R, Data);	* 运用规则,生成新状态
NewDataList := Cons(Newdata, DataList);	
Path := Back Track(NewDataList);	* 递归
If Path := FAIL then go Loop	
else return Cons(R, Path);	

说明:变量符号 Data, DataList, Rules, R, Newdata, NewdataList, Path 分别表示当前状态、状态序列表、规则集序列表、当前运用的规则、生成的新状态、生成的新状态序列表、当前解路径表;常量符号 FAIL, NIL, Bound, Loop 分别表示回溯点标记、空表、深度限制值、循环标号;函数 First(y)表示在 y 表中取第一个元素,函数 Tail(x)是取除了第一个元素的 x 表的其余部分,函数 Member(x, y)表示变量 x 是否为 y 表的一个元素,函数 Goal(x)表示变量状态 x 是否为目的状态,函数 DeadEnd(x)表示变量状态 x 是否为不可能解路径上的状态,函数 Length(x)是取 x 表的表长度,函数 AppRules(x)是求出在变量状态 x 上可运用的规则集,函数 Null(x)表示 x 表是否为空表,函数 Gen(x, y)是求 x 规则运用在 y 状态后所生成的新状态,函数 Cons(x, y)是将变量状态 x 加在 y 表的前部。

图 5.7 为一个状态空间中应用回溯搜索的示意图,图中虚线箭头的方向表示搜索的轨迹,结点边的数字表示被搜索到的次序。

回溯搜索的算法用三张表来保存状态空间中的不同性质结点。

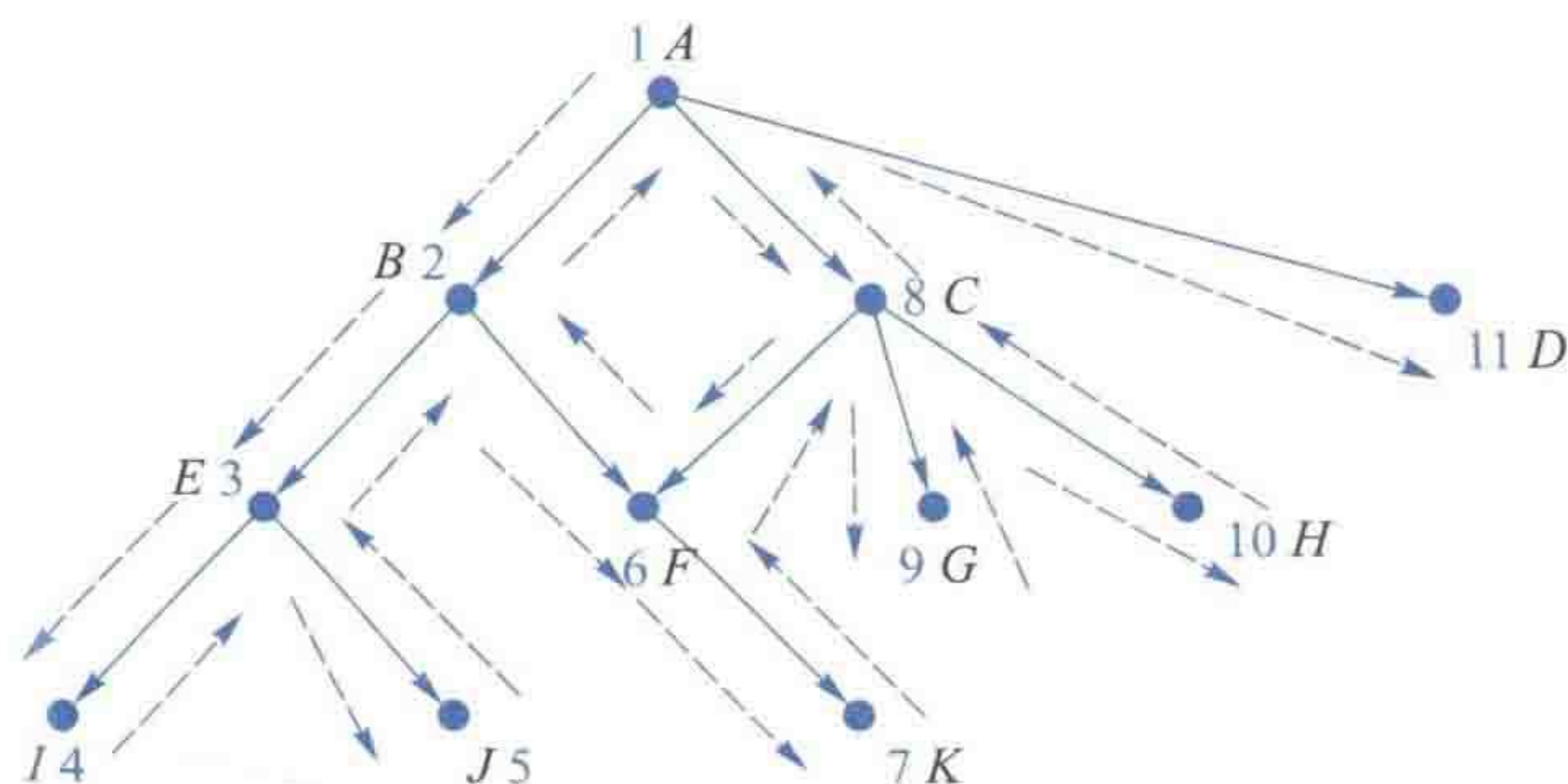


图 5.7 回溯搜索示意图

(1) 路径状态表

路径状态(path states, PS)表保存当前搜索路径上的状态。如果找到了目的, PS 就是解路径上的状态有序集。

(2) 新的路径状态表

新的路径状态(new path states, NPS)表包含了等待搜索的状态, 其后裔状态还未被搜索到, 即未被生成扩展。

(3) 不可解状态表

不可解状态(no solvable states, NSS)表列出了找不到解路径的状态。如果在搜索中扩展出的状态是它的元素, 则可立即将之排除, 不必沿该状态继续搜索。

为了避免造成无穷循环搜索, 需要检测并删除多次出现的那些状态。具体检测可通过判断每一个新生成的状态是否在 PS, NPS, NSS 三张表中来实现。如果它属于其中一张表, 就说明它已被搜索过而不必再考虑。

当前正在被检测的状态, 记作 CS (current state)。CS 总是等于最近加入 PS 中的状态, 是当前正在探寻解题路径的“前锋”。各种合适的推理规则或其他问题求解操作都可应用于 PS。应用后一般便得到一些新状态, 即 PS 的子状态的有序集, 然后再将该集合中的第一个子状态作为当前状态 CS, 并加入 PS 中, 其余的则按序放入 NPS 中, 用于以后的搜索。如果应用后 CS 没有子状态, 则要从 PS、NPS 中删除它, 同时将其加入 NSS, 之后回溯查找 NPS 中表首位置的状态。

Function backtrack:

begin

PS := [Start]; NPS := [Start]; NSS := []; CS := Start; * 初始化

while NPS ≠ [] do

begin

if CS = 目的状态 then return(PS);

* 成功, 返回解题路径

if CS 没有子状态(不包括 PS, NPS 和 NSS 中已有的状态)

then

begin

```
while ((PS 非空) and (CS=PS 中的第一个元素)) do
    begin
        将 CS 加入 NSS                                * 标明此状态不可解
        从 PS 中删除第一个元素 CS                      * 回溯
        从 NPS 中删除第一个元素 CS;
        CS:= NPS 中的第一个元素;
    end;
    将 CS 加入 PS;
end
else
    begin
        将 CS 子状态(不包括 PS、NPS 和 NSS 中已有的)加入 NPS;
        CS:= NPS 中的第一个元素;
        将 CS 加入 PS;
    end
end;
return FAIL;
end.
```

图 5.7 的回溯轨迹如下：
初值:PS=[A];NPS=[A];NSS=[];CS=A。

重复	CS	PS	NPS	NSS
0	A	[A]	[A]	[]
1	B	[BA]	[BCDA]	[]
2	E	[EBA]	[EFBDCA]	[]
3	I	[IEBA]	[IJEFBCDA]	[]
4	J	[JEBA]	[JEFBCDA]	[I]
5	F	[FBA]	[FBCDA]	[EJI]
6	K	[KFBA]	[KFBCDA]	[EJI]
7	C	[CA]	[CDA]	[BFKEJI]
8	G	[GCA]	[GHCD A]	[BFKEJI]

上面的搜索过程显示出回溯是状态空间中的一个正向搜索。它将初始条件作为初始状态,对其子状态进行搜索以寻找目的。如将目的作为搜索图的根(即初始状态),本算法便可看作逆向搜索。如对算法中“成功,返回解题路径”的判别条件“CS=目的状态”修改为“搜索路径的性质优劣”,那算法必须通过检查 PS 中的路径来确定是否到达目的。

回溯是状态空间搜索的一个基本算法。各种图搜索算法,包括深度优先、宽度优先、最好优先搜索等都有回溯的思想,其中宽度、深度优先搜索是状态空间的最基本的搜索策略。

5.3.2 宽度优先搜索策略

宽度优先搜索策略(breadth first search, BFS)是按照图 5.8 所示的次序来搜索状态的。它类似树的层次遍历,是树的按层次遍历的推广。由 S_0 生成状态 1、2,然后扩展状态 1,生成状态 3、4、5,接着扩展状态 2,生成状态 6、7、8,该层扩展完后,再进入下一层,对状态 3 进行扩展,如此一层一层地扩展下去,直到搜索到目的状态(如果目的状态存在)。

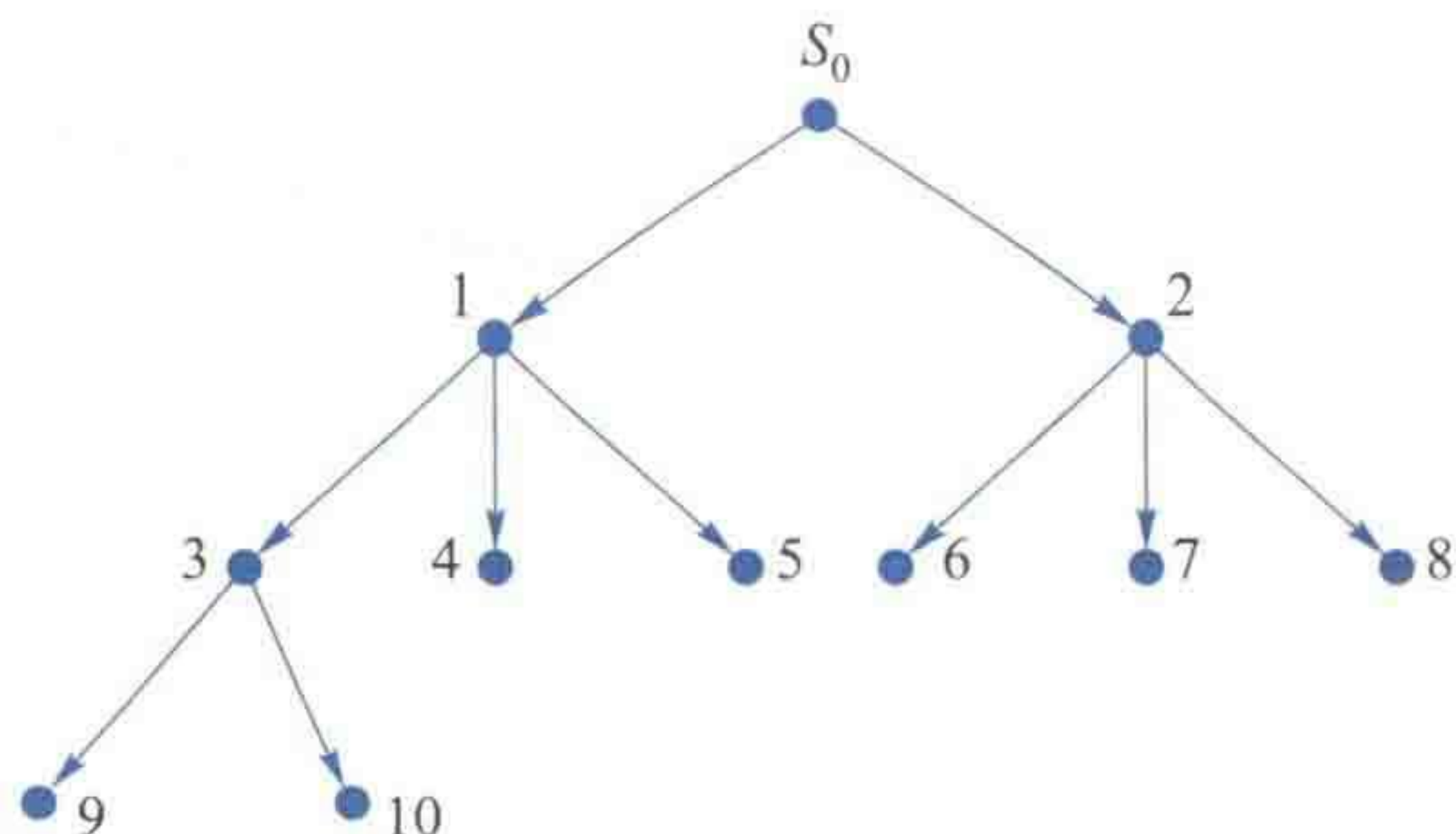


图 5.8 宽度优先搜索策略中状态的搜索次序

在实际宽度优先搜索时,为了保存状态空间搜索的轨迹,用到了两个表:open 表和 closed 表。open 表与回溯算法中的 NPS 表相似,包含了已经生成出来但其子状态未被搜索的状态。open 表中状态的排列次序就是搜索的次序。closed 表记录了已被生成扩展过的状态,它相当于回溯算法中 PS 表和 NSS 表的合并。下面是宽度优先搜索过程。

Procedure breadth_first_search

begin

open := [start]; closed := []

* 初始化

while open ≠ [] do

begin

从 open 表中删除第一个状态,称之为 n;

将 n 放入 closed 表中;

if n = 目的状态 then return (success);

生成 n 的所有子状态;

从 n 的子状态中删除已在 open 或 closed 表中出现过的状态; * 避免循环搜索

将 n 的其余子状态,按生成的次序加入 open 表的后段;

end;

end;

注意,open 表是一个队列结构,即先进先出(FIFO)的数据结构。曾在 open 表或 closed 表中出现过的子状态要删去。

如果过程因 while 循环条件(open ≠ [])不满足而结束,则表明已搜索完整个状态空间但未搜索到目的状态,说明搜索失败了。

如果整个状态空间是无限的并不能满足 while 的循环条件即无解,过程便会一直搜索下去,所以在过程中应增加“搜索超时而结束”的终止部分。

下面举一个宽度优先搜索的例子。

例 5.4 如图 5.9 所示,通过搬动积木块,希望从初始状态达到一个目的状态,即三块积木堆叠在一起。积木 A 在顶部,积木 B 在中间,而积木 C 在底部。

这个问题的唯一操作算子为 $MOVE(X, Y)$, 即把积木 X 搬到 Y (积木或桌面) 上面。如“搬动积木 A 到桌面上”表示为 $MOVE(A, Table)$ 。该操作算子可运用的先决条件是:

- ① 被搬动积木的顶部必须为空;
- ② 如果 Y 是积木(不是桌面), 则积木 Y 的顶部也必须为空;
- ③ 同一状态下, 运用操作算子的次数不得多于一次(可从 open 表和 closed 表加以检查)。

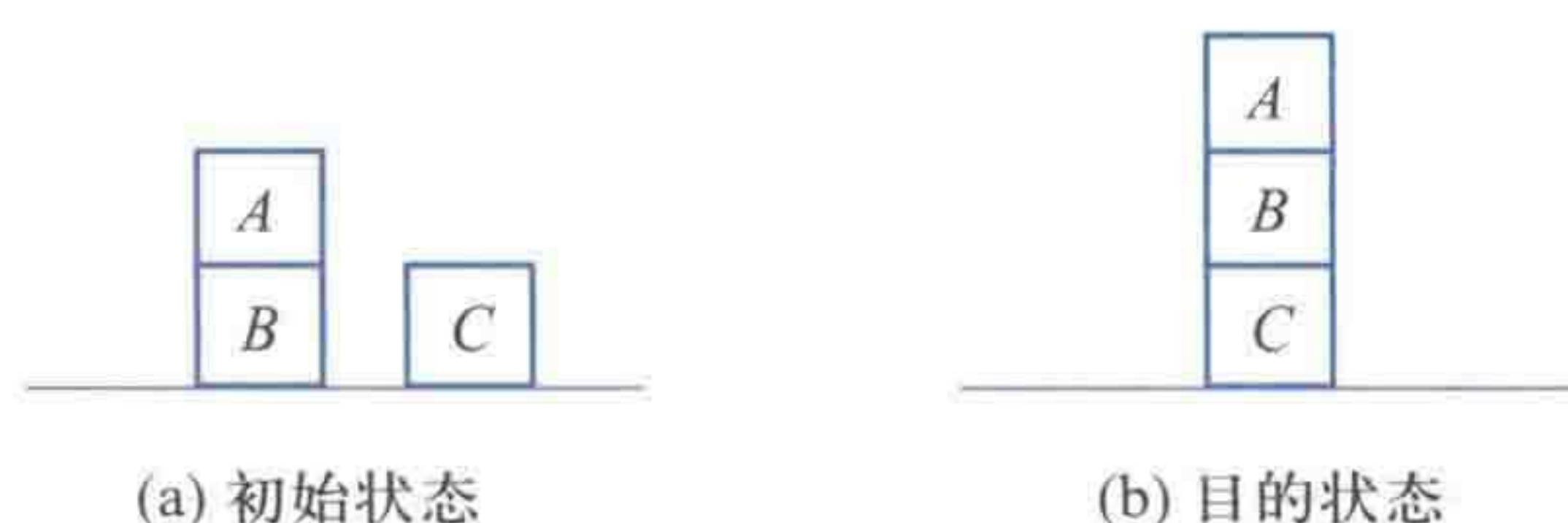


图 5.9 积木问题

图 5.10 表示了由宽度优先搜索所产生的搜索树。各结点是以产生和扩展的先后次序编下标的。当搜索到 S_{10} 目的状态时,过程便结束。此时, open 表包含 S_6 至 S_{10} , 而 closed 表包含 S_0 至 S_5 。扩展结点数为 6 个,生成结点数 10 个。

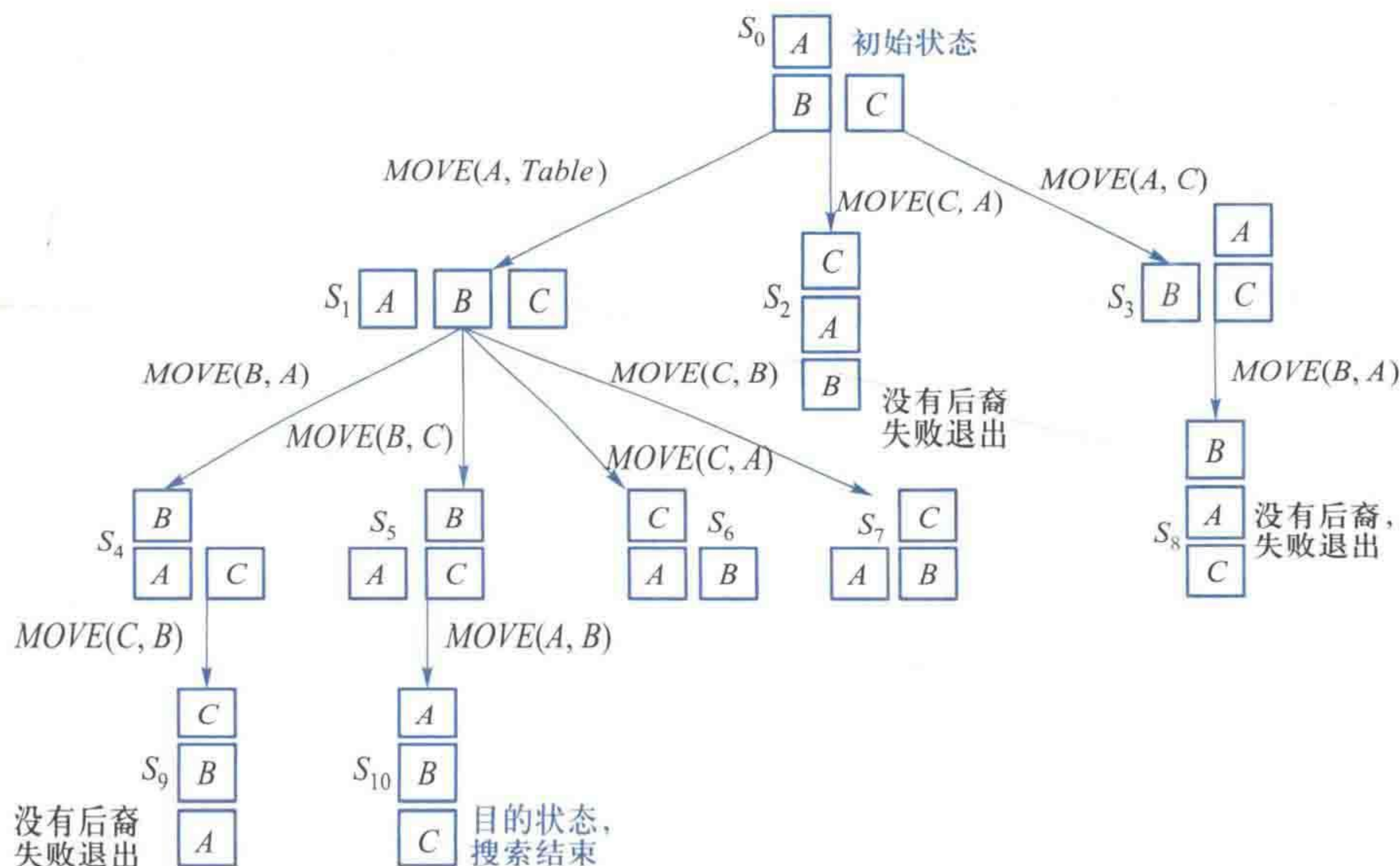


图 5.10 积木问题的宽度优先搜索树

由于宽度优先搜索总是在生成扩展完 N 层的所有结点之后才转向 $N+1$ 层,所以它总能找到最好的解(如有解),但当图的分支数太多,即状态的后裔数的平均值较大,这种组合爆炸就会使算法耗尽资源,从而在可利用的空间中找不到解。这是由于每层搜索中所有生成的未扩展的结点都要保存到 open 表中,如果解题路径较长,这个数目将会大得使搜索无法进行。

5.3.3 深度优先搜索策略

深度优先搜索策略(depth-first search, DFS)是按图 5.11 所示的次序来搜索状态的。它类似树的先根遍历,是树的先根遍历的推广。

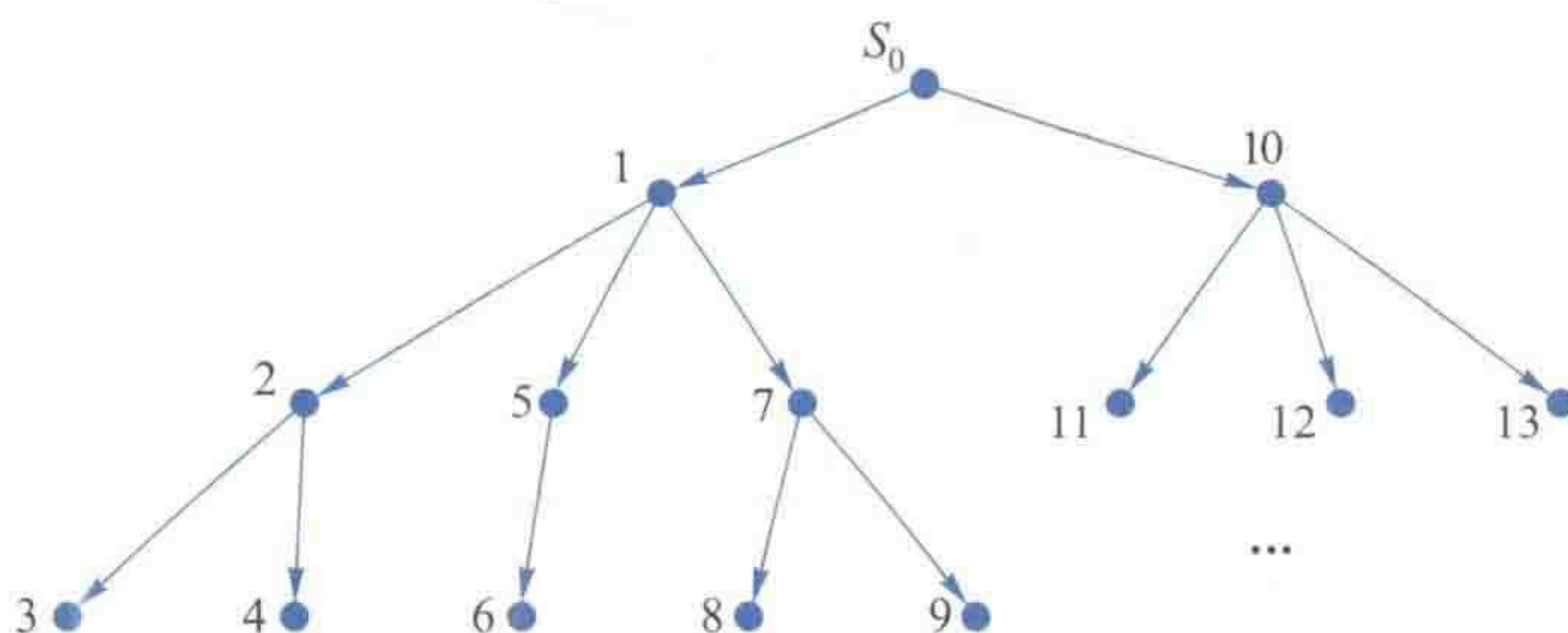


图 5.11 深度优先搜索策略中状态的搜索次序

搜索从 S_0 出发,沿一个方向一直扩展下去,如状态 $1 \rightarrow 2 \rightarrow 3$,直到达到一定的深度(这里假定为 3 层)。如果未找到目的状态或无法再扩展时,便回溯到另一条路径(状态 4)继续搜索;若还未找到目的状态或无法再扩展时,再回溯到另一条路径(状态 $5 \rightarrow 6$)搜索……

在深度优先搜索中,当搜索到某一个状态时,它所有的子状态以及子状态的后裔状态都必须先于该状态的兄弟状态被搜索。深度优先搜索在搜索空间时应尽量往深处去,只有再也找不出某状态的后裔状态时,才能考虑它的兄弟状态。

很明显,深度优先搜索法不一定能找到最优解,并且可能由于深度的限制,会找不到解(待求问题存在着解),然而,如果不加深度限制值,则可能会沿着一条路径无限地扩展下去,这当然是不希望的。为了保证找到解,那就应选择合适的深度限制值,或采取不断加大深度限制值的办法,反复搜索,直到找到解。

修改宽度优先搜索过程便可得到深度优先搜索过程:

```

Procedure depth_first_search
begin
  open := [ start ]; closed := [ ]; d := 深度限制值
  while open ≠ [ ] do
    begin
      从 open 表中删除第一个状态,称之为 n;
      将 n 放入 closed 表中;
      if n = 目的状态 then return ( success );
    end
  end

```

```
if n 的深度<d then continue;  
生成 n 的所有子状态;  
从 n 的子状态中删除已在 open 或 closed 表中出现的状态;  
将 n 的其余子状态,按生成的次序加入 open 表的前端;  
end;  
end;
```

注意,open 表是一个堆栈结构,即先进后出(FILO)的数据结构。open 表用堆栈实现的方法使得搜索偏向于最后生成的状态;曾在 open 表或 closed 表中出现过的子状态要删去。和 breadth_first_search 中一样,此处 open 表列出了所有已生成但未做扩展的状态(搜索的“前锋”),closed 表记录了已扩展过的状态。同 breadth_first_search 一样,两个算法都可以把每个结点同它的父结点一起保存,以便构造一条从起始状态到目的状态的路径。

与宽度优先搜索不同的是,深度优先搜索并不能保证第一次搜索到的某个状态时的路径是到这个状态的最短路径。对任何状态而言,以后的搜索有可能找到另一条通向它的路径。如果路径的长度对解题很关键的话,当算法多次搜索到同一个状态时,它应该保留最短路径。具体可把每个状态用一个三元组来保存(状态,父状态,路径长度)。当生成子状态时,将路径长度加 1,和子状态一起保存起来。当有多条路径可到达某子状态时,这些信息可帮助选择最优的路径。必须指出,深度优先搜索中,即使保存这些信息也不能保证算法能得到的解题路径是最优的。

下面举一个深度优先搜索的例子。

例 5.5 卒子穿阵问题。要求一卒子从顶部通过图 5.12 所示的阵列到达底部。卒子行进中不可进入代表敌兵驻守的区域(标注 1),且不准后退。假定深度限制值为 5。

行	1	2	3	4	列
1	1	0	0	0	
2	0	0	1	0	
3	0	1	0	0	
4	1	0	0	0	

图 5.12 阵列图

由深度优先搜索策略产生的搜索树如图 5.13 所示。在结点 S_0 ,卒子还没有进入阵列,在其他结点,其所处的阵列位置用一对数字(行号,列号)表示,结点的编号代表搜索的次序。

当搜索过程终止时,open 表含有结点 S_{17} (为一目的结点)和 S_{18} ,而其他结点($S_0 \sim S_{16}$)都在 closed 表中。很明显,所求得的解路($S_0 \rightarrow S_8 \rightarrow S_{14} \rightarrow S_{15} \rightarrow S_{16} \rightarrow S_{17}$)比最优路径[从(1,4)进入]多走一步。

此外,由于本算法把状态空间作为搜索树来考虑,而不是当作一般的搜索图考虑,所以,忽略了两个不同的结点 S_2 和 S_9 实际上是代表了同一个状态这个问题。因而,从结点 S_9 向下的

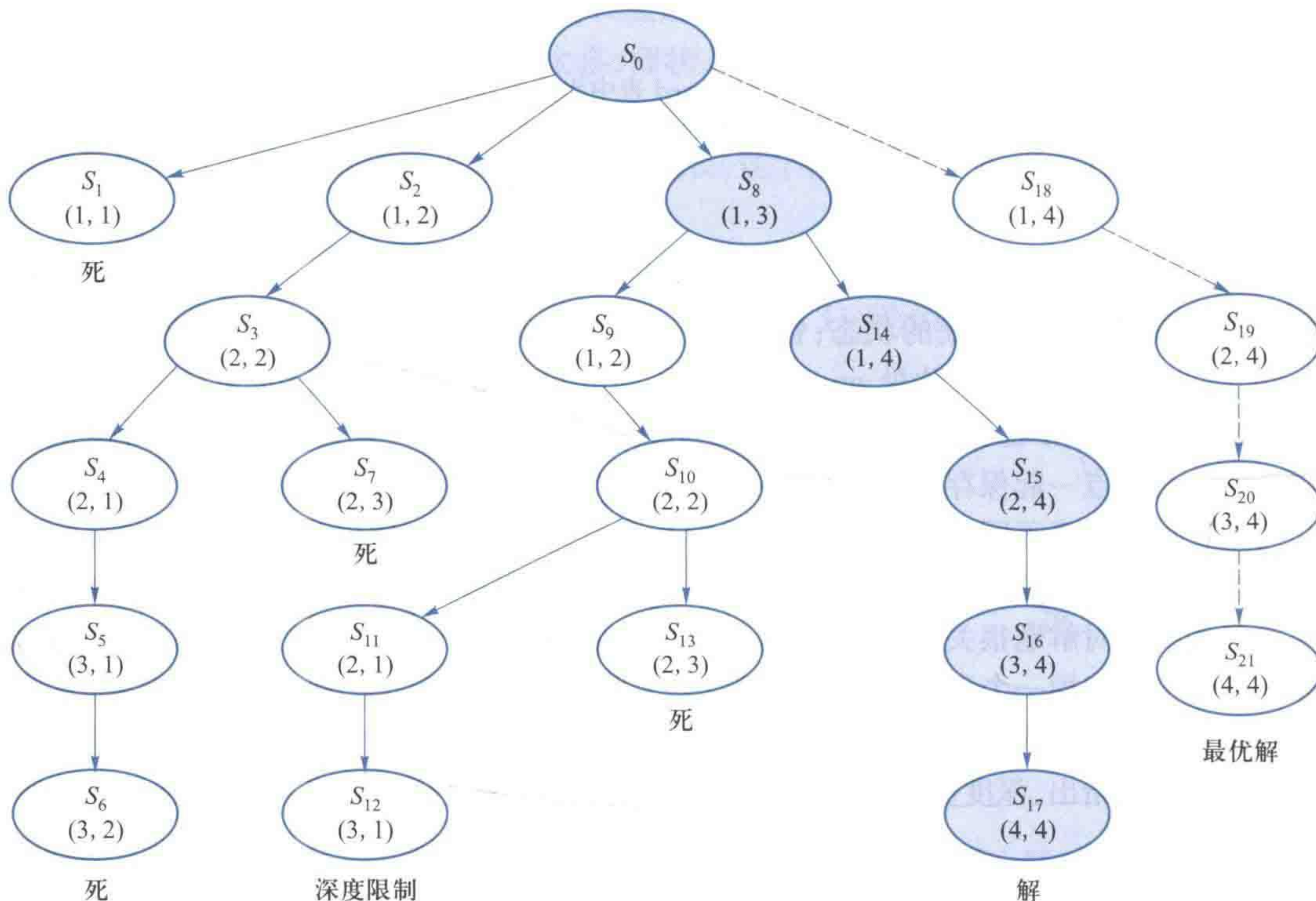


图 5.13 卒子穿阵的深度优先搜索树

搜索实际是在重复从结点 S_2 向下的搜索,但如果沿着 $S_0 \rightarrow S_8 \rightarrow S_9 \rightarrow S_{10} \rightarrow S_{11} \rightarrow S_{12}$ 这条路径时,搜索到 S_{12} 就遇到了“深度限制”。

深度优先搜索能尽快地深入下去,如果已知解题路径很长,深度搜索就不会在开始状态的周围即“浅层”状态上浪费时间;但另一方面,深度优先搜索会在搜索的深处“迷失方向”,找不到通向目的的更短路径或陷入一个不通往目的的无限长的路径中。深度优先搜索在搜索有大量分支的状态空间时有相当高的效率,它不需要把某一层上的所有结点都进行扩展。

5.4 启发式图搜索策略

前节所介绍的大部分搜索方法都是盲目搜索方法,其搜索的复杂性往往是很高的。为了提高算法的效率,必须放弃利用纯数学的方法来决定搜索结点的次序,而需要对具体问题做具体分析,利用与问题有关的信息,从中得到启发来引导搜索,以达到减少搜索量的目的,这就是启发式搜索。

本节先对启发及启发式策略所涉及的问题作一简要介绍,然后具体介绍启发式图搜索算法——A 及 A* 算法,最后讨论启发式搜索算法的性质。

5.4.1 启发式策略

启发式(heuristic)策略就是利用与问题有关的启发信息引导搜索。

在状态空间搜索中,启发式被定义成一系列操作算子,并能从状态空间中选最最有希望到达问题解的路径。

问题求解系统可在两种基本情况下运用启发式策略。

① 由于在问题陈述和数据获取方面存在模糊性,可能会使一个问题没有一个确定的解,这就要求系统能运用启发式策略做出最有可能的解释。

② 虽然一个问题可能有确定解,但是其状态空间特别大,搜索中生成扩展的状态数会随着搜索的深度呈指数级增长。穷尽式搜索策略如宽度优先或深度优先搜索,在一个给定的较实际的时空内很可能得不到最终的解,而启发式策略则通过引导搜索向最有希望的方向进行来降低搜索复杂度。

但是,启发式策略也是极易出错的。在解决问题过程中启发仅仅是下一步将要采取措施的一个猜想。它常常根据经验和直觉来判断。由于启发式搜索只利用特定问题的有限的信息,很难准确地预测下一步在状态空间中采取的具体的搜索行为。一个启发式搜索可能得到一个次优解,也可能一无所获。这是启发式搜索固有的局限性,而这种局限性不可能由所谓更好的启发式策略或更有效的搜索算法来彻底消除。

在问题求解中,需要用启发式知识来剪枝以减少状态空间,否则只能求解一些小规模的问题。因此,启发式策略及算法设计一直是人工智能的核心问题。

启发式搜索通常由两部分组成:启发方法和使用该方法搜索状态空间的算法。

例 5.6 一字棋游戏。在九宫棋盘上,从空棋盘开始,双方轮流在棋盘上摆各自的棋子×或○(每次一枚),谁先取得三子一线(一行、一列或一条对角线)的结果就取胜。

×和○在棋盘中摆成的各种不同的棋局就是问题空间中的不同状态。在9个位置上摆放{空,×,○}有 3^9 种棋局。当然,其中大多数不会在实际对局中出现。任一方的摆棋就是状态空间中的一条弧。由于第三层及更底层的某些状态可以通过不同路径到达,所以其状态空间是图而不是树。但图中不会出现回路,因为弧具有方向性(下棋时不允许悔棋)。这样搜索路径时就不必检测是否有回路。

在一字棋游戏中,第一步有9个空格便有9种可能的走法,第二步8种,第三步7种……如此递减,所以共有 $9 \times 8 \times 7 \times \cdots \times 1$ 即 $9!$ 种不同的棋局状态,其状态空间较大,穷尽搜索的组合数较大。

可以利用启发方法来剪枝以减小状态空间的大小。根据棋盘的对称性可以减少搜索空间的大小。棋盘上很多棋局是等价的,如第一步实际上只有3种走法,角、边的中央和棋盘正中,这时状态空间的大小为 $3 \times 8!$ 。在状态空间的第二层上,由对称性还可进一步减少到 $3 \times 2 \times 7!$ 种,当然还可以再进一步减少状态空间的大小。

此外,使用启发方法进行搜索可以简化搜索过程。最初的三种状态显示在图5.14中。这

样,可设计一种算法(完全实现启发式搜索),它选择具有最高启发值的棋局状态放棋子。例如,对于图 5.14(a) $\times$ 方有 8 种布子成为一线,而 O 方只有 5 种布子成为一线,所以 $\times$ 方赢的概率 $8-5=3$; 对于图 5.14(b) O 方有 6 种布子成为一线,所以 $\times$ 方赢的概率为 $8-6=2$; 对于图 5.14(c) O 方有 4 种布子成为一线,所以 $\times$ 方赢的概率为 $8-4=4$,是 $\times$ 方的最佳走步。因此,在本例的这种情况下,只需搜索 $\times$ 占据棋盘正中位置的棋局状态,而其他的各种棋局状态连同它们的延伸棋局状态都不必再考虑了。如图 5.15 所示 $2/3$ 的状态空间就不必搜索了。

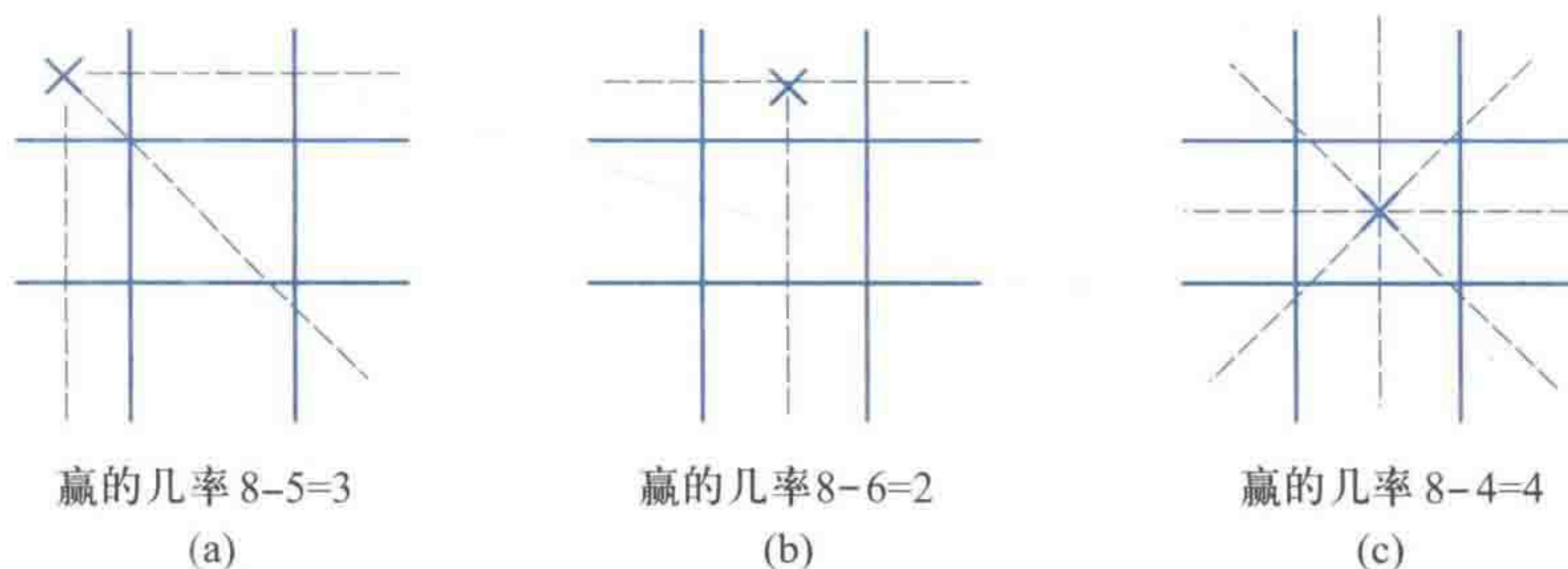


图 5.14 启发式策略的运用

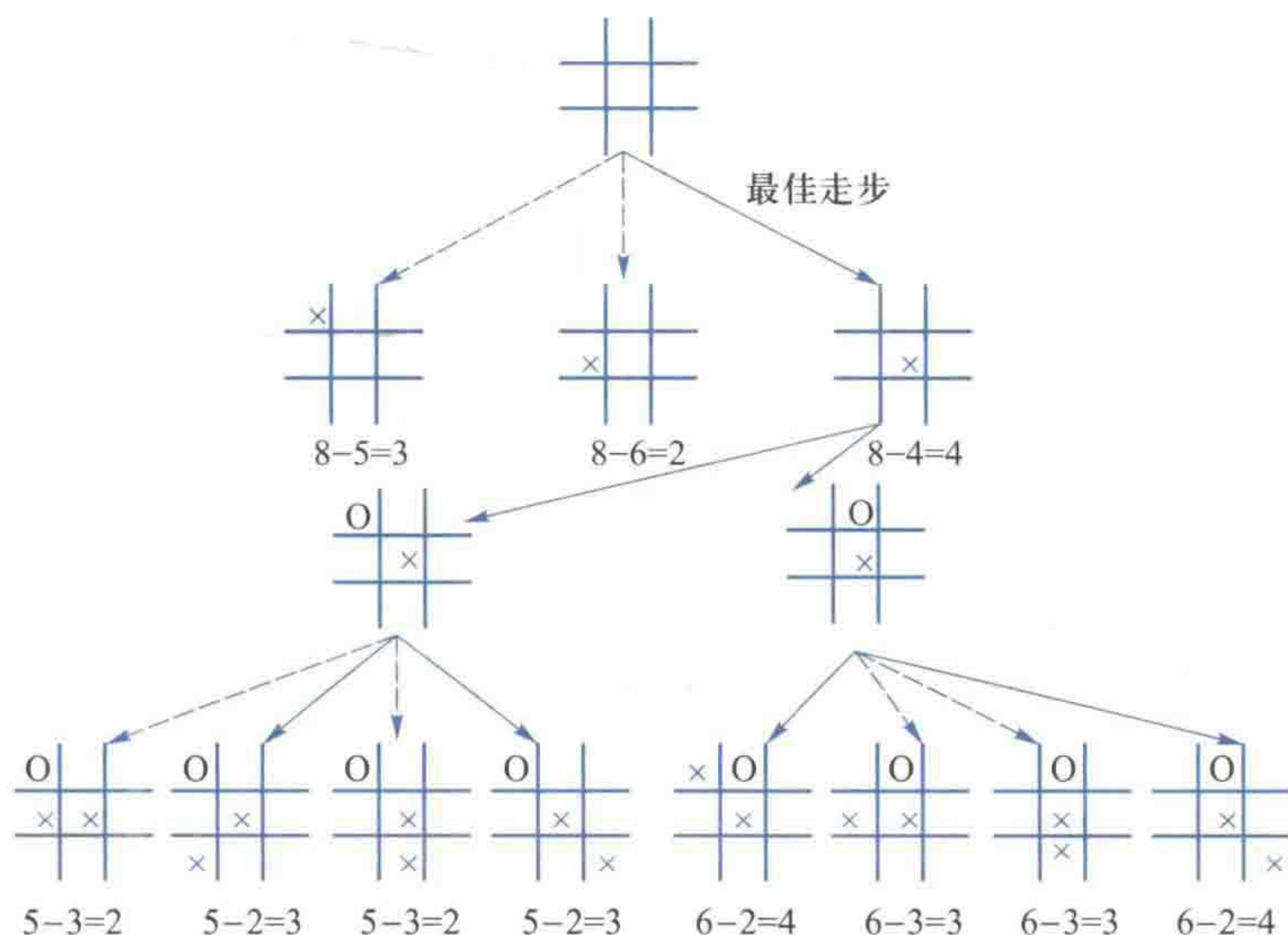


图 5.15 启发式搜索下缩减的状态空间

第一步棋下完后,对方只能有两种走法。无论选择哪种走法,我方均可以通过启发式搜索来选择下一步可能的走法。在搜索过程中,每一步只需估计单个结点的子结点便可决定下哪步棋。图 5.15 显示了游戏前三步简化了的搜索过程。每种状态都标记了它的启发值,图中实线表示最佳走步。

要精确地计算待搜索的状态数目比较难,但可以大致计算它的上限。一盘棋最多走 9 步,

每步的下一步平均有 4、5 种走法,这样大约就是 4.5×9 ,近 40 种状态,这比原来 $9!$ 大小的状态空间缩小了很多。

国际象棋软件采用启发式搜索算法,在搜索棋局时加入剪枝策略。AlphaGo 就是利用深度学习算法学习人类的棋谱,模拟人类选择几个优势点然后通过蒙特卡洛树搜索,穷举计算这几个点胜率,从中优选。AlphaGo 中有两个深度神经网络,价值网络(value networks)和策略网络(policy networks)。其中,价值网络评估棋盘选点位置,策略网络选择落子。深度神经网络不仅向人类专家学习棋谱,而且能够自己和自己下棋(self-play),进行强化学习,不断提高下棋水平。

5.4.2 启发信息和估价函数

在实际解决一个具体问题时,人们常常把一个复杂的实际问题抽象化,保留某些主要因素,忽略大量次要因素,从而将这个实际问题转化成具有明确结构的有限或无限的状态空间问题,这个状态空间中的状态和变换规律都是已知的集合,因此可以找到一个求解该问题的算法。

在具体求解中,启发式搜索能够利用与该问题有关的信息来简化搜索过程,称此类信息为启发信息。然而,在求解问题中能利用的大多不是具有完备性的启发信息,而是非完备的启发信息,其原因如下。

① 大多数情况下,求解问题系统不可能知道与实际问题有关的全部信息,因而无法知道该问题的全部状态空间,也不可能用一套算法来求解所有的问题。这样就只能依靠部分状态空间、一些特殊的经验和有关信息来求解其中的部分问题。

② 有些问题在理论上虽然存在着求解算法,但是在工程实践中,这些算法不是效率太低,就是无法实现。为了提高求解问题的效率,不得不放弃使用这些“完美的”算法,而求助于一些启发信息来进行启发式搜索。

如在博弈问题中,计算机为了保证最后胜利,可以将所有可能的走法都试一遍,然后选择最佳走步。这样的算法是可以找到的,但计算所需的时空代价十分惊人。对于可能有的棋局数来说,一字棋是 $9! \approx 3.6 \times 10^5$,西洋跳棋是 10^{78} ,国际象棋是 10^{120} ,围棋是 10^{761} 。对于围棋,宋代《梦溪笔谈》里早有论述:“大约连书万字五十二个,即是局之大数。”意思是围棋所有可能的走法数目超过 52 位。扣除气孔等必需的空位后,仍然有 43 位数的可能性。地球上气体的分子总数也大约有 46 位数。假设每步可以搜索一个棋局,用极限并行速度(10^{-104} 年/步)来处理,搜索一遍国际象棋的全部棋局也得 10^{16} 年即 1 亿亿年才可以算完,而已知的宇宙寿命才 100 亿年!因此,必须采用启发式的求解方法。

启发信息按运用的方法不同可分为三种。

① 陈述性启发信息。一般被用于更准确、更精炼地描述状态,使问题的状态空间缩小,如待求问题的特定状况等属于此类信息。

② 过程性启发信息。一般被用于构造操作算子,使操作算子少而精,如一些规律性知识等属于此类信息。

③ 控制性启发信息。它是表示控制策略方面的知识,包括协调整整个问题求解过程中所使用的各种处理方法、搜索策略、控制结构等有关的知识。

为提高搜索效率就需要利用上述三种启发信息作为搜索的辅助性策略。这里主要介绍控制性的启发信息。

利用控制性的启发信息有两种极端的情况:一种是没有任何控制性知识作为搜索的依据,因而搜索的每一步完全是随意的,如随机搜索、宽度搜索、深度搜索等;另一种是有充分的控制知识作为依据,因而搜索的每一步选择都是正确的,但这是不现实的。一般情况介于二者之间。在搜索过程中需要根据这些启发信息估计各个结点的重要性。

用估价函数(evaluation function)估计待搜索结点的“有希望”程度,并依次给它们排定次序。估价函数 $f(x)$ 可以是任意一种函数,如定义为结点 x 处于最佳路径上的概率,或是 x 结点和目的结点之间的距离或差异,或是 x 格局的得分等。

一般来说,估计一个结点的价值,必须综合考虑两方面的因素:已经付出的代价和将要付出的代价。因此,估价函数 $f(n)$ 定义为从初始结点经过 n 结点到达目的结点的路径的最小代价估计值,其一般形式是

$$f(n) = g(n) + h(n) \quad (5.3)$$

其中, $g(n)$ 是从初始结点到 n 结点的实际代价; $h(n)$ 是从 n 结点到目的结点的最佳路径的估计代价,称为启发函数。因为实际代价 $g(n)$ 可以根据已生成的搜索树实际计算出来,而启发函数 $h(n)$ 是对未生成的搜索路径作某种经验性的估计。这种估计来源于对问题解的某些特性的认识,希望依靠这些特性来更快地找到问题的解,因此,主要是 $h(n)$ 体现了搜索的启发信息。

$g(n)$ 的作用一般是不可忽略的。因为它代表了从初始结点经过 n 结点到达目的结点的总代价估值中实际已付出的那一部分。 $g(n)$ 保持了搜索的宽度优先成分, $g(n)$ 的比重越大,越倾向于宽度优先搜索方式。这有利于搜索的完备性,但会影响搜索的效率。 $h(n)$ 的比重越大,表示启发性能越强。在特殊情况下,如果只希望找到达到目的结点的路径而不关心会付出什么代价,则 $g(n)$ 的作用可以忽略。另外,当 $h(n) \gg g(n)$ 时,也可忽略 $g(n)$,这时有 $f(n) = h(n)$,有利于提高搜索的效率,但影响搜索的完备性。

给定一个问题后,根据该问题的特性和解的特性,可以有多种方法定义估价函数,用不同的估价函数指导搜索,其效果相差很远。因此,尽可能选择最能体现问题特性的估价函数。

例 5.7 八数码问题的启发函数。

八数码问题的启发函数的设计方法有多种,并且不同的启发函数对求解八数码问题有不同的影响。

① 启发函数是一格局与目的格局位置不符的数码数目。例如,图 5.2 的初始状态的启发函数为 7。这是最简单的设计方法。直观感觉这种函数很有效,因为在其他条件相同的情况下,某格局位置不符的数码数目越少,则它和最终目的越接近,因而它是下一个搜索格局。但是,这种函数并没有充分利用所能获得的信息。它没有考虑数码所需移动的距离。

② 启发函数是各数码移到目的位置所需移动的距离的总和。例如,图 5.2 的初始状态的启发函数为 14。

这两种启发函数都没有考虑数码逆转(与目的格局中数码排列的先后顺序)的情况。如果两块数码相邻但与目标格局相比位置相反,则至少需移动 3 次才能将它们移到正确的位置上。

③ 启发函数对每一对逆转数码乘以一个倍数。例如,图 5.2(a)的数码逆转次数为 9。

④ 启发函数将位置不符数码数目的总和与 3 倍数码逆转数目相加。它克服了仅计算数码逆转数目策略的局限。

这个例子说明,设计一个好的启发函数具有相当的难度。好的启发函数的设计是一个经验问题,判断和直觉是很重要的因素,但是衡量其好坏的最终标准是在具体应用时的搜索效果。

5.4.3 A 及 A* 搜索算法

启发式图搜索法的关键是如何寻找并设计一个与问题有关的 $h(n)$ 及构造出 $f(n) = g(n) + h(n)$, 然后以 $f(n)$ 的大小来排列待扩展状态的次序,每次选择 $f(n)$ 值最小者进行扩展。

与宽度优先和深度优先搜索算法一样,启发式图搜索法算法使用两张表记录状态信息:在 open 表中保留所有已生成而未扩展的状态;在 closed 表中记录已扩展过的状态。算法中有一步是根据某些启发信息来排列 open 表的。它既不同于宽度优先所使用的队列(先进先出),也不同于深度优先所使用的堆栈(先进后出),而是一个按状态的启发估价函数值的大小排列的一个表。进入 open 表的状态不是简单地排在队尾(或队首),而是根据其估值的大小插入到表中合适的位置,每次从表中优先取出启发估价函数值最小的状态加以扩展。

A 算法是基于估价函数的一种加权启发式图搜索算法,具体步骤如下:

步骤 1 把附有 $f(S_0)$ 的初始结点 S_0 放入 open 表;

步骤 2 若 open 表为空,则搜索失败,退出;

步骤 3 移出 open 表中第一个结点 N 放入 closed 表中,并顺序编号 n ;

步骤 4 若目标结点把附有 $f(S_0)$ 的初始 $S_g = N$, 则搜索成功,结束;

步骤 5 若 N 不可扩展,则转步骤 2;

步骤 6 扩展 N ,生成一组附有 $f(x)$ 的子结点,对这组子结点做如下处理。

① 考察是否有已在 open 表或 closed 表中存在的结点。若有则再考察其中有无 N 的先辈结点,若有则删除之,对于其余结点也删除之,但由于它们又被第二次生成,因此需要考虑是否修改已经存在于 open 表或 closed 表中的这些结点及其后裔的返回指针和 $f(x)$ 的值。修改原则是:选 $f(x)$ 值小的路径走。

② 为其余子结点配上指向 N 的返回指针后放入 open 表中,并对 open 表按 $f(x)$ 值以升序排序,转步骤 2。

启发式图搜索的 A 算法描述如下:

```
procedure heuristic_search
```

```
  open := [ start ]; closed := [ ]; f(s) := g(s) + h(s);
```

* 初始化

```
  while open ≠ [ ] do
```

```
  begin
```

```
    从 open 表中删除第一个状态,称之为 n;
```

```
    if n = 目的状态 then return( success );
```

```
    生成 n 的所有子状态;
```

```
    if n 没有任何子状态 then continue;
```

```
    for n 的每个子状态 do
```

```
      case 子状态 is not already on open 表 or closed 表;
```

```
        begin
```

```
          计算该子状态的估价函数值;
```

```
          将该子状态加到 open 表中;
```

```
        end;
```

```
      case 子状态 is already on open 表:
```

```
        if 该子状态是沿着一条比在 open 表已有的更短路径而到达
```

```
          then 记录更短路径走向及其估价函数值;
```

```
      case 子状态 is already on closed 表:
```

```
        if 该子状态是沿着一条比在 closed 表已有的更短路径而到达
```

```
          then
```

```
            begin
```

```
              将该子状态从 closed 表移到 open 表中;
```

```
              记录更短路径走向及其估价函数值;
```

```
            end;
```

```
      case end;
```

```
      将 n 放入 closed 表中;
```

```
      根据估价函数值,从小到大重新排列 open 表;
```

```
    end;
```

* open 表中结点已耗尽

```
  return( failure );
```

从上面的描述可见,在 A 搜索算法中,从 open 表中取出第一个状态,如果该状态满足目的条件,则算法返回到该状态的搜索路径,在这里,每个状态都保留了其父状态的信息,以保证能返回完整的搜索路径。如果 open 表的第一个状态不是目的状态,则算法利用与之相匹配的一系列操作算子进行相应的操作来产生它的子状态。如果某个子状态已在 open 表(或 closed 表)中出现过,即该状态再一次被发现时,则通过刷新它的祖先状态的历史记录,使算法找到到达目的状态更短路径的概率大大提高。接着,用 A 搜索算法 open 表中每个状态的估价函数值,按照值的大小重新排序,将值最小的状态放在表头,使其第一个被扩展。

启发信息给得越多即估价函数值越大,则 A 算法需要搜索处理的状态数就越少,其效率就越高。但也不是估价函数值越大越好,因为估价函数值太大会使 A 算法不一定能搜索到最优解。

A* 搜索算法是由著名的人工智能学者 Nilsson 提出的,它是目前最有影响的启发式图搜索

算法,也称为最佳图搜索算法。

定义 $h^*(n)$ 为状态 n 到目的状态的最优路径的代价,则当 A 搜索算法的启发函数 $h(n)$ 小于等于 $h^*(n)$,即满足

$$h(n) \leq h^*(n), \quad \text{对所有结点 } n \quad (5.4)$$

时,则称为 A* 搜索算法。

如果某一问题有解,那么利用 A* 搜索算法对该问题进行搜索则一定能搜索到解,并且一定能搜索到最优的解。因此,A* 搜索算法比 A 搜索算法好。

例 5.8 图 5.16 给出了利用 A 搜索算法求解八数码问题的搜索树,解的路径为 $s \rightarrow B \rightarrow E \rightarrow I \rightarrow K \rightarrow L$ 。

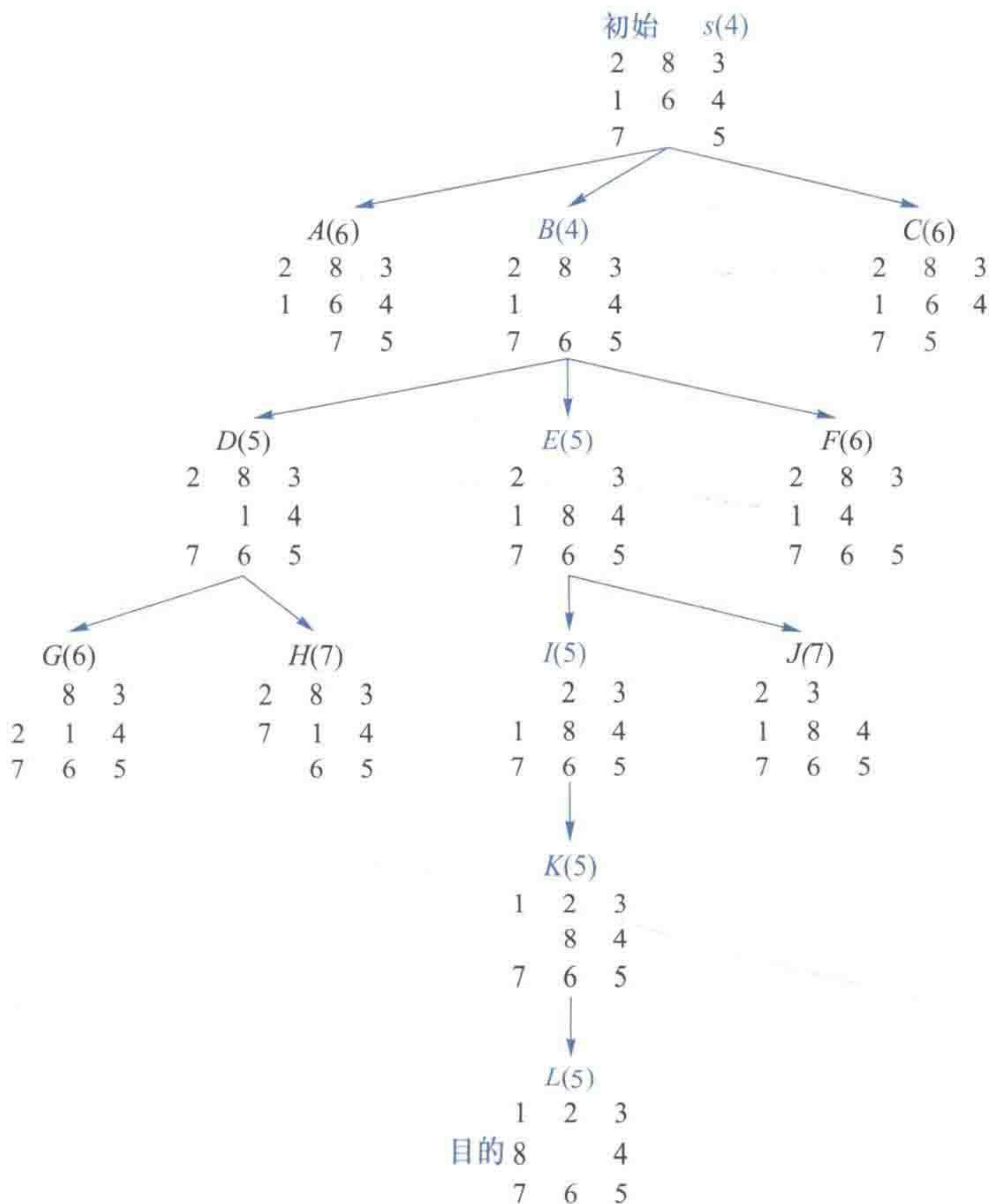


图 5.16 八数码问题的 A 搜索树

图 5.16 中状态旁括号内的数字表示该状态的估价函数值,其估价函数定义为

$$f(n) = d(n) + w(n) \quad (5.5)$$

其中, $d(n)$ 代表状态的深度,每步为单位代价; $w(n)$ 表示以“不在位”的数码数作为启发信息的

度量。例如, A 的状态深度为 1, 不在位的数码数为 5, 所以 A 的启发函数值为 6。又如, E 的状态深度为 2, 不在位的数码数为 3, 所以 E 的启发函数值为 5。

搜索过程中 open 表和 closed 表内状态排列的变化情况如表 5.1 所示。

表 5.1 搜索过程中 open 表和 closed 表内状态排列的变化情况

open 表	closed 表
初始化: $(s(4))$	$()$
一次循环后:	
$(B(4) A(6) C(6))$	$(s(4))$
二次循环后:	
$(D(5) E(5) A(6) C(6) F(6))$	$(s(4) B(4))$
三次循环后:	
$(E(5) A(6) C(6) F(6) G(6) H(7))$	$(s(4) B(4) D(5))$
四次循环后:	
$(I(5) A(6) C(6) F(6) G(6) H(7) J(7))$	$(s(4) B(4) D(5) E(5))$
五次循环后:	
$(K(5) A(6) C(6) F(6) G(6) H(7) J(7))$	$(s(4) B(4) D(5) E(5) I(5))$
六次循环后:	
$(L(5) A(6) C(6) F(6) G(6) H(7) J(7) M(7))$	$(s(4) B(4) D(5) E(5) I(5) K(5))$
七次循环后:	
L 为目的状态, 则成功退出, 结束搜索	$(s(4) B(4) D(5) E(5) I(5) K(5) L(5))$

在例 5.8 所描述的八数码问题中的 $w(n)$ 即为 $h(n)$, 它表示了“不在位”的数码数。这个 $w(n)$ 满足了 $h(n) \leq h^*(n)$ 的条件, 因此, 图 5.16 的八数码 A 搜索树也是 A^* 搜索树, 所得的解路 $(s \rightarrow B \rightarrow E \rightarrow I \rightarrow K \rightarrow L)$ 为最优解路, 其步数为状态 $L(5)$ 上所标注的 5, 因为这时不在位的数码数为 0。

宽度优先算法是 A^* 的一个特例, 该算法相当于 A^* 算法中取 $h(n) = 0$ 和 $f(n) = g(n) + 0$ 。宽度优先搜索时对某一状态只考虑它同起始状态的距离代价。这是由于该算法在考虑 $n+1$ 层状态之前, 已考察了 n 层中的任意一种状态, 所以每个目的状态都是沿着最短的可能路径而找到的, 但宽度优先搜索算法的搜索效率太低。

5.4.4 蒙特卡洛树搜索算法

人们下棋时, 每走一步前不可能在脑子里把所有可能的棋步都列出来, 而是根据“棋感”大

致筛选出了几种“最可能”的走法,然后再比较选出自己感觉“最好”的走法。这就是蒙特卡洛树搜索算法的设计思路。

蒙特卡洛树搜索(Monte Carlo tree search, MCTS)算法是 Remi Coulom 于 2006 年在围棋人机对战引擎 Crazy Stone 中首次发明并使用的。蒙特卡洛树搜索是在完美信息博弈场景中进行决策的一种通用技术,除游戏之外,它还在很多现实世界的应用中有着广阔前景。

蒙特卡洛树搜索算法是一种基于树数据结构的启发式搜索算法,是一类树搜索算法的统称。MCTS 并不是一种“模拟人”的算法,而是一种博弈树搜索方法,是通过随机对游戏进行推演来逐渐建立一棵不对称搜索树的过程。

在开始阶段,搜索树只有一个结点,称为根结点。搜索树中的每一个结点包含了三个基本信息:代表的局面、被访问的次数、累计评分。

蒙特卡洛树搜索大概可以被分成 4 步:选择(selection)、拓展(expansion)、模拟(simulation)、反向传播(backpropagation)。

1. 选择

选择是指从搜索树的根结点开始,也就是要做决策的局面 R 出发,向下选择一个最急需被拓展的结点 N 。结点 R 是每一次迭代中第一个被检查到的结点。对于被检查到的结点可能有以下 3 种可能:

(1) 完全展开结点:该结点所有可行动作都已经被拓展过。那么计算该结点所有子结点的 UCT 值,并找到一个值最大的子结点继续检查。反复向下迭代。

(2) 未完全展开结点:该结点有可行动作还未被拓展过。例如有 10 个可行动作,但是在搜索树中才创建了 9 个子结点,那么就认为这个结点是本次迭代的目标结点 N ,并找出 N 还未被拓展的结点 L 执行“拓展”步骤。

(3) 未访问结点:该结点所有可行动作都未被拓展过。那么从该结点直接执行“回溯”步骤。

在反复迭代之后,将在搜索树的底端找到一个结点,来继续后面的步骤。

直观的想法:胜率大的结点是最有可能被选择的结点。但这样选择会导致一开始随机走子的时候赢了一盘,就会一直走这个结点。为了避免这些情况发生,构造一个函数

$$UCT(v_i, v) = \frac{Q(v_i)}{N(v_i)} + c \sqrt{\frac{\log N(v)}{N(v_i)}} \quad (5.6)$$

其中, $Q(v)$ 是结点 v 赢的次数; $N(v)$ 是结点 v 模拟的次数; c 是一个常数。

从根结点出发,遵循最大最小原则,每次选择己方 UCT 值最优的一个结点,向下搜索,直到找到一个“未完全展开的结点”,然后随机选一个结点进行扩展。上式第一部分是模拟的胜率;随着访问次数的增加,第二部分的值会越来越小。因此,会倾向于选择那些还没有怎么被统计过的结点。

2. 拓展

如果结点不是一个终止结点,则随机选择一个尚未拓展的结点 A ,在搜索树中创建一个新

的结点 M 。

3. 模拟

从新结点 M 出发,模拟拓展搜索树,直到找到一个终止结点。为了让新的结点 M 得到一个初始的评分,从 M 开始让游戏随机进行,直到得到一个游戏结局,这个结局将作为 M 的初始评分。一般使用胜利或者失败的次数作为评分,例如,模拟了 10 盘,赢了 4 盘,则记为 4/10。

4. 反向传播

在 M 的模拟结束后,它的父结点 N 以及从根结点到 N 的路径上所有结点都会根据本次模拟的结果来添加自己的累计评分。如果在“选择”步骤中直接发现了一个游戏结局的话,根据该结局来更新评分。

每一次迭代都会拓展搜索树,随着迭代次数的增加,搜索树的规模也不断增加。当到了一定的迭代次数或者时间之后结束,选择根结点下最好的子结点作为本次决策的结果。一般来说,最佳走法就是具有最高访问次数的结点。

对于比较复杂的搜索,如国际象棋游戏等,搜索的时间开销是巨大想,可以采用如 α - β 剪枝方法等减小搜索空间,以减小时间开销。

5.5 小 结

1. 搜索的概念

在搜索中需要解决是否一定能找到一个解、是否终止运行、找到的解是否最佳解、搜索过程的时间与空间复杂性如何等基本问题。

搜索的方向有正向搜索和逆向搜索。

盲目搜索是在不具有对特定问题的任何有关信息的条件下,按固定的步骤(依次或随机调用操作算子)进行的搜索。

启发式搜索则是考虑特定问题领域可应用的知识,动态地确定调用操作算子的步骤,优先选择较适合的操作算子。

2. 状态空间知识表示方法

状态空间是利用状态变量和操作符号,表示系统或问题的有关知识的符号体系,状态空间是一个四元组 (S, O, S_0, G) 。

任何类型的数据结构都可以用来描述状态,如符号、字符串、向量、多维数组、树和表格等。

从 S_0 结点到 G 结点的路径被称为求解路径。状态空间的一个解是一个有限的操作算子序列,它使初始状态转换为目标状态。

3. 回溯策略

带回溯策略的搜索是从初始状态出发,不停地、试探性地寻找路径,若它遇到不可解结点就回溯到路径中最近的父结点上,查看该结点是否还有其他的子结点未被扩展。若有,则沿这些子结点继续搜索;如果找到目标,就成功退出搜索,返回解题路径。

回溯是状态空间搜索的一个基本算法。各种图搜索算法,包括深度优先、宽度优先、最好优先搜索等,都有回溯的思想。

4. 宽度优先搜索法

宽度优先搜索法是由 S_0 生成新状态,然后依次扩展这些状态,再生成新状态,该层扩展完后,再进入下一层,如此一层一层地扩展下去,直到搜索到目的状态(如果目的状态存在)。

5. 深度优先搜索法

深度优先搜索法是从 S_0 出发,沿一个方向一直扩展下去,直到达到一定的深度。如果未找到目的状态或无法再扩展时,便回溯到另一条路径继续搜索;若还未找到目的状态或无法再扩展时,再回溯到另一条路径搜索……

6. 启发式图搜索策略

在具体求解中,能够利用与该问题有关的信息来简化搜索过程,称此类信息为启发信息,而称这种利用启发信息的搜索过程为启发式搜索。

A 搜索算法是寻找并设计一个与问题有关的 $h(n)$ 及构造出 $f(n) = g(n) + h(n)$,然后以 $f(n)$ 的大小来排列待扩展状态的次序,每次选择 $f(n)$ 值最小者进行扩展。

定义 $h^*(n)$ 为状态 n 到目的状态的最优路径的代价。对于一具体问题,只要有解,则一定存在 $h^*(n)$ 。当要求估价函数中的 $h(n)$ 都小于等于 $h^*(n)$ 时,A 搜索算法就成为 A^* 搜索算法。

7. 蒙特卡洛树搜索算法

蒙特卡洛树搜索算法是一种基于树数据结构的启发式搜索算法,是一类树搜索算法的统称。

蒙特卡洛树搜索大概可以被分成 4 步:选择、拓展、模拟、反向传播。

思考题

- 5.1 什么是搜索? 有哪两大类不同的搜索方法? 两者的区别是什么?
- 5.2 什么是启发式搜索? 什么是启发信息?
- 5.3 用状态空间法表示问题时,什么是问题的解? 求解过程的本质是什么? 什么是最优解? 最优解唯一吗?
- 5.4 请写出状态空间图的一般搜索过程。在搜索过程中 open 表和 closed 表的作用分别是什么? 有何区别?
- 5.5 什么是盲目搜索? 主要有几种盲目搜索策略?
- 5.6 在深度优先搜索中,每个结点的子结点是按某种次序生成和扩展的,在决定生成子状态的最优次序时,应该用什么标准来衡量?
- 5.7 宽度优先搜索与深度优先搜索有何不同? 分析深度和宽度优先的优缺点。在何种情况下,宽度优先搜索优于深度优先搜索? 在何种情况下,深度优先搜索优于宽度优先搜索?
- 5.8 什么是 A^* 搜索算法? 它的估价函数是如何确定的? A^* 搜索算法与 A 搜索算法的区别是什么?

习 题

5.1 修道士和野人的问题。设有3个修道士和3个野人来到河边,打算用一条船从河的左岸渡到河的右岸。但该船每次只能装载2个人,在任何岸边野人的数目都不得超过修道士的人数,否则修道士就会被野人吃掉。假设野人服从任何一种过河安排,如何规划过河计划才能把所有人安全地渡过河去。用状态空间表示法表示修道士和野人的问题,画出状态空间图。

5.2 用状态空间搜索法求解农夫、狐狸、鸡、小米问题。农夫、狐狸、鸡、小米都在一条河的左岸,现在要把它们全部送到右岸去,农夫有一条船,过河时,除农夫外,船上至多能载狐狸、鸡和小米中的一样。狐狸要吃鸡,鸡要吃小米,除非农夫在那里。试规划出一个确保全部安全的过河计划。(提示:a. 用四元组(农夫、狐狸、鸡、小米)表示状态,其中每个元素都可为0或1。0表示在左岸,1表示在右岸。b. 每次过河的一种安排作为一个算符,每次过河都必须有农夫,因为只有他可以划船。)

5.3 用有界深度优先搜索方法求解图5.17所示八数码难题。初始状态为 S_0 ,目标状态为 S_g ,要求寻找从初始状态到目标状态的路径。

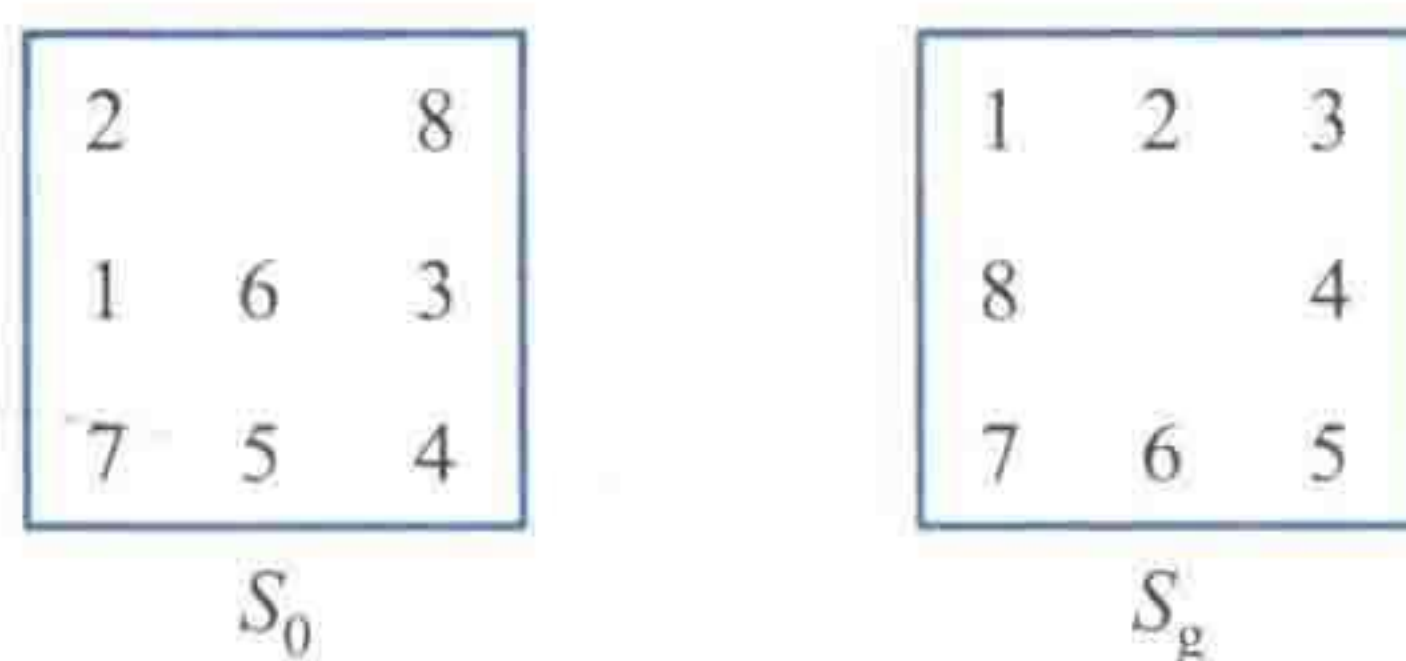


图 5.17 初始状态和目标状态

读一读

诺贝尔化学奖得主、AI 预测蛋白质结构三巨头——贝克、哈萨比斯和詹伯

2024 年 10 月 9 日,瑞典皇家科学院正式揭晓 2024 诺贝尔化学奖:AI 预测蛋白质结构,破解了蛋白质惊人结构的密码。其中,一半授予大卫·贝克(David Baker),奖励他在“计算蛋白质设计”方面的贡献;另一半共同授予戴密斯·哈萨比斯(Demis Hassabis)和约翰·詹伯(John M. Jumper),奖励他们在“蛋白质结构预测”方面的贡献。这是人工智能成果获得的第二项诺贝尔奖。

大卫·贝克 1962 年出生于美国华盛顿州西雅图,1989 年获美国加州大学伯克利分校博士学位,现为美国华盛顿大学西雅图分校教授。他成功实现了构建全新蛋白质种类这一几乎不可能的壮举,创造了第一种与所有已知现有蛋白质完全不同的蛋白质。此后,他的研究团队创造了许多新的蛋白质,包括可用于药物、疫苗、纳米材料、微小传感器等的蛋白质。

他的共同获奖者戴密斯·哈萨比斯和约翰·詹伯于 2020 年开发了一种名为 AlphaFold2 的 AI 模型,解决了一个 50 年来的难题:预测蛋白质的复杂结构。这种 AI 模型已成功预测几乎所有已知蛋白质的结构,具有巨大的潜力。在它的帮助下,他们已经能够预测已识别的所有 2 亿种蛋白质的结构。AlphaFold2 已被来自 190 个国家和地区的超过 200 万人使用。

戴密斯·哈萨比斯 1976 年出生于英国伦敦。2009 年获得英国伦敦大学博士学位,现任 Google DeepMind 首席执行官。约翰·詹伯 1985 年出生于美国阿肯色州小石城。2017 年获得美国伊利诺伊州芝加哥大学博士学位,现任 Google DeepMind 高级研究科学家。

智能计算(intelligence computational, IC)是模拟人类社会和自然界、生物界或者更高级的社会系统等外部环境的原理设计的优化方法。广义的智能计算也称为计算智能(computational intelligence, CI),主要包括神经计算、模糊计算和进化计算三大部分。本章主要介绍进化计算,包括进化算法、群智能算法等,这些技术已经广泛应用于组合优化、机器学习、智能控制、模式识别、规划设计、网络安全等领域,是 21 世纪有关智能计算中的重要技术之一。

本章首先简要介绍进化算法的概念,详细介绍基本遗传算法,这是进化算法的基本框架。然后简要介绍双倍体、双种群、自适应等比较典型的改进遗传算法及其应用。最后介绍了群体智能算法产生的背景和粒子群、蚁群两种代表性优化算法及其应用。

6.1 进化算法的产生与发展

6.1.1 进化算法的概念

进化算法(evolutionary algorithm, EA)是基于自然选择和自然遗传等生物进化机制的一种搜索算法。进化算法是以达尔文的进化论思想为基础,通过模拟生物进化过程与机制的求解问题的自组织、自适应的人工智能技术,是一类借鉴生物界自然选择和自然遗传机制的随机搜索算法,这些方法本质上从不同的角度对达尔文的进化原理进行了不同的运用和阐述,非常适用于处理传统搜索方法难以解决的复杂和非线性优化问题。生物进化是通过繁殖、变异、竞争和选择实现的;而进化算法则主要通过选择、重组和变异这三种操作实现优化问题的求解。

进化算法是一个“算法簇”,包括遗传算法(genetic algorithm, GA)、遗传规划(genetic programming)、进化策略(evolution strategy)和进化规划(evolution programming)等。尽管它有很多的变化,有不同的遗传基因表达方式,不同的交叉和变异算子,特殊算子的引用,以及不同的再生和选择方法,但它们产生的灵感都来自大自然的生物进化,进化算法的基本框架是遗传算法所描述的框架。

与普通搜索算法一样,进化算法也是一种迭代算法。不同的是在最优解的搜索过程中,普通搜索算法是从某个单一的初始点开始搜索,而进化算法是从原问题的一组解出发改进到另

一组较好的解,再从这组改进的解出发进一步改进。而且,进化算法不是直接对问题的具体参数进行处理,而是要求当原问题的优化模型建立后,还必须对原问题的解进行编码。

进化算法在搜索过程中利用结构化和随机性的信息,使最满足目标的决策获得最大的生存可能,是一种概率型的算法。在进化搜索中用目标函数值的信息,可以不必用目标函数的导数信息或与具体问题有关的特殊知识,因而进化算法具有广泛的应用性,高度的非线性,易修改性和可并行性。因此,与传统的基于微积分的方法和穷举法等优化算法相比,进化算法是一种具有高健壮性和广泛适用性的全局优化方法,具有自组织、自适应、自学习的特性,能够不受问题性质的限制,能适应不同的环境和不同的问题,有效地处理传统优化算法难以解决的大规模复杂优化问题。

6.1.2 进化算法的生物学背景

进化算法类似于生物进化,需要经过长时间的成长演化,最后收敛到最优化问题的一个或者多个解。因此,了解一些生物进化过程,有助于理解遗传算法的工作过程。

“适者生存”揭示了大自然生物进化过程中的一个规律:最适合自然环境的个体产生后代的可能性大。生物进化的基本过程如图 6.1 所示。

生物遗传物质的主要载体是染色体(chromosome),DNA 是其中最主要的遗传物质。染色体中基因的位置称作基因座,而基因所取的值又叫作等位基因。基因和基因座决定了染色体的特征,也决定了生物个体(individual)的性状。如头发的颜色是黑色、棕色或者金黄色等。

以一个初始生物群体(population)为起点,经过竞争后,一部分个体被淘汰而无法再进入这个循环圈,而另一部分则成为种群。竞争过程遵循生物进化中“适者生存,优胜劣汰”的基本规律,所以都有一个竞争标准,或者生物适应环境的评价标准。适应程度高的个体只是进入种群的可能性比较大,并不一定进入种群。而适应程度低的个体只是进入种群的可能性比较小,但并不一定被淘汰。这一重要特性保证了种群的多样性。

生物进化中种群经过婚配产生子代群体(简称子群)。在进化的过程中,可能会因为变异而产生新的个体。每个基因编码了生物机体的某种特征,如头发的颜色、耳朵的形状等。综合变异的作用使子群成长为新的群体而取代旧群体。在新的一个循环过程中,新的群体代替旧的群体而成为循环的开始。

6.1.3 进化算法的设计原则

一般来说,进化算法的求解包括以下几个步骤:给定一组初始解;评价当前这组解的性能;从当前这组解中选择一定数量的解作为迭代后的解的基础;再对其进行操作,得到迭代后的解;若这些解满足要求则停止,否则将这些迭代得到的解作为当前解重新操作。

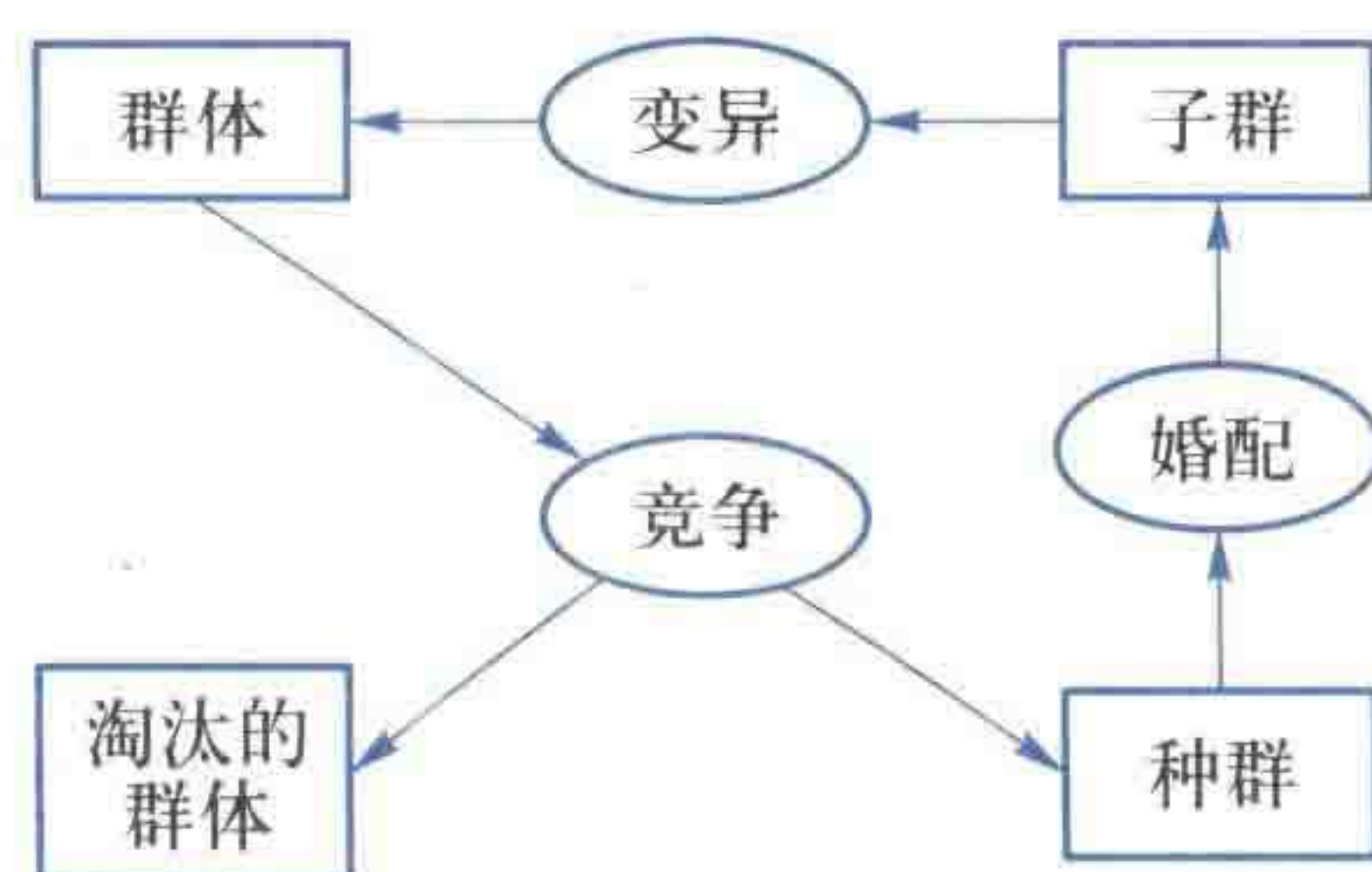


图 6.1 生物进化的基本过程

设计进化算法的基本原则如下。

① 适用性原则:一个算法的适用性是指该算法所能适用的问题种类,它取决于算法所需的限制与假定。优化问题的不同,则相应的处理方式也不同。

② 可靠性原则:一个算法的可靠性是指算法对于所设计的问题,以适当的精度求解其中大多数问题的能力。因为演化计算的结果带有一定的随机性和不确定性,所以,在设计算法时应尽量经过较大样本的检验,以确认算法是否具有较大的可靠度。

③ 收敛性原则:指算法能否收敛到全局最优。在收敛的前提下,希望算法具有较快的收敛速度。

④ 稳定性原则:指算法对其控制参数及问题的数据的敏感度。如果算法对其控制参数或问题的数据十分敏感,则依据它们取值的不同,将可能产生不同的结果,甚至过早地收敛到某一局部最优解。所以,在设计算法时应尽量使得算法对一组固定的控制参数能在较广泛的问题的数据范围内解题,而且对一组给定的问题数据,算法对其控制参数的微小扰动不很敏感。

⑤ 生物类比原则:因为进化算法的设计思想是基于生物演化过程的,所以那些在生物界被认为是有效的方法及操作可以通过类比的方法引入到算法中,有时会带来较好的结果。

6.2 基本遗传算法

对于自然界中生物遗传与进化机理的模仿,针对不同的问题设计了许多不同的编码方法来表示问题的可行解,产生了多种不同的遗传算子来模仿不同环境下的生物遗传特性。这样,由不同的编码方法和不同的遗传算子就构成了各种不同的遗传算法。但这些遗传算法都具有共同的特点,即通过对生物遗传和进化过程中选择、交叉、变异机理的模仿,来完成对问题最优解的自适应搜索过程。基于这个共同的特点,Goldberg 总结出了基本遗传算法,只使用选择算子、交叉算子和变异算子三种基本遗传算子,其遗传进化操作过程简单,容易理解,它给各种遗传算法提供了一个基本框架。进化算法的基本框架也是基本遗传算法所描述的框架。

6.2.1 遗传算法的基本思想

遗传算法主要借用生物进化中“适者生存”的规律。在遗传算法中,染色体对应的是数据或数组,通常是由一维的串结构数据来表示的。串上各个位置对应上述的基因座,而各位置上所取的值对应上述的等位基因。遗传算法处理的是染色体,或者称为基因型个体。一定数量的个体组成了群体。群体中个体的数量称为种群的大小,也叫种群规模。各个个体对环境的适应程度称为适应度。适应度大的个体被选择进行遗传操作产生新个体,体现了生物遗传中适者生存的原理。选择两个染色体进行交叉产生一组新的染色体的过程,类似生物遗传中的婚配。编码的某一个分量发生变化的过程,类似生物遗传中的变异。

遗传算法包含两个数据转换操作,一个是从表现型到基因型的转换,将搜索空间中的参数或解转换成遗传空间中的染色体或个体,这个过程称为编码(coding)。另一个是从基因型到表

现型的转换,即将个体转换成搜索空间中的参数,这个过程称为解码(decode)。

遗传算法在求解问题时从多个解开始,然后通过一定的法则进行逐步迭代以产生新的解。这多个解的集合称为一个种群,记为 $p(t)$ 。这里 t 表示迭代步,称为演化代。一般地, $p(t)$ 中元素的个数在整个演化过程中是不变的,可将群体的规模记为 M 。 $p(t)$ 中的元素称为个体或染色体,记为 $x_1(t), x_2(t), \dots$ 在进行演化时,要选择当前解进行交叉以产生新解。这些当前解称为新解的父解(parent),产生的新解称为后代解(offspring)。

6.2.2 遗传算法的发展历史

遗传算法的研究兴起在20世纪80年代末,但它的历史起源可追溯到20世纪60年代初期。早期的研究大多以对自然遗传系统的计算机模拟为主,其特点是侧重于对某些复杂操作的研究。虽然其中像自动博弈、生物模拟、模式识别和函数优化等给人以深刻的印象,但总的来说,这是一个无明确目标的发展时期,缺乏带有指导性的理论和计算工具的开拓。这种现象直到20世纪70年代中期,因美国密歇根大学J. Holland和DeJong的创造性研究成果的发表才得到改观。当然,早期的研究成果对于遗传算法的发展仍然有一定的影响,尤其是其中一些有代表性的技术和方法已被后来的遗传算法所吸收和发展。

在遗传算法作为搜索方法用于人工智能系统中之前,已有不少生物学家用计算机来模拟自然遗传系统,尤其是Fraser的模拟研究。Fraser于1962年提出了和现在的遗传算法十分相似的概念和思想,但是,他和其他一些学者并未认识到自然遗传算法可以转化为人工遗传算法,而Holland教授和他的学生不久就认识到这一转化的重要性。Holland认为比起寻找这种或那种具体求解方法来说,开拓一种能模拟自然选择遗传机制的、带有一般性的理论和方法更有意义。在这一时期,Holland不但发现了基于适应度的人工遗传选择的基本作用,而且还对群体操作等进行了认真的研究。1965年,他首次提出了人工遗传操作的重要性,并把这些应用于自然系统和人工系统中。

1967年,Holland教授的学生Bagley在他的博士论文中首次提出了遗传算法这一术语,并讨论了遗传算法在博弈中的应用。他所提出的选择、交叉和变异操作与目前遗传算法中的相应操作十分接近,尤其是他对选择做了十分有意义的研究。他认识到,在遗传进化过程的前期和后期,选择概率应合适地变动。为此,他引入了适应度定标概念。他还首次提出了遗传算法自我调整概念,即把交叉和变异的概率融于染色体本身的编码中,从而可使算法自我调整优化。尽管Bagley没有进行计算机模拟实验,但这些思想对于后来遗传算法的发展具有重要意义。

在同一时期,Rosenberg也对遗传算法进行了研究。他的研究依然是以模拟生物进化为主,但他在遗传操作方面提出了不少独特的设想。1970年,Cavicchio把遗传算法应用于模式识别中,实际上他并未直接涉及模式识别,而仅用遗传算法设计一组应用于识别的检测器。Cavicchio对于遗传操作以及遗传算法的自动调整也做了不少有特色的研究。

1971年,Hollstien第一个把遗传算法用于函数优化。他在论文《计算机控制系统中人工遗传自适应方法》中阐述了遗传算法用于数字反馈控制的方法,但实际上,主要讨论了对于二变

量函数的优化问题,其中,对于优势基因控制、交叉和变异以及各种编码技术进行了深入的研究。

1975年是遗传算法研究历史上十分重要的一年。这一年,Holland出版了他的专著《自然系统和人工系统的适配》,系统地阐述了遗传算法的基本理论和方法,并提出了对遗传算法的理论研究和发展极为重要的模式理论。同年,DeJong完成了重要论文《遗传自适应系统的行为分析》,把Holland的模式理论与他的计算实验结合起来。这是遗传算法发展中的又一个里程碑。尽管DeJong和Hollstien一样主要侧重于函数优化的应用研究,但他将选择、交叉和变异操作进一步完善和系统化,同时又提出了诸如代沟等新的遗传操作技术。DeJong的研究工作为遗传算法及其应用打下了坚实的基础,他所得出的许多结论具有普遍的指导意义。

20世纪80年代以后,遗传算法进入兴盛发展时期,无论是理论研究还是应用研究都成为十分热门的课题。遗传算法广泛应用于自动控制、生产计划、图像处理、机器人等研究领域。

6.2.3 编码

遗传算法中包含了五个基本要素:参数编码、初始群体的设定、适应度函数的设计、遗传操作设计和控制参数设定。

由于遗传算法不能直接处理问题空间的参数,因此,必须通过编码将要求解的问题表示成遗传空间的染色体或者个体。它们由基因按一定结构组成。由于遗传算法的健壮性,对编码的要求并不苛刻。对一个具体的应用问题如何编码是应用遗传算法求解的首要问题,也是遗传算法应用的难点。事实上,还不存在一种通用的编码方法,特殊的问题往往采用特殊的方法。

1. 位串编码

将问题空间的参数编码为一维排列的染色体的方法,称为一维染色体编码方法。一维染色体编码中最常用的符号集是二值符号集 $\{0,1\}$,即采用二进制编码(binary encoding)。

(1) 二进制编码

二进制编码是用若干二进制数表示一个个体,将原问题的解空间映射到位串空间 $B=\{0,1\}$ 上,然后在位串空间上进行遗传操作。

优点:二进制编码类似于生物染色体的组成,从而使算法易于用生物遗传理论来解释,并使得遗传操作如交叉、变异等很容易实现。另外,采用二进制编码时,算法处理的模式数最多。

缺点:

① 相邻整数的二进制编码可能具有较大的汉明距离(Hamming distance)。例如,15和16的二进制表示为01111和10000,因此,算法要从15改进到16则必须改变所有的位。这种缺陷造成了Hamming悬崖(Hamming cliffs),将降低遗传算子的搜索效率。

② 二进制编码时,一般要先给出求解的精度。但求解的精度确定后,就很难在算法执行过程中进行调整,从而使算法缺乏微调(fine-tuning)的功能。若在算法一开始就选取较高的精度,那么串长就很大,这样也将降低算法的效率。

③ 在求解高维优化问题时,二进制编码串将非常长,从而使得算法的搜索效率很低。

(2) Gray 编码

Gray 编码是将二进制编码通过一个变换进行转换得到的编码。设二进制串 $\langle \beta_1 \beta_2 \cdots \beta_n \rangle$ 对应 Gray 串 $\langle \gamma_1 \gamma_2 \cdots \gamma_n \rangle$, 则从二进制编码到 Gray 编码的变换为

$$\gamma_k = \begin{cases} \beta_1 & k = 1 \\ \beta_{k-1} \oplus \beta_k & k > 1 \end{cases} \quad (6.1)$$

其中, $\oplus$ 表示模 2 的加法。从一个 Gray 串到二进制串的变换为

$$\beta_k = \sum_{i=1}^k \gamma_i (\bmod 2) = \begin{cases} \gamma_1 & k = 1 \\ \beta_{k-1} \oplus \gamma_k & k > 1 \end{cases} \quad (6.2)$$

Gray 编码的优点是克服了二进制编码的 Hamming 悬崖的缺点。

2. 实数编码

为克服二进制编码的缺点,对问题的变量是实向量的情形,可以直接采用实数编码。

实数编码是用若干实数表示一个个体,然后在实数空间上进行遗传操作。

采用实数表达法不必进行数制转换,可直接在解的表现型上进行遗传操作。从而可引入与问题领域相关的启发式信息来增加算法的搜索能力。近年来,遗传算法在求解高维或复杂优化问题时一般使用实数编码。

3. 多参数级联编码

对于多参数优化问题的遗传算法,常采用多参数级联编码。其基本思想是把每个参数先进行二进制编码得到子串,再把这些子串连成一个完整的染色体。多参数级联编码中的每个子串对应各自的编码参数,所以,可以有不同的串长度和参数的取值范围。

6.2.4 群体设定

由于遗传算法是对群体进行操作的,所以,必须为遗传操作准备一个由若干初始解组成的初始群体。群体设定主要包括两个方面:初始种群的产生和种群规模的确定。

1. 初始种群的产生

遗传算法中初始群体中的个体可以是随机产生的,但最好采用如下策略设定:

① 根据问题固有知识,设法把握最优解所占空间在整个问题空间中的分布范围,然后,在此分布范围内设定初始群体。

② 先随机产生一定数目的个体,然后从中挑选最好的个体加入初始群体中。这种过程不断迭代,直到初始群体中个体数目达到了预先确定的规模。

2. 种群规模的确定

群体中个体的数量称为种群规模。

种群规模影响遗传优化的结果和效率。当种群规模太小时,遗传算法的优化性能一般不太好,容易陷入局部最优解。而当种群规模太大时,则计算复杂。

种群规模的确定受遗传操作中选择操作的影响很大。模式定理表明:若种群规模为 M , 则遗传操作可从这 M 个个体中生成和检测 M^3 个模式, 并在此基础上不断形成和优化积木块, 直到找到最优解。

显然, 种群规模越大, 遗传操作所处理的模式就越多, 产生有意义的积木块并逐步进化为最优解的机会就越高。种群规模太小, 会限制遗传算法的搜索空间范围, 因而搜索有可能停止在未成熟阶段, 出现未成熟收敛现象, 使算法陷入局部最优解。因此, 必须保持种群的多样性, 即种群规模不能太小。

另一方面, 种群规模太大会带来若干弊病: 一是群体越大, 其适应度评估次数增加, 所以计算量也增加, 从而影响算法效率; 二是群体中个体生存下来的概率大多采用和适应度成比例的方法, 当群体中个体非常多时, 少量适应度很高的个体会被选择而生存下来, 但大多数个体却被淘汰, 这会影响配对库的形成, 从而影响交叉操作。

种群规模一般取为 20~100。

6.2.5 适应度函数

遗传算法遵循自然界优胜劣汰的原则, 在进化搜索中基本上不用外部信息, 而是用适应度值表示个体的优劣, 作为遗传操作的依据。适应度是评价个体优劣的标准。个体的适应度高, 则被选择的概率就高, 反之就低。适应度函数 (fitness function) 是用来区分群体中的个体好坏的标准, 是算法演化过程的驱动力, 是进行自然选择的唯一依据。改变种群内部结构的操作都是通过适应度值加以控制的。因此, 对适应度函数的设计非常重要。

在具体应用中, 适应度函数的设计要结合求解问题本身的要求而定。一般而言, 适应度函数是由目标函数变换得到的。下面讨论将目标函数变换成适应度函数的方法。

1. 将目标函数映射成适应度函数的方法

最直观的方法是直接将待求解优化问题的目标函数作为适应度函数。

若目标函数 $f(x)$ 为最大化问题, 则适应度函数可以取为

$$Fit[f(x)] = f(x) \quad (6.3)$$

若目标函数 $f(x)$ 为最小化问题, 则适应度函数可以取为

$$Fit[f(x)] = \frac{1}{f(x)} \quad (6.4)$$

2. 适应度函数的尺度变换

在遗传算法中, 将所有妨碍适应度值高的个体产生, 从而影响遗传算法正常工作的问题统称为欺骗问题 (deceptive problem)。

在设计遗传算法时, 群体的规模一般在几十至几百, 与实际物种的规模相差很远。因此, 个体繁殖数量的调节在遗传操作中就显得比较重要。如果群体中出现了超级个体, 即该个体的适应值大大超过群体的平均适应度值, 则当按照适应度值比例进行选择时, 该个体很快会在群体中占有绝对的比例, 从而导致算法较早地收敛到一个局部最优点, 这种现象称为过早收

敛,是一种欺骗问题。为了防止过早收敛问题,应该缩小这些个体的适应度,以降低这些超级个体的竞争力。另一方面,在搜索过程的后期,虽然群体具有足够的多样性,但群体的平均适应值可能会接近群体的最优适应度值,群体中实际上已不存在竞争,从而搜索目标也难以得到改善,出现了停滞现象,也是一种欺骗问题。在这两种情况下,都应该改变原始适应度值的比例关系,以提高个体之间的竞争力。

对适应度函数值域的某种映射变换称为适应度函数的尺度变换(fitness scaling)或者定标。

(1) 线性变换

设原适应度函数为 f ,定标后的适应度函数为 f' ,则线性变换可采用下式表示为

$$f' = af + b \quad (6.5)$$

其中,系数 a 和 b 可以有多种途径设定,但要满足两个条件:

① 变换后的适应度的平均值 f'_{avg} 要等于原适应度平均值 f_{avg} ,以保证适应度为平均值的个体在下一代的期望复制数为1,即

$$f'_{\text{avg}} = f_{\text{avg}} \quad (6.6)$$

② 变换后适应度函数的最大值 f'_{max} 要等于原适应度函数平均值 f_{avg} 的指定倍数,以控制适应度最大的个体在下一代中的复制数,即

$$f'_{\text{max}} = C_{\text{mult}} f_{\text{avg}} \quad (6.7)$$

其中, C_{mult} 是为得到所期待的最优群体个体的复制数。实验表明,对于不太大的群体($n = 50 \sim 100$), C_{mult} 可在1.2~2.0范围内取值。

根据上述条件,可以确定线性变换的系数为

$$a = \frac{(C_{\text{mult}} - 1)f_{\text{avg}}}{f_{\text{max}} - f_{\text{avg}}} \quad (6.8a)$$

$$b = \frac{(f_{\text{max}} - C_{\text{mult}} f_{\text{avg}})f_{\text{avg}}}{f_{\text{max}} - f_{\text{avg}}} \quad (6.8b)$$

线性变换法变换了适应度之间的差距,保持了种群的多样性,计算简便,易于实现。如果种群里某些个体适应度远远低于平均值时,有可能出现变换后适应度值为负的情况。为满足最小适应度值非负的条件,可以进行如下变换:

$$a = \frac{f_{\text{avg}}}{f_{\text{avg}} - f_{\text{min}}} \quad (6.9a)$$

$$b = \frac{-f_{\text{min}} f_{\text{avg}}}{f_{\text{avg}} - f_{\text{min}}} \quad (6.9b)$$

(2) 非线性变换

幂函数变换法变换公式为

$$f' = f^K \quad (6.10)$$

其中,幂指数 K 与求解问题有关,而且在算法过程中可按需要修正。

指数变换法变换公式为

$$f' = e^{-af} \quad (6.11)$$

这种变换方法的基本思想来源于模拟退火过程,式中的系数 a 决定了复制的强制性,其值越小,复制的强制性就越趋向于那些具有最大适应度的个体。

6.2.6 选择

选择操作也称为复制(reproduction)操作,是从当前群体中按照一定概率选出优良的个体,使它们有机会作为父代繁殖下一代子孙。判断个体优良与否的准则是各个个体的适应度值。显然这一操作借用了达尔文适者生存的进化原则,即个体适应度越高,其被选择的机会就越多。

需要注意的是:如果总挑选最好的个体,遗传算法就变成了确定性优化方法,使种群过快地收敛到局部最优解。如果只作随机选择,则遗传算法就变成完全随机方法,需要很长时间才能收敛,甚至不收敛。因此,选择方法的关键是找一个策略,既要使得种群较快地收敛,也要维持种群的多样性。

选择操作的实现方法很多。这里,介绍几种常用的选择方法。

1. 个体选择概率分配方法

在遗传算法中,哪个个体被选择进行交叉是按照概率进行的。适应度大的个体被选择的概率大,但不是说一定能够被选上。同样,适应度小的个体被选择的概率小,但也可能被选上。所以,首先要根据个体的适应度确定被选择的概率。个体选择概率的常用分配方法有以下两种。

(1) 适应度比例方法

适应度比例方法(fitness proportional model)也称为蒙特卡洛(Monte Carlo)法,是目前遗传算法中最基本也是最常用的选择方法。适应度比例方法中,各个个体被选择的概率和其适应度值成比例。设群体规模大小为 M ,个体 i 的适应度值为 f_i ,则这个个体被选择的概率为

$$p_{si} = \frac{f_i}{\sum_{i=1}^M f_i} \quad (6.12)$$

(2) 排序方法

排序方法(rank-based model)是计算每个个体的适应度后,根据适应度大小顺序对群体中个体进行排序,然后把事先设计好的概率按排序分配给个体,作为各自的选择概率。在排序方法中,选择概率仅仅取决于个体在种群中的序位,而不是实际的适应度值。排在前面的个体有较多的被选择的机会。

它的优点是克服了适应度值比例选择策略的过早收敛和停滞现象,而且对于极大值或极小值问题,不需要进行适应度值的标准化和调节,可以直接使用原始适应度值进行排名选择。排序方法比比例方法具有更好的健壮性,是一种比较好的选择方法。

① 线性排序。线性排序选择最初由 J. E. Baker 提出,他首先假设群体成员按适应度值大小

从好到坏依次排列为 $x_1, x_2, \dots, x_M$, 然后根据一个线性函数给第 i 个个体 x_i 分配选择概率 p_i , 即

$$p_i = \frac{a - bi}{M(M + 1)} \quad (6.13)$$

其中, a, b 是常数。

② 非线性排序。Z. Michalewicz 提出将群体成员按适应度值从好到坏依次排列, 并按下式分配选择概率为

$$p_i = \begin{cases} q(1 - q)^{i-1} & i = 1, 2, \dots, M - 1 \\ (1 - q)^{M-1} & i = M \end{cases} \quad (6.14)$$

其中, i 为个体排序序号。 q 是一个常数, 表示最好的个体的选择概率。

也可使用其他非线性函数来分配选择概率 p_i , 只要满足以下条件:

a. 若 $P = \{x_1, x_2, \dots, x_M\}$ 且 $f(x_1) \geq f(x_2) \geq \dots \geq f(x_M)$, 则分配的概率 p_i 满足 $p_1 \geq p_2 \geq \dots \geq p_M$ 。

b. $\sum_{i=1}^M p_i = 1$ 。

2. 选择个体方法

选择操作是根据个体的选择概率确定哪些个体被选择进行交叉、变异等操作。基本的选择方法如下。

(1) 轮盘赌选择方法

轮盘赌选择 (roulette wheel selection) 方法在遗传算法中使用得最多。

在轮盘赌选择方法中先按个体的选择概率产生一个轮盘, 轮盘每个区的角度与个体的选择概率成比例, 然后产生一个随机数, 它落入转盘的哪个区域就选择相应的个体交叉。

显然, 选择概率大的个体被选中的可能性大, 获得交叉的机会就大。

在实际计算时, 可以按照个体顺序求出每个个体的累积概率, 然后产生一个随机数, 它落入累积概率的哪个区域就选择相应的个体交叉。例如, 表 6.1 所示 11 个个体的适应度、选择概率和累积概率。为了选择交叉个体, 需要进行多轮选择。例如, 第 1 轮产生一个随机数为 0.81, 落在第 5 个和第 6 个个体之间, 则第 6 个个体被选中。第 2 轮产生一个随机数为 0.32, 落在第 1 个和第 2 个个体之间, 则第 2 个个体被选中, 以此类推。

表 6.1 个体适应度、选择概率和累积概率

个体	1	2	3	4	5	6	7	8	9	10	11
适应度	2.0	1.8	1.6	1.4	1.2	1.0	0.8	0.6	0.4	0.2	0.1
选择概率	0.18	0.16	0.14	0.13	0.11	0.09	0.07	0.05	0.04	0.02	0.01
累积概率	0.18	0.34	0.48	0.61	0.72	0.81	0.88	0.93	0.97	0.99	1.00

(2) 锦标赛选择方法

锦标赛选择方法 (tournament selection model) 是从群体中随机选择 k 个个体, 将其中适应

度最高的个体保存到下一代。这一过程反复执行,直到保存到下一代的个体数达到预先设定的数量为止。参数 k 称为竞赛规模。

锦标赛选择方法的优点是克服了基于适应度值比例的选择和基于排名的选择在群体规模很大时,其额外计算量(如计算总体适应度值或排序)很大的问题。它常常能比轮盘赌选择方法得到更加多样化的群体。

显然,这种方法也使得适应度值高的个体具有较大的生存机会。同时,由于它只使用适应度值的相对值作为选择的标准,而与适应度值的数值大小不成直接比例,从而也能避免超级个体的影响,一定程度上避免了过早收敛和停滞现象的发生。

作为锦标赛选择方法的一种特殊情况,随机竞争方法(stochastic tournament model)是每次按轮盘赌选择方法选取一对个体,然后让这两个个体进行竞争,适应度高者获胜。如此反复,直到选满为止。

(3) 最佳个体保存方法

最佳个体保存方法或称为精英选拔方法(elitist model)是把群体中适应度最高的一个或者多个个体不进行交叉而直接复制到下一代中,保证遗传算法终止时得到的最后结果一定是历代出现过的最高适应度的个体。使用这种方法能够明显提高遗传算法的收敛速度,但可能使种群过快收敛,从而只找到局部最优解。实验结果表明:保留种群个体总数的 2%~5% 的适应度最高的个体,效果最为理想。

在使用其他选择方法时,一般都同时使用最佳个体保存方法,以保证不会丢失最优个体。

6.2.7 交叉

当两个生物机体配对或者复制时,它们的染色体相互混合,产生一对由双方基因组成的新的染色体。这一过程称为交叉(crossover)或者重组(recombination)。

交叉得到的后代可能继承了上代的优良基因,其后代会比它们的父母更加优秀,但也可能继承了上代的不良基因,其后代则会比它们的父母差,难以生存,甚至不能再复制自己。越能适应环境的后代越能继续复制自己并将其基因传给后代。由此形成一种趋势:每一代总是比其父母一代生存和复制得更好。

这里举个简单的例子。假设雌性动物仅仅青睐大眼睛的雄性。这样眼睛尺寸越大的雄性越受到雌性的青睐,生出更多的后代。可以说动物的适应性正比于它的眼睛的直径。因此,从一个具有不同大小眼睛的雄性群体出发,当动物进化时,在同位基因中,能够产生大眼睛雄性动物的基因,相对于产生小眼睛雄性动物的基因,就更有可能复制到下一代。当进化几代以后,大眼睛雄性群体将会占据优势。生物逐渐向一种特殊遗传类型收敛。

遗传算法中起核心作用的是交叉算子,也称为基因重组(recombination)。采用的交叉方法应能够使父串的特征遗传给子串。子串应能够部分或者全部继承父串的结构特征和有效基因。

1. 基本的交叉算子

(1) 一点交叉

一点交叉(single-point crossover)又称为简单交叉。其具体操作是:在个体串中随机设定一个交叉点,实行交叉时,该点前或后的两个个体的部分结构进行互换,并生成两个新的个体。

(2) 二点交叉

二点交叉(two-point crossover)的操作与一点交叉类似,只是设置了两个交叉点(仍然是随机设定),将两个交叉点之间的码串相互交换。

类似于二点交叉,可以采用多点交叉(multiple-point crossover)。

2. 修正的交叉方法

由于交叉,可能出现不满足约束条件的非法染色体。为解决这一问题,一种处理方法是构造惩罚函数的方法,但试验效果不佳,使得本已复杂的问题更加复杂。另一种处理方法是对交叉、变异等遗传操作进行适当地修正,使其满足优化问题的约束条件。例如,在 TSP 问题中采用部分匹配交叉(partially matched crossover, PMX),顺序交叉(order crossover, OX)和循环交叉(cycle crossover, CX)等。这些方法对于其他一些问题也同样适用。

下面简单介绍部分匹配交叉。PMX 是由 Goldberg D. E. 和 R. Lingle 在 1985 年提出的。在 PMX 操作中,先依据均匀随机分布产生两个位串交叉点,定义这两点之间的区域为一匹配区域,并使用位置交换操作交换两个父串的匹配区域。例如,在任务排序问题中,两父串及匹配区域为

$$\begin{array}{l} A = 9 \quad 8 \quad 4 \quad | \quad 5 \quad 6 \quad 7 \quad | \quad 1 \quad 3 \quad 2 \\ B = 8 \quad 7 \quad 1 \quad | \quad 2 \quad 3 \quad 9 \quad | \quad 5 \quad 4 \quad 6 \end{array}$$

首先交换 A 和 B 的两个匹配区域,得到

$$\begin{array}{l} A' = 9 \quad 8 \quad 4 \quad | \quad 2 \quad 3 \quad 9 \quad | \quad 1 \quad 3 \quad 2 \\ B' = 8 \quad 7 \quad 1 \quad | \quad 5 \quad 6 \quad 7 \quad | \quad 5 \quad 4 \quad 6 \end{array}$$

显然, A' 和 B' 中出现重复的任务,所以是非法的调度。解决的方法是将 A' 和 B' 中匹配区域外出现的重复任务,按照匹配区域内的位置映射关系进行交换,从而使排列成为可行调度,即

$$\begin{array}{l} A'' = 7 \quad 8 \quad 4 \quad | \quad 2 \quad 3 \quad 9 \quad | \quad 1 \quad 6 \quad 5 \\ B'' = 8 \quad 9 \quad 1 \quad | \quad 5 \quad 6 \quad 7 \quad | \quad 2 \quad 4 \quad 3 \end{array}$$

交叉概率是用来确定两个染色体进行局部的互换以产生两个新的子代的概率。采用较大的交叉概率 P_c 可以增强遗传算法开辟新的搜索区域的能力,但高性能模式遭到破坏的可能性增加。采用太低的交叉概率会使搜索陷入迟钝状态。 P_c 一般取值为 0.25~1.00。实验表明交叉概率通常取值为 0.7 左右是理想的。每次从群体中选择两个染色体,同时生成 0 和 1 之间的一个随机数,然后根据这个随机数确定这两个染色体是否需要交叉。如果这个随机数低于交叉概率(0.7),就进行交叉。然后沿着染色体的长度随机地选择一个位置,并把此位置之后的所有的位进行互换。

6.2.8 变异

进化机制除了能够改进已有的特征,也能够产生新的特征。例如,可以设想某个时期动物没有眼睛,而是靠嗅觉和触觉来躲避捕食它们的动物。然而,两个动物有次交配时,基因传递给子孙后代的过程中,会有很小的概率发生差错,从而使基因发生微小的改变,这就是基因的变异。例如,某次变异在它们后代的头部皮肤上,发育出了一个具有光敏效应的细胞,使它们的后代能够识别周围环境是亮还是暗的。它能够感知捕食者的到来,能够知道现在是白天还是夜晚,它在地上还是地下等信息,有利于它的生存。这个光敏细胞会进一步变异,逐渐成为一个区域,从而成为眼睛。

如果生物繁殖仅仅是上述交叉过程,那么即使经历成千上万代,适应能力最强的成员的眼睛尺寸也只能像初始群体中的最大眼睛一样。而根据对自然的观察可以看到,人类的眼睛尺寸实际存在一代比一代大的趋势。发生变异的概率通常都很小,但在经历许多代以后变异就会很明显。一些变异对生物是不利的,一些对生物的适应性可能没有影响,但也有一些可能会给生物带来一些好处,使它们超过其他同类的生物。例如前面的例子,变异可能会产生眼睛更大的生物。当经历许多代以后,眼睛会越来越大。

在遗传算法中,变异是将个体编码中的一些位进行随机变化。变异的主要目的是维持群体的多样性,为选择、交叉过程中可能丢失的某些遗传基因进行修复和补充。变异算子的基本内容是对群体中的个体串的某些基因座上的基因值作变动。变异操作是按位进行的,即把某一位的内容进行变异。变异概率是在一个染色体中按位进行变化的概率。主要变异方法有以下4种。

(1) 位点变异

位点变异是指对群体中的一个个体码串,随机挑选一个或多个基因座,并对这些基因座的基因值以变异概率 P_m 作变动。

对于二进制编码的个体来说,若某位原为0,则通过变异操作就变成了1,反之亦然。

对于整数编码,将被选择的基因变为以概率选择的其他基因。如果需要消除非法性,将其他基因所在的基因座上的基因变为被选择的基因。

(2) 逆转变异

在个体码串中随机选择两点(称为逆转点),然后将两个逆转点之间的基因值以逆向排序插入到原位置中。

(3) 插入变异

在个体码串中随机选择一个码,然后将此码插入随机选择的插入点中间。

(4) 互换变异

随机选取染色体的两个基因进行简单互换。

在遗传算法中,变异属于辅助性的搜索操作。变异概率 P_m 一般不能大,以防止群体中重要的、单一的基因被丢失。事实上,变异概率太大将使遗传算法趋于纯粹的随机搜索。通常取变异概率 P_m 为0.001左右。

6.2.9 遗传算法的一般步骤

综上所述,遗传算法步骤如下:

Step 1 使用随机方法或者其他方法,产生一个有 M 个染色体的初始群体 $pop(1)$, $t:=1$;

Step 2 对群体 $pop(t)$ 中的每一个染色体 $pop_i(t)$, 计算它的适应值

$$f_i = fitness[pop_i(t)]$$

Step 3 若满足停止条件,则算法停止;否则,以概率

$$p_i = f_i / \sum_{j=1}^N f_j$$

Step 4 从 $pop(t)$ 中随机选择一对染色体,以概率 P_c 进行交叉产生两个新的染色体,替换 $pop(t)$ 中被交叉的两个染色体,得到一个新的群体

$$newpop(t+1) = \{pop_j(t) \mid j = 1, 2, \dots, N\} \\ crosspop(t+1)$$

Step 5 以一个较小的变异概率 P_m 从新群体中选择一个基因发生变异,然后用变异后的染色体替换变异前的染色体,形成 $mutpop(t+1)$; $t:=t+1$, 成为一个新的群体 $pop(t) = mutpop(t+1)$; 返回 Step 2。

遗传算法的基本流程如图 6.2 所示。

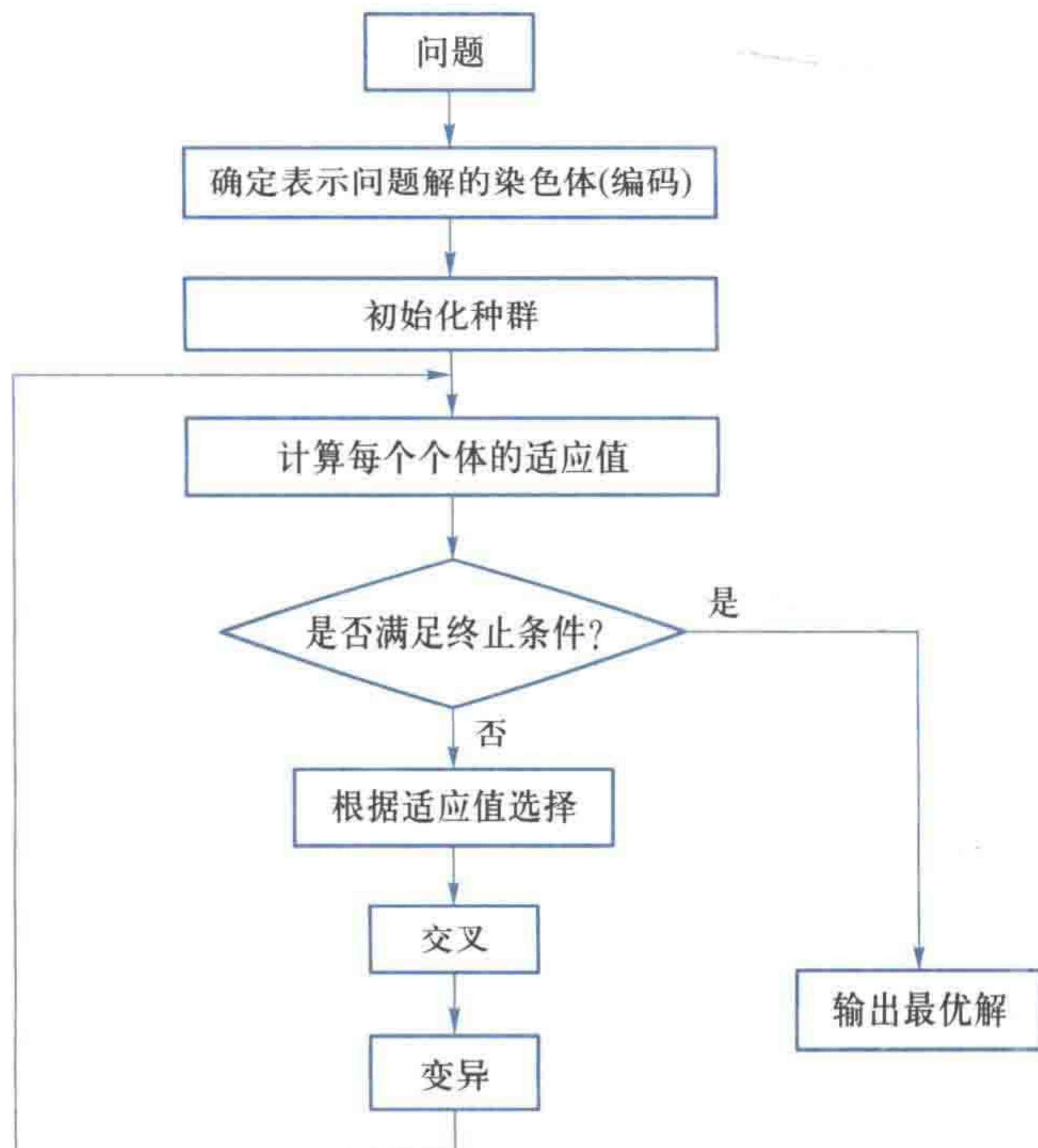


图 6.2 遗传算法的基本流程

6.2.10 遗传算法的特点

遗传算法比起其他普通的优化搜索,采用了许多独特的方法和技术。归纳起来,主要有以下几个方面。

① 遗传算法的编码操作使得它可以直接对结构对象进行操作。所谓结构对象泛指集合、序列、矩阵、树、图、链和表等各种一维、二维,甚至三维结构形式的对象。因此,遗传算法具有非常广泛的应用领域。

② 遗传算法是一个利用随机技术来指导对一个被编码的参数空间进行高效率搜索的方法,而不是无方向的随机搜索。这与其他随机搜索是不同的。

③ 许多传统搜索方法都是单解搜索算法,即通过一些变动规则,将问题的解从搜索空间中的当前解移到另一解。对于多峰分布的搜索空间,这种点对点的搜索方法常常会陷于局部的某个单峰的优解。而遗传算法采用群体搜索策略,即采用同时处理群体中多个个体的方法,同时对搜索空间中的多个解进行评估,从而使遗传算法具有较好的全局搜索性能,减少了陷于局部优解的风险,但还是不能保证每次都得到全局最优解。遗传算法本身也十分易于并行化。

④ 在基本遗传算法中,基本上不用搜索空间的知识或其他辅助信息,而仅用适应度函数值来评估个体,并在此基础上进行遗传操作,使种群中个体之间进行信息交换。特别是遗传算法的适应度函数不仅不受连续可微的约束,而且其定义域也可以任意设定。对适应度函数的唯一要求是能够算出可以比较的正值。遗传算法的这一特点使它的应用范围大大扩展,非常适用于传统优化方法难以解决的复杂优化问题。

6.3 遗传算法的改进算法

为了改进遗传算法的优化性能,人们提出了许多改进算法。下面介绍几种主要的改进算法。

6.3.1 双倍体遗传算法

1. 基本思想

Holland 提出的遗传算法通常被称作基本遗传算法(simple genetic algorithm, SGA)。SGA 是一种“单倍体遗传”,每个基因型由一条染色体组成。自然界中一些简单的植物采用这种遗传,而大多数动物和高级植物都采用双倍体遗传,即每个基因型由一对染色体组成。

双倍体遗传算法(double chromosomes genetic algorithm, DCGA)采用显性和隐性两个染色体同时进行进化。因此,双倍体遗传算法提供了一种记忆以前有用的基因块的功能:在某些低适应度染色体中,其局部基因块十分有用,是最优解中的基因片段,但由于基因块在当前染色体中的位置不适等因素,导致当前染色体的适应度值不高。保留这些基因块,有利于提高物种的适应能力。

双倍体遗传算法采用显性遗传,如图 6.3 所示。当一对染色体对应的基因块不同时,显性基因遗传给后代,图中大写字母为显性基因。



图 6.3 显性遗传

这种双倍体遗传延长了有用基因块的寿命,提高了算法的收敛能力,并且在变异概率低的情况下,也能保持一定水平的多样性。Goldberg 用动态 Knapsack 问题进行了比较研究,实验表明双倍体比单倍体的动态跟踪能力强。

2. 双倍体遗传算法的设计

(1) 编码/解码

对于双倍体遗传算法,群体中的每个个体具有两个染色体,一个是显性染色体,一个是隐性染色体。每个染色体的编码/解码方式与基本的遗传算法相同。

(2) 复制算子

当进行复制操作时,计算显性染色体的适应度,按照显性染色体的复制概率将个体复制到下一代群体当中。

(3) 交叉算子

对于交叉操作,从群体中选取两个个体,两个个体的显性染色体进行交叉操作,隐性染色体也同时进行交叉操作。显性交叉率和隐性交叉率可以不同,为了工程实现方便也可将显性交叉率和隐性交叉率设为相同,然后进入下一代群体。

(4) 变异算子

对于变异操作,个体的显性染色体按照正常的变异概率 P_m 执行变异操作,而个体的隐性染色体则按照较大的变异概率执行变异操作。

(5) 双倍体遗传算法显隐性重排算子

当三个遗传算子都执行完成以后,将个体的染色体显隐性进行重新排定,个体中适应度较大的染色体设定为显性染色体,适应度较小的染色体设定为隐性染色体。

双倍体遗传算法程序流程如图 6.4 所示。

6.3.2 双种群遗传算法

1. 基本思想

基本遗传算法针对一个宏观的种群进行复制、交叉、变异三种操作,类似于人类进化过程,一群人随着时间的推移而不断地进化,并具备越来越多的优良品质。然而由于他们的生长、演化、环境和原始祖先的局限性,经过相当长的时间后,他们将逐渐进化到某些特征相对优势的状态,称之为平衡态。当一个种群进化到这种状态,这个种群的特性就不会再有很大的变化。为了解决这个问题,多种群遗传算法是使用多种群同时进化,并交换种群之间优秀个体所携带

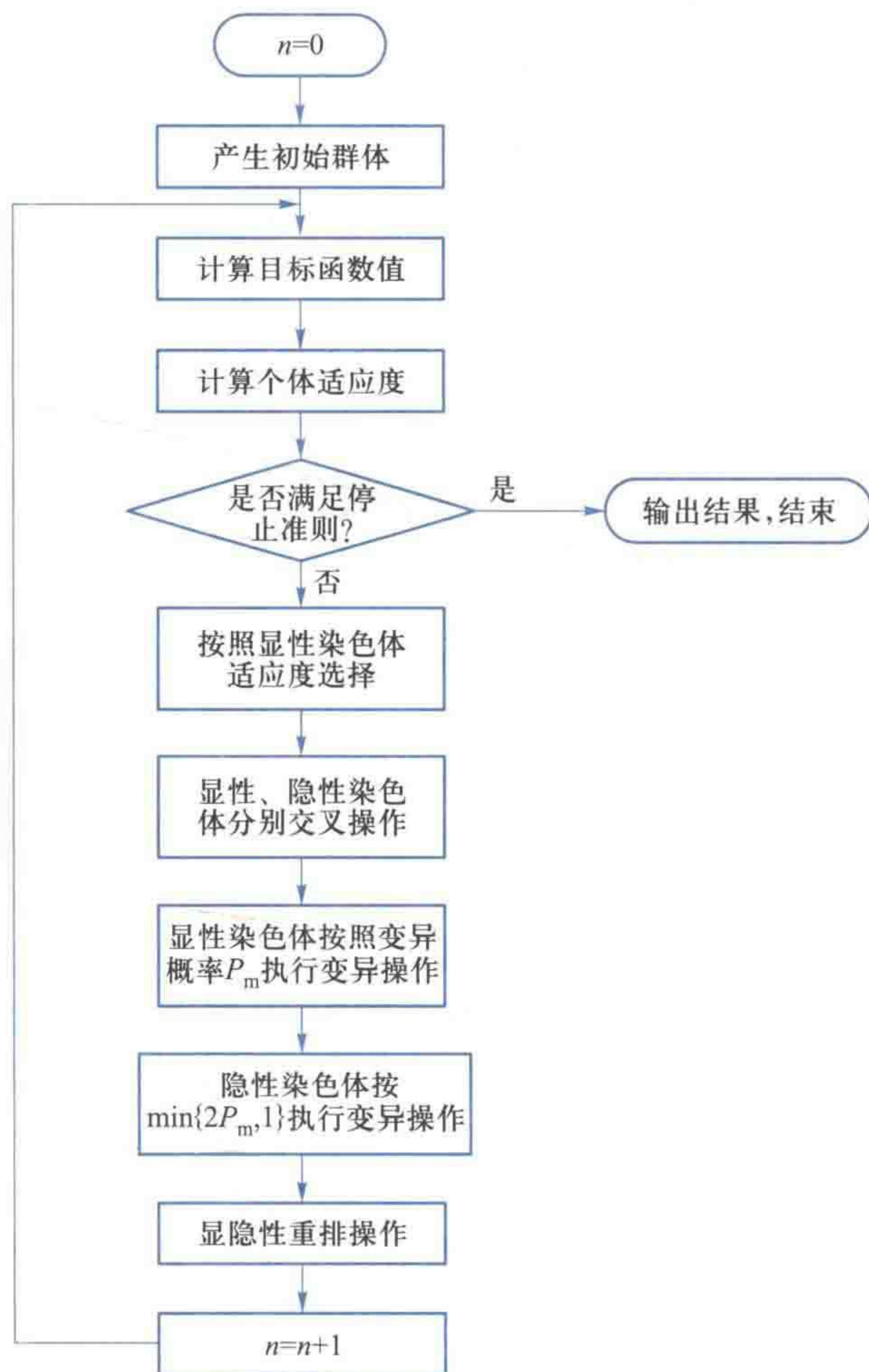


图 6.4 双倍体遗传算法程序流程

的遗传信息,以打破种群内的平衡态达到更高的平衡态,有利于算法跳出局部最优。就本质而言,多种群遗传算法是一种并行算法,可以提高算法的效率。

下面介绍双种群遗传算法(dual populations genetic algorithm, DPGA)。

2. 双种群遗传算法的设计

建立两个遗传算法群体,分别独立地运行复制、交叉、变异操作,当一代或几代运行结束以后,分别选择两个种群中的最优个体进行交换,并在两个种群中随机选择若干个个体进行交换。

(1) 编码/解码设计

编码/解码方法与基本遗传算法相同。

(2) 交叉算子、变异算子

两个种群分别进行选择、交叉、变异等操作,且交叉概率、变异概率不同。

(3) 杂交算子

设种群 A 与种群 B ,当 A 与 B 种群都完成了选择、交叉、变异算子后,产生一个随机数 num ,随机选择 A 中 num 个个体与 A 中最优个体,随机选择 B 中 num 个个体与 B 中最优个体,交换两者,以打破平衡态。

双种群遗传算法程序流程如图 6.5 所示。

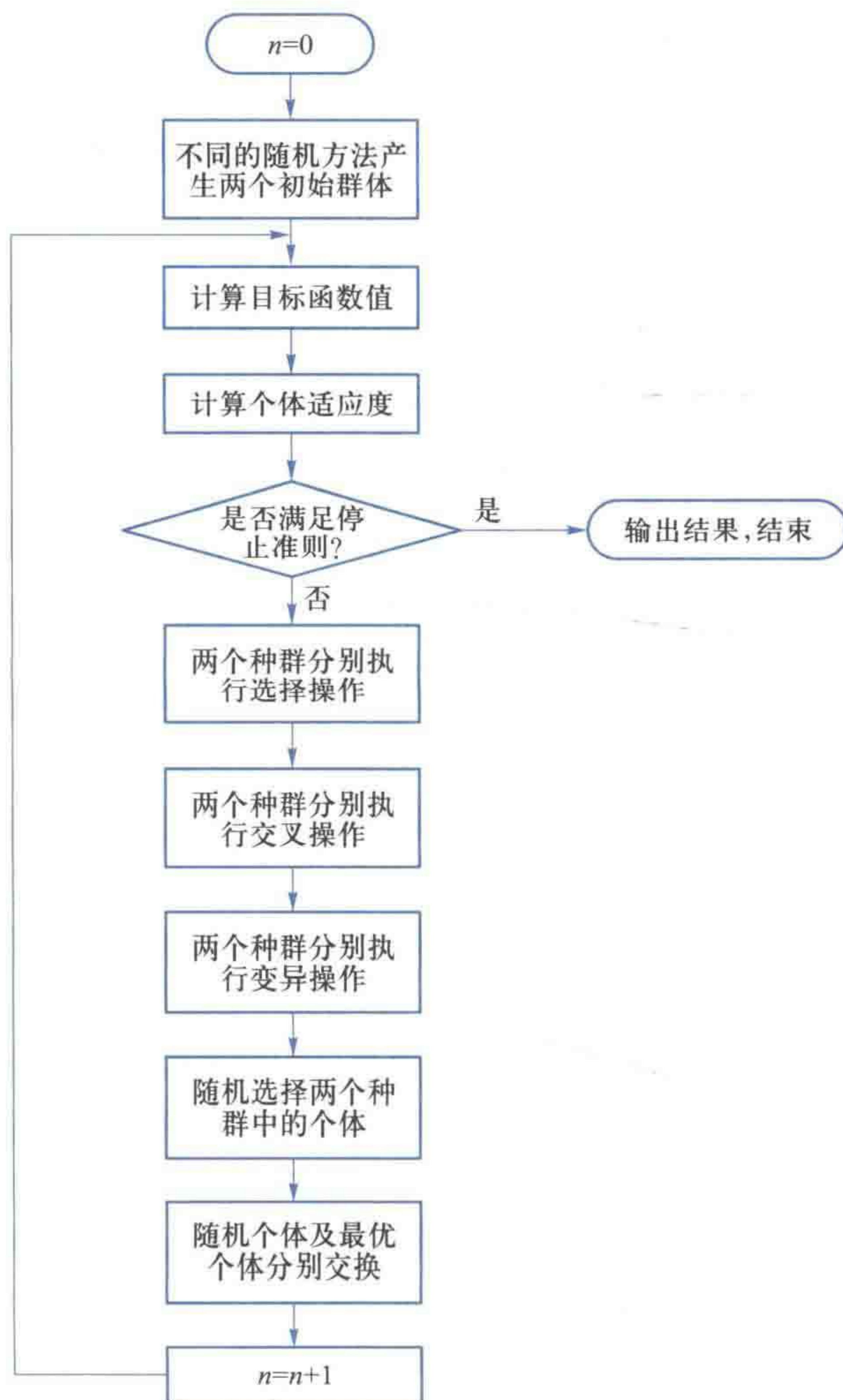


图 6.5 双种群遗传算法程序流程

6.3.3 自适应遗传算法

1. 基本思想

遗传算法的交叉概率 P_c 和变异概率 P_m 的选择是影响遗传算法行为和性能的关键参数,直接影响算法的收敛性。 P_c 越大,新个体产生的速度就越快;然而, P_c 过大时遗传模式被破坏的可能性也越大,使得具有高适应度的个体结构很快被破坏;但是如果 P_c 过小,会使搜索过程缓慢,以致停滞不前。对于变异概率 P_m ,如果 P_m 过小,就不易产生新的个体结构;如果 P_m 取值过大,则遗传算法变成了纯粹的随机搜索算法。针对不同的优化问题,需要反复实验来确定 P_c 和 P_m 。这是一件烦琐的工作,而且很难找到适应于每个问题的最佳值。

Srinivas M, Patnaik L M 等在 1994 年提出一种自适应遗传算法 (adaptive genetic algorithm, AGA)。自适应遗传算法是使交叉概率 P_c 和变异概率 P_m 能够随适应度变化自动改变。当种群各个体适应度趋于一致或者趋于局部最优时,使 P_c 和 P_m 增加,以跳出局部最优;而当群体适应度比较分散时,使 P_c 和 P_m 减少,以利于优良个体的生存。同时,对于适应度高于群体平均适应度的个体,选择较低的 P_c 和 P_m ,使得该解得以保护进入下一代;对低于平均适应度的个体,选择较高的 P_c 和 P_m 值,使该解被淘汰。因此,自适应的 P_c 和 P_m 能够提供相对某个解的最佳 P_c 和 P_m 。自适应遗传算法在保持群体多样性的同时,保证遗传算法的收敛性。

2. 自适应遗传算法的步骤

Step1 编码/解码设计同基本遗传算法。

Step2 用初始种群产生的一些方法,产生 N (N 是偶数) 个候选解,组成初始解集。

Step3 定义适应度函数,计算适应度 f_i 。

Step4 按照轮盘赌规则选择 N 个个体,计算群体的平均适应度 f_{avg} 和最大适应度 f_{max} 。

Step5 将群体中的各个个体随机搭配成对,共组成 $N/2$ 对,对每一对个体,按照自适应公式计算自适应交叉概率 P_c ,以 P_c 为交叉概率进行交叉操作,即随机产生 $R(0,1)$,如果 $R < P_c$ 则对该对染色体进行交叉操作。

Step6 对于群体中的所有个体,共 N 个,按照自适应变异公式计算自适应变异概率 P_m ,以 P_m 为变异概率进行变异操作,即随机产生 $R(0,1)$,如果 $R < P_m$ 则对该染色体进行变异操作。

Step7 计算由交叉和变异生成新个体的适应度,新个体与父代一起构成新群体。

Step8 判断是否达到预定的迭代次数,是则结束寻优过程,否则转 Step4。

3. 自适应的交叉概率与变异概率

在自适应遗传算法中, P_c 和 P_m 按照以下公式进行调整

$$P_c = \begin{cases} \frac{k_1(f_{max} - f')}{f_{max} - f_{avg}} & f' > f_{avg} \\ k_2 & f' \leq f_{avg} \end{cases} \quad (6.15)$$

$$P_m = \begin{cases} \frac{k_3(f_{\max} - f)}{f_{\max} - f_{\text{avg}}} & f > f_{\text{avg}} \\ k_4 & f \leq f_{\text{avg}} \end{cases} \quad (6.16)$$

其中, $f_{\max}$ 是群体中最大的适应度值; f_{avg} 是每代群体的平均适应度值; f' 是要交叉的两个个体中较大的适应度值; f 是要变异的个体的适应度值。 k_1, k_2, k_3, k_4 是 $(0,1)$ 之间的常数。

6.4 遗传算法的应用

下面以生产调度这个典型的大规模优化求解问题为例, 介绍遗传算法的应用。由于生产调度问题的解容易进行编码, 而且, 遗传算法可以处理大规模问题, 所以, 遗传算法成为求解生产调度问题的重要方法之一。

1. 流水车间调度问题

流水车间调度问题 (flow-shop scheduling problem, FSP) 是与城市不对称情况下的旅行商问题难度相当的同一类型的 NP 完全问题中最困难的问题之一。自从 Johnson 在 1954 年发表第一篇关于 FSP 的文章以来, FSP 引起了许多学者的关注。整数规划和分支定界法是寻求最优解的常用方法, 但 FSP 是 NP 完全问题, 对于一些大规模甚至中等规模的问题, 整数规划和分支定界方法仍是很困难的。在多数情况下很难用数学方法求解生产调度问题, 数学计算和智能算法的结合往往是有效的。

FSP 一般可以描述为 n 个工件要在 m 台机器上加工, 每个工件需要经过 m 道工序, 每道工序要求不同的机器, n 个工件在 m 台机器上的加工顺序相同。工件在机器上的加工时间是给定的, 设为 t_{ij} ($i=1, \dots, n; j=1, \dots, m$)。问题的目标是确定 n 个工件在每台机器上的最优加工顺序, 使最大流程时间达到最小。

对该问题常常做如下假设:

- ① 每个工件在机器上的加工顺序是给定的;
- ② 每台机器同时只能加工一个工件;
- ③ 一个工件不能同时在不同的机器上加工;
- ④ 工序不能预定;
- ⑤ 工序的准备时间与顺序无关, 且包含在加工时间中;
- ⑥ 工件在每台机器上的加工顺序相同, 且是确定的。

令 $c(j_i, k)$ 表示工件 j_i 在机器 k 上的加工完工时间, $\{j_1, j_2, \dots, j_n\}$ 表示工件的调度, 那么, 对于无限中间存储方式, n 个工件、 m 台机器的流水车间调度问题的完工时间可表示为

$$\begin{aligned} c(j_1, 1) &= t_{j_1 1} \\ c(j_1, k) &= c(j_1, k-1) + t_{j_1 k} \quad k = 2, \dots, m \\ c(j_i, 1) &= c(j_{i-1}, 1) + t_{j_i 1} \quad i = 2, \dots, n \end{aligned} \quad (6.17)$$

$$c(j_i, k) = \max \{ c(j_{i-1}, k), c(j_i, k-1) \} + t_{j_i k}$$

$$i = 2, \dots, n; k = 2, \dots, m$$

最大流程时间为

$$c_{\max} = c(j_n, m) \quad (6.18)$$

调度目标为确定 $\{j_1, j_2, \dots, j_n\}$, 使得 $c_{\max}$ 最小。

2. 求解 FSP 的遗传算法设计

由于遗传算法所固有的全局搜索与收敛特性, 由它得到的次优解往往优于传统方法得到的局部极值解, 加之搜索效率比较高, 因而被认为是一种切实有效的方法, 得到了日益广泛的研究。

下面介绍遗传算法求解 FSP 的编码与适应度函数的设计。

(1) FSP 的编码方法

对于调度问题, 通常不采用二进制编码, 而使用实数编码。将各个生产任务编码为相应的整数变量。一个调度方案是生产任务的一个排列, 其排列中每个位置对应于每个带编号的任务。GA 算法根据一定评价函数, 求出最优的排列。

对于 FSP, 最自然的编码方式是用染色体表示工件的顺序, 例如, 对于有四个工件的 FSP, 第 k 个染色体 $v_k = \{1, 2, 3, 4\}$, 表示工件的加工顺序为: j_1, j_2, j_3, j_4 。

(2) FSP 的适应度函数

令 $c_{\max}^k$ 表示 k 个染色体 v_k 的最大流程时间, 那么, FSP 的适应度函数取为

$$eval(v_k) = \frac{1}{c_{\max}^k} \quad (6.19)$$

3. 求解 FSP 的遗传算法实例

例 6.1 由 Ho 和 Chang(1991) 给出的 5 个工件、4 台机器问题的加工时间如表 6.2 所示。

表 6.2 加工时间表

工件 j	t_{j_1}	t_{j_2}	t_{j_3}	t_{j_4}
1	31	41	25	30
2	19	55	3	34
3	23	42	27	6
4	13	22	14	13
5	33	5	57	19

为了便于比较, 先用穷举法求得最优解为 4-2-5-1-3, 加工时间为 213; 最劣解为 1-4-2-3-5, 加工时间为 294; 平均解的加工时间为 265。下面用遗传算法求解。选择交叉概率 $P_c = 0.6$, 变异概率 $P_m = 0.1$, 种群规模为 20, 迭代次数 $N = 50$ 。运算结果如表 6.3 和图 6.6~图 6.8 所示。图 6.8 中的三元组 (i, j, k) 表示第 i 个工件的第 j 道工序在第 k 台机器上加工。

可见,用遗传算法绝大部分都能找到最优解,即使没有找到最优解,找到的解中最差的也有 221,比起平均解的加工时间 265 还是要好很多。这说明:遗传算法一般能找到全局最优解,至少能找到比较好的解。

表 6.3 遗传算法运行的结果

总运行次数	最好解	最坏解	平均	最好解的频率	最好解的平均代数
20	213	221	213.95	0.85	12

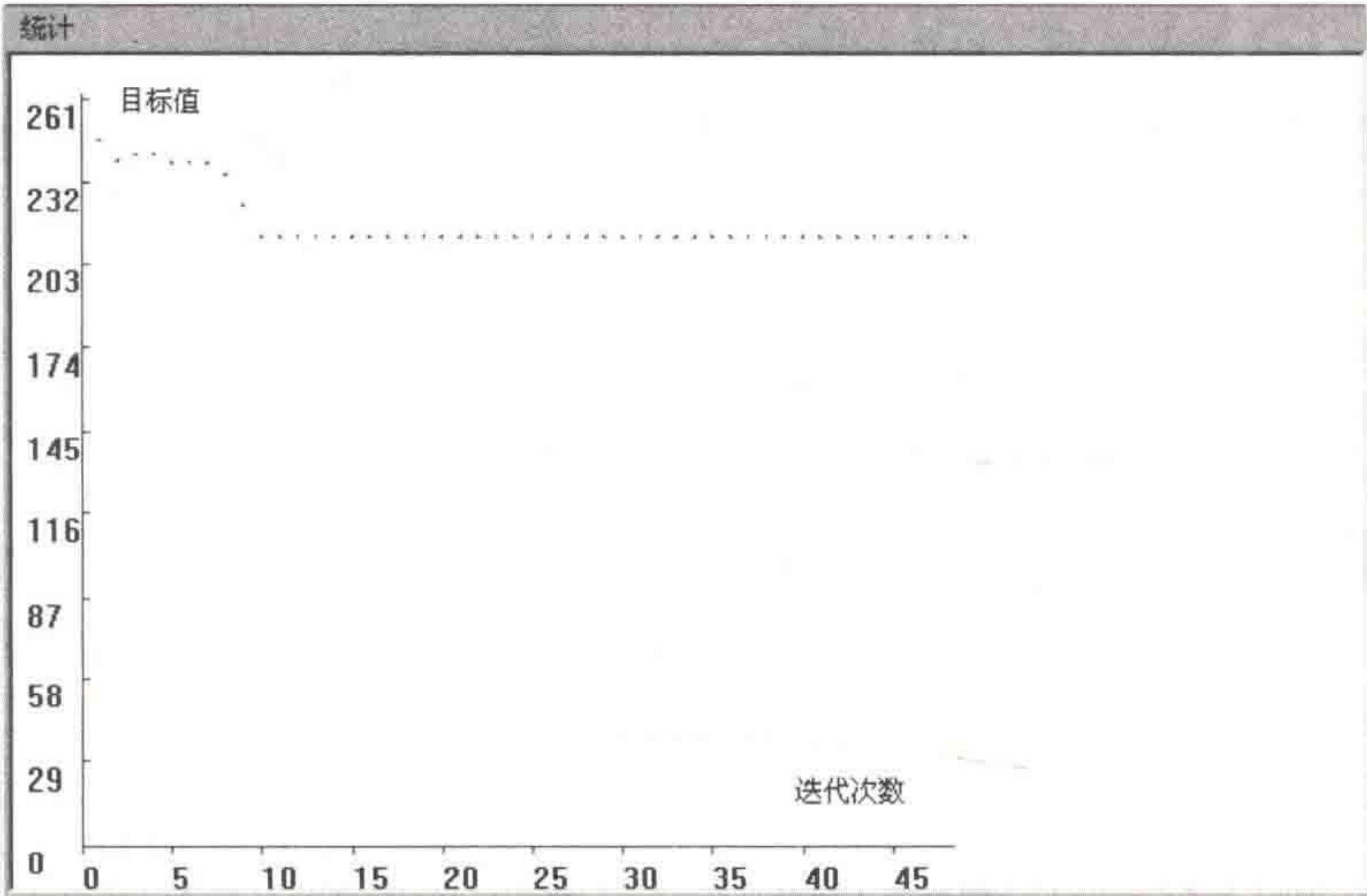


图 6.6 最优解收敛图

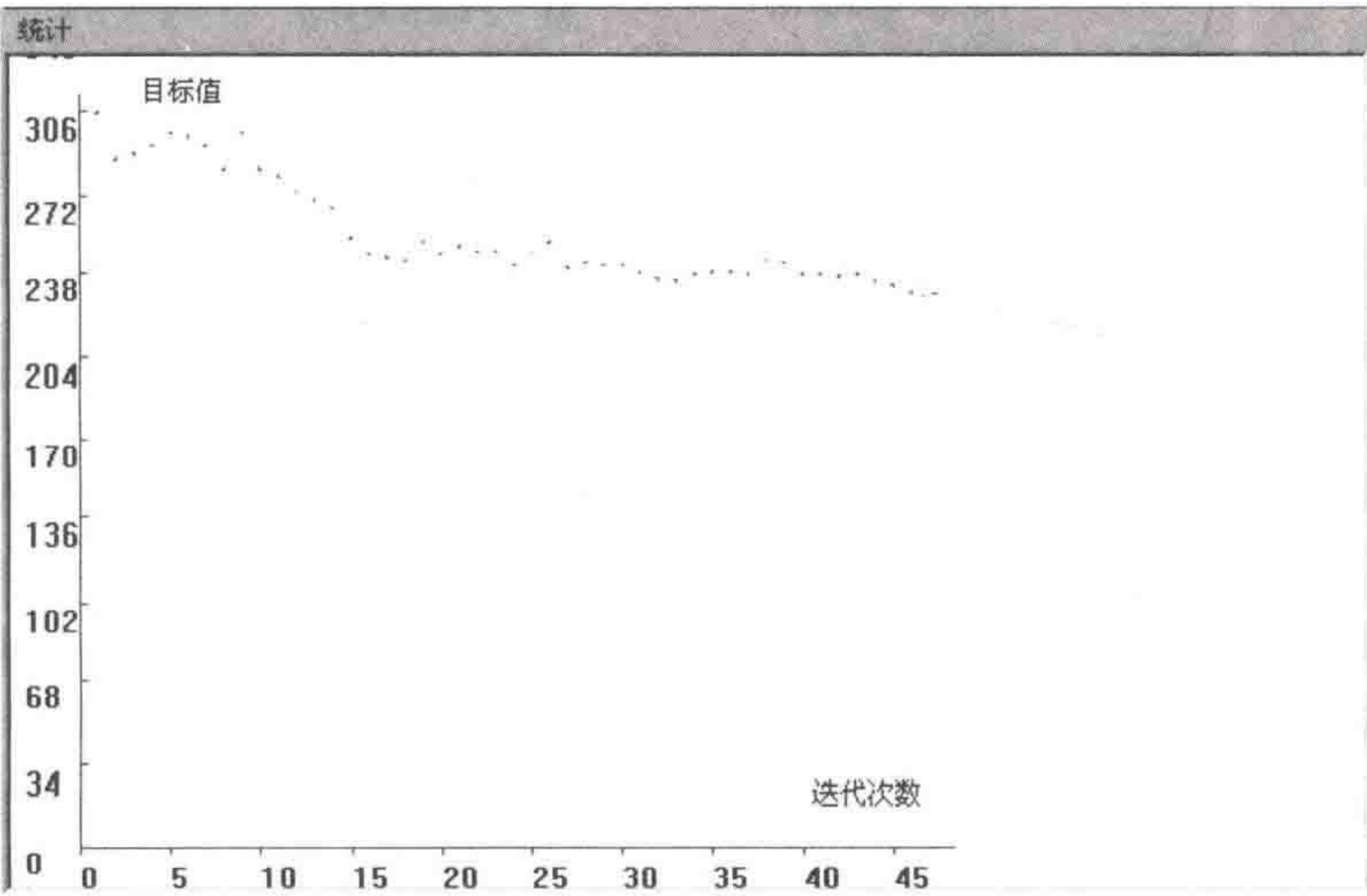


图 6.7 平均值收敛图

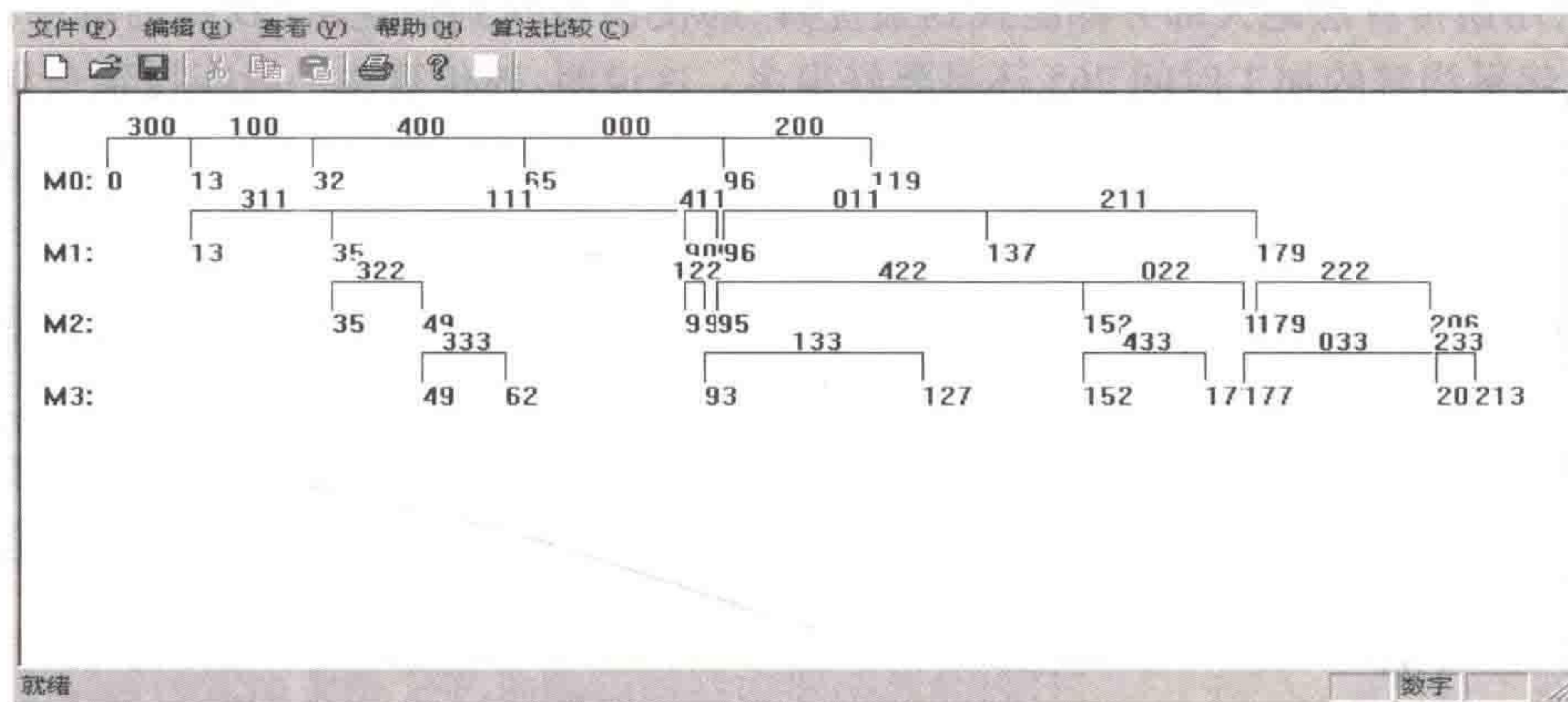


图 6.8 机器甘特图

6.5 群智能算法产生的背景

在众多智能计算方法中,受动物群体智能启发的算法称为群智能算法,如图 6.9 所示。

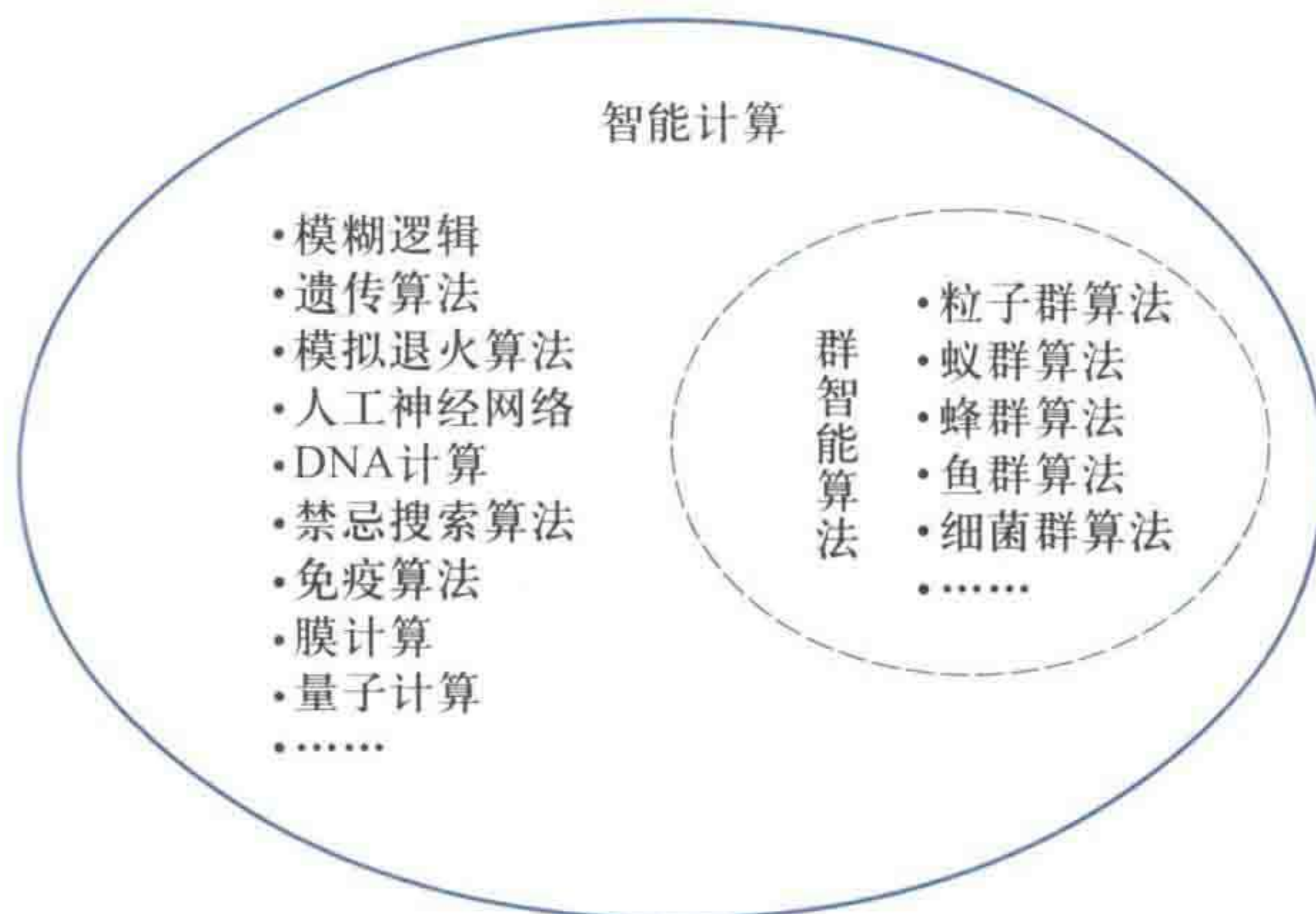


图 6.9 智能计算

自然界中有许多现象令人惊奇,如蚂蚁搬家、鸟群觅食、蜜蜂筑巢等,这些现象不仅吸引生物学家去研究,也让计算机学家痴迷。

鸟群的排列看起来似乎是随机的,其实它们有着惊人的同步性,这种同步性使得鸟群的整体运动非常流畅。有几位科学家对鸟群的飞行进行了计算机仿真,他们让每个个体按照特定的规则飞行,形成鸟群整体的复杂行为。模型成功的关键在于对个体间距离的操作,也就是说群体行为的同步性是因为个体努力维持自身与邻居之间的距离为最优,为此每个个体必须知道自身位置和邻居的信息。生物社会学家 E. O. Wilson 也曾说过“至少从理论上,在搜索食物

的过程中群体中的个体成员可以得益于所有其他成员的发现和先前的经历。当食物源不可预测地零星分布时,这种协作带来的优势是决定性的,远大于对食物的竞争带来的劣势。”

这些在由简单个体组成的群落与环境以及个体之间的互动行为,称为“群体智能”(swarm intelligence, SI)。在计算智能领域,群智能算法包括:粒子群优化算法、蚁群算法和人工免疫算法等。粒子群优化算法起源于对简单社会系统的模拟。最初设想是用粒子群优化算法模拟鸟群觅食的过程,但后来发现它是一种很好的优化工具。蚁群算法是对蚂蚁群采集食物过程的模拟,已经成功地运用在很多离散优化问题上。图 6.10 表示了生物学上的现象与对应的仿生智能计算的关系。

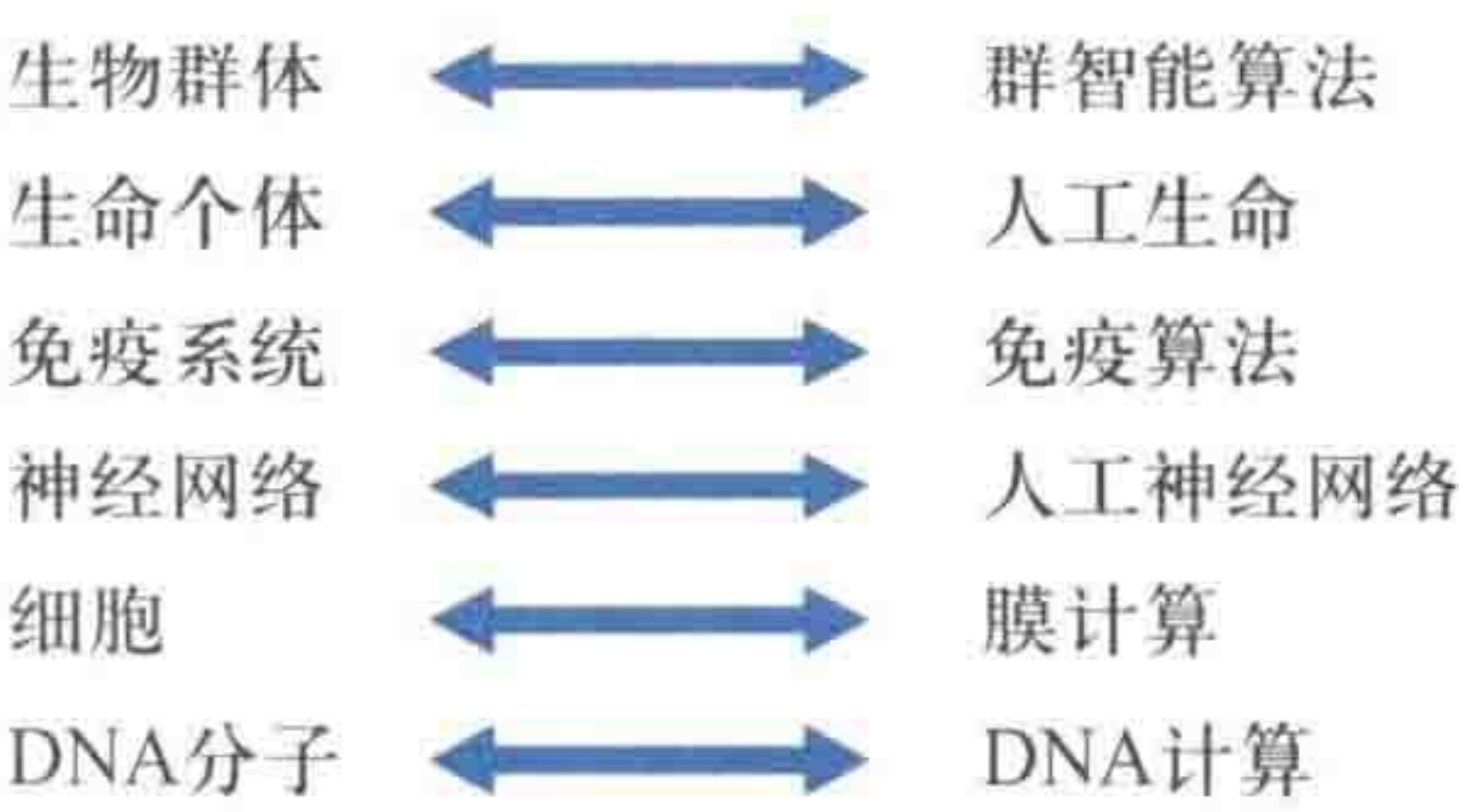


图 6.10 生物层次与仿生智能计算的对应关系

群智能算法与进化算法既有相同之处,也有明显的不同之处。相同之处:首先,EC 和 SI 都是受自然现象的启发,基于抽取出的简单自然规则而发展出的计算模型。其次,两者又都是基于种群的方法,且种群中的个体之间、个体与环境之间存在相互作用。最后,两者都是一种元启发式随机搜索方法。不同之处:EC 方法强调种群的达尔文主义的进化模型,而 SI 优化方法则注重对群体中个体之间的相互作用与分布式协同的模拟。

6.6 粒子群优化算法及其应用

6.6.1 粒子群优化算法的基本原理

粒子群优化 (particle swarm optimization, PSO) 算法是美国普渡大学的 Kennedy 和 Eberhart 受到他们早期对鸟类群体行为研究结果的启发,于 1995 年在 IEEE International Conference on Neural Networks 国际会议上提出的一种仿生优化算法,他们利用并改进了生物学家的生物群体模型,使粒子能够飞向解空间并在最优解处降落。PSO 算法是一种基于群智能理论的全局优化算法,通过群体中粒子间的合作与竞争产生的群体智能指导优化搜索。

PSO 算法与其他进化算法相似,也是基于群体的,根据对环境的适应度将群体中的个体移动到好的区域,然而它不像其他演化算法那样对个体使用演化算子,而是将每个个体看作 n 维搜索空间中一个没有体积质量的粒子,在搜索空间中以一定的速度飞行。

在粒子群优化算法中,在 n 维连续搜索空间中,对粒子群中的第 $i(i=1,2,\cdots,m)$ 个粒子,定义 n 维当前位置向量 $\mathbf{x}^i(k)=[x_1^i \ x_2^i \ \cdots \ x_n^i]^T$ 表示搜索空间中粒子的当前位置, n 维最优

位置向量 $\mathbf{p}^i(k) = [p_1^i \ p_2^i \ \cdots \ p_n^i]^T$ 表示该粒子至今所获得的具有最优适应度 $f_p^i(k)$ 的位置, n 维速度向量 $\mathbf{v}^i(k) = [v_1^i \ v_2^i \ \cdots \ v_n^i]^T$ 表示该粒子的搜索方向。

每个粒子经历过的最优位置 (pbest) 记为 $\mathbf{p}^i(k) = [p_1^i \ p_2^i \ \cdots \ p_n^i]^T$, 群体经历过的最优位置 (gbest) 记为 $\mathbf{p}^g(k) = [p_1^g \ p_2^g \ \cdots \ p_n^g]^T$, 则基本的 PSO 算法为

$$v_j^i(k+1) = \omega(k)v_j^i(k) + \varphi_1 \text{rand}(0, a_1)(p_j^i(k) - x_j^i(k)) + \varphi_2 \text{rand}(0, a_2)(p_j^g(k) - x_j^i(k)) \quad (6.20a)$$

$$x_j^i(k+1) = x_j^i(k) + v_j^i(k+1) \quad (6.20b)$$

$$i = 1, 2, \dots, m; \quad j = 1, 2, \dots, n$$

其中, ω 是惯性权重因子; φ_1, φ_2 是加速度常数, 均为非负值; $\text{rand}(0, a_1)$ 和 $\text{rand}(0, a_2)$ 为 $[0, a_1]$ 、 $[0, a_2]$ 范围内的具有均匀分布的随机数; a_1 与 a_2 为相应的控制参数。

式(6.20a)右边的第一部分是粒子在前一时刻的速度对下一时刻速度的影响; 第二部分为个体“认知(cognition)”分量, 表示粒子本身的思考, 将现有的位置和曾经经历过的最优位置相比。第三部分是群体“社会(social)”分量, 表示粒子间的信息共享与相互合作。 φ_1 和 φ_2 分别控制个体认知分量和群体社会分量相对贡献的学习率。引入 $\text{rand}(0, a_1)$ 和 $\text{rand}(0, a_2)$ 将增加认知和社会搜索方向的随机性和算法多样性。

基于学习率 φ_1, φ_2 , Kennedy 给出以下 4 种类型的 PSO 模型。

- ① 若 $\varphi_1 > 0, \varphi_2 > 0$, 则称该算法为 PSO 全模型。
- ② 若 $\varphi_1 > 0, \varphi_2 = 0$, 则称该算法为 PSO 认知模型。
- ③ 若 $\varphi_1 = 0, \varphi_2 > 0$, 则称该算法为 PSO 社会模型。
- ④ 若 $\varphi_1 = 0, \varphi_2 > 0$ 且 $g \neq i$, 则称该算法为 PSO 无私模型。

标准的粒子群优化算法分为两个版本: 全局版和局部版。上面介绍的是全局版粒子群优化算法。局部版与全局版的差别在于: 用局部领域内最优邻居的状态代替整个群体的最优状态。局部版的收敛速度比较快, 但容易陷入局部极值点, 而全局版搜索到的解可能更优, 但速度较慢。

粒子群优化算法的流程如下。

- ① 初始化每个粒子, 即在允许范围内随机设置每个粒子的初始位置和速度。
 - ② 评价每个粒子的适应度, 计算每个粒子的目标函数。
 - ③ 设置每个粒子经历过的最好位置 p^i 。对每个粒子, 将其适应度与其经历过的最好位置 p^i 进行比较, 如果优于 p^i , 则将其作为该粒子的最好位置 p^i 。
 - ④ 设置全局最优值 p^g 。对每个粒子, 将其适应度与群体经历过的最好位置 p^g 进行比较, 如果优于 p^g , 则将其作为当前群体的最好位置 p^g 。
 - ⑤ 根据式(6.20)更新粒子的速度和位置。
 - ⑥ 检查终止条件。如果未达到设定条件(预设误差或者迭代的次数), 则返回第②步。
- 粒子群优化算法的流程如图 6.11 所示。

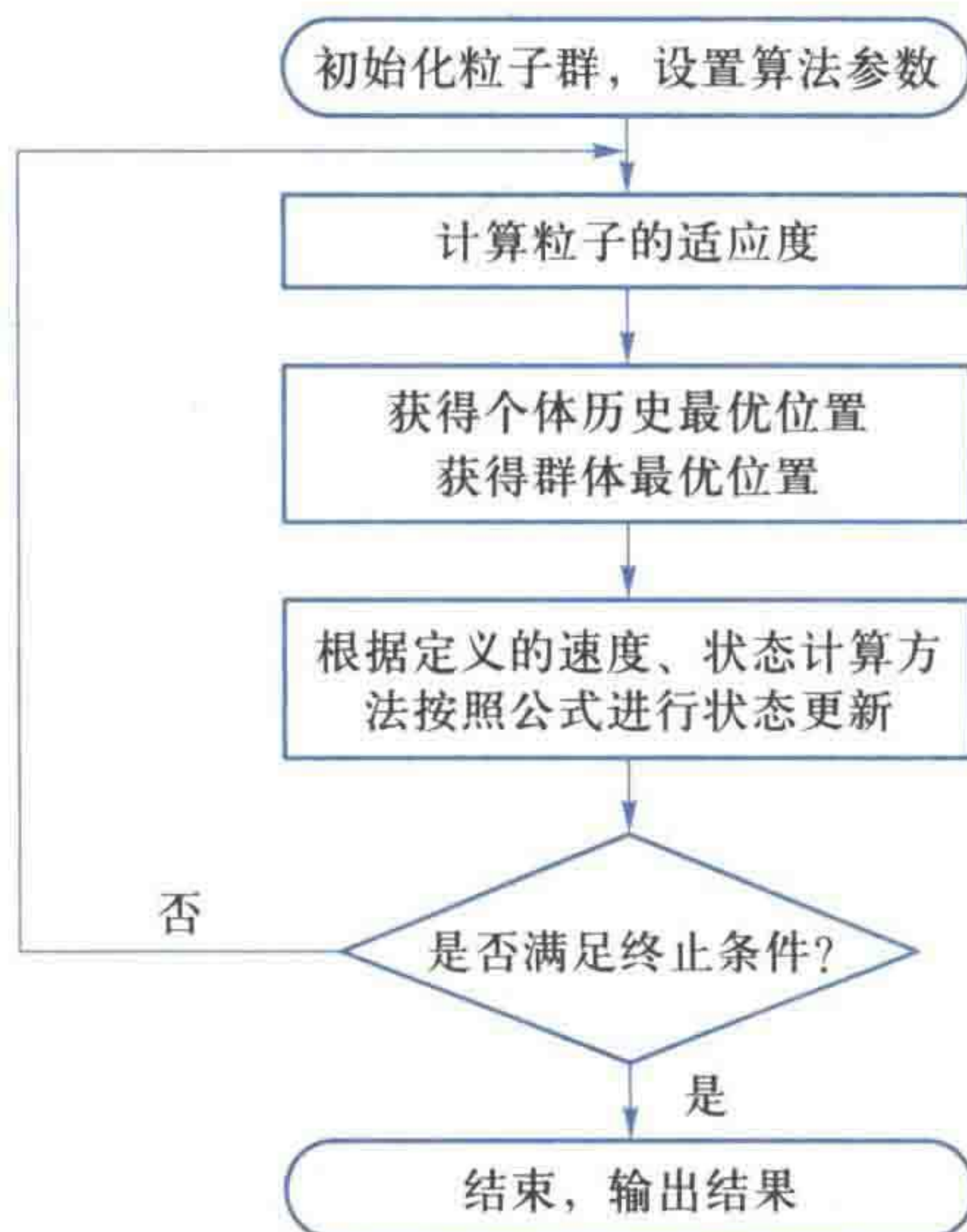


图 6.11 粒子群优化算法流程

6.6.2 粒子群优化算法的参数分析

1. PSO 算法的参数

PSO 算法的参数包括:群体规模 m , 惯性权重 ω , 加速度 φ_1, φ_2 , 最大速度 $V_{\max}$, 最大代数 $G_{\max}$ 。

(1) 最大速度 $V_{\max}$

对速度 v_i , 算法中有最大速度 $V_{\max}$ 作为限制, 如果当前粒子的某维速度大于最大速度 $V_{\max}$, 则该维的速度就被限制为最大速度 $V_{\max}$ 。

最大速度 $V_{\max}$ 决定当前位置与最好位置之间的区域的分辨率(或精度)。如果 $V_{\max}$ 太高, 粒子可能会飞过好的解; 如果 $V_{\max}$ 太小, 粒子容易陷入局部最优值。

(2) 权重因子

在 PSO 算法中有 3 个权重因子: 惯性权重 ω , 加速度常数 φ_1, φ_2 。

惯性权重 ω 使粒子保持运动惯性, 使其有扩展搜索空间的趋势, 并有能力搜索新的区域。

加速度常数 φ_1 和 φ_2 代表将每个粒子推向 P_i 和 P_g 位置的统计加速度项的权重。低的值允许粒子在被拉回之前可以在目标区域外徘徊, 而高的值则导致粒子突然冲向或者越过目标区域。

2. 位置更新方程中各部分的影响

对于式(6.20), 如果只有第一部分, 而没有后两部分, 即 $\varphi_1 = \varphi_2 = 0$, 则粒子将一直以当前的速度飞行, 直到达边界。由于它只能搜索有限的区域, 所以很难找到最优解。

如果没有第一部分, 即 $\omega = 0$, 则速度只取决于粒子当前位置和其历史最好位置 p^i 和 p^g , 速度本身没有记忆性。假设一个粒子位于全局最好位置, 它将保持静止。而其他粒子则飞向它。

本身最好位置 p^i 和全局最好位置 p^g 的加权中心。在这种条件下,粒子群将收敛到当前的全局最好位置,更像一个局部算法。在加上第一部分后,粒子有扩展搜索空间的趋势,即第一部分有全局搜索能力。这也使得 ω 的作用为针对不同的搜索问题,调整算法全局和局部搜索能力的平衡。

如果没有第二部分,即 $\varphi_1=0$,则粒子没有认知能力,也就是“只有社会模型”。在粒子的相互作用下,有能力达到新的搜索空间。它的收敛速度比标准版本更快,但对复杂问题,则比标准版本更容易陷入局部最优解。

如果没有第三部分,即 $\varphi_2=0$,则粒子间没有社会共享信息,也就是“只有认知”模型。因为个体间没有交互,一个规模为 M 的群体等价于 M 个单个粒子的运行,因而得到最优解的概率非常小。

3. 参数设置

早期的实验将 ω 固定为 1.0, φ_1 和 φ_2 固定为 2.0,因此 $V_{\max}$ 成为唯一需要调节的参数,通常设为每维变化范围 10%~20%。Suganthan 的实验表明, φ_1 和 φ_2 为常数时可以得到较好的解,但不一定必须为 2。

这些参数也可以通过模糊系统进行调节。Shi 和 Eberhart 提出一个模糊系统来调节 ω ,该系统包括 9 条规则,有两个输入和一个输出。一个输入为当前代的全局最好适应值,另一个输入为当前的 ω ;输出为 ω 的变化。每个输入和输出定义了 3 个模糊集,结果显示该方法能显著提高平均适应值。

粒子群优化算法初始群体的产生方法与遗传算法类似。可以随机产生,也可以根据问题的固有知识产生。群体的初始化虽然也是影响算法性能的一个方面,但 Angeline 对不对称的初始化进行了实验,发现 PSO 只是略微受影响。粒子群优化算法的种群的大小根据问题的规模而定,同时要考虑运算的时间。

粒子的适应度函数根据具体问题而定,将目标函数转换成适应度函数的方法与遗传算法类似。

在基本的粒子群优化算法中,粒子的编码使用实数编码方法。这种编码方法在求解连续的函数优化问题中十分方便,同时对粒子的速度求解与粒子的位置更新也很自然。

6.6.3 粒子群优化算法的应用领域

粒子群优化算法已在诸多领域得到应用,简单归纳如下。

① 神经网络训练。利用 PSO 来训练神经网络,将遗传算法与 PSO 结合来设计递归/模糊神经网络等。利用 PSO 设计神经网络是一种快速、高效并具有潜力的方法。

② 化工系统领域。利用 PSO 求解苯乙烯聚合反应的最优稳态操作条件,获得了最大的转化率和最小的聚合体分散性;使用 PSO 来估计在化工动态模型中产生不同动态现象(如周期振荡、双周期振荡、混沌等)的参数区域,仿真结果显示提高了传统动态分叉分析的速度;利用 GP 和 PSO 辨识最优生产过程模型及其参数。

③ 电力系统领域。将 PSO 用于最低成本发电扩张 GEP 问题,结合罚函数法解决带有强约束的组合优化问题;利用 PSO 优化电力系统稳压器参数;利用 PSO 解决考虑电压安全的无功功率和电压控制问题;利用 PSO 算法解决满足发电机约束的电力系统经济调度问题;利用 PSO 解决满足开、停机热备约束的机组调度问题。

④ 机械设计领域。利用 PSO 优化设计碳纤维强化塑料;利用 PSO 对降噪结构进行最优化设计。

⑤ 通信领域。利用 PSO 设计电路;将 PSO 用于光通信系统的 PMD 补偿问题。

⑥ 机器人领域。利用 PSO 和基于 PSO 的模糊控制器对可移动式传感器进行导航;利用 PSO 求解机器人路径规划问题。

⑦ 经济领域。利用 PSO 求解博弈论中的均衡解;利用 PSO 和神经网络解决最大利益的股票交易决策问题。

⑧ 图像处理领域。离散 PSO 方法解决多边形近似问题,提高多边形近似结果;利用 PSO 对用于放射治疗的模糊认知图的模型参数进行优化;利用基于 PSO 的微波图像法来确定电磁散射体的绝缘特性;利用结合局部搜索的混合 PSO 算法对生物医学图像进行配准。

⑨ 生物信息领域。利用 PSO 训练隐马尔可夫模型来处理蛋白质序列比对问题,克服利用 Baum-Welch 算法训练 HMMs 时容易陷入局部极小的缺点;利用基于自组织映射和 PSO 的混合聚类方法来解决基因聚类问题。

⑩ 医学领域。离散 PSO 选择 MLR 模型和 PLS 模型的参数,并预测血管紧张素的对抗性。

⑪ 运筹学领域。基于可变领域搜索的 VNS 的 PSO,解决满足最小耗时指标的置换问题。

6.6.4 粒子群优化算法求解车辆路径问题

1. 车辆路径问题(VRP)的模型

车辆路径问题:假定配送中心最多可以用 $K(k=1,2,\dots,K)$ 辆车对 $L(i=1,2,\dots,L)$ 个客户进行运输配送, $i=0$ 表示仓库。每个车辆载重为 $b_k(k=1,2,\dots,K)$,每个客户的需求为 $d_i(i=1,2,\dots,L)$,客户 i 到客户 j 的运输成本为 c_{ij} (可以是距离、时间、费用等)。定义如下变量:

$$y_{ik} = \begin{cases} 1 & \text{客户 } i \text{ 由车辆 } k \text{ 配送} \\ 0 & \text{其他} \end{cases}$$

$$x_{ijk} = \begin{cases} 1 & \text{车辆 } k \text{ 从 } i \text{ 访问 } j \\ 0 & \text{其他} \end{cases}$$

则车辆路径问题的数学模型如下表示:

$$\min \sum_{k=1}^K \sum_{i=0}^L \sum_{j=0}^L c_{ij} x_{ijk} \quad (6.21a)$$

$$\sum_{i=1}^L d_i y_{ik} \leq b_k \quad \forall k \quad (6.21b)$$

$$\sum_{k=1}^K y_{ik} = 1 \quad \forall i \quad (6.21c)$$

$$\sum_{i=1}^L x_{ijk} = y_{jk} \quad \forall j, k \quad (6.21d)$$

$$\sum_{j=1}^L x_{ijk} = y_{ik} \quad \forall i, k \quad (6.21e)$$

$$\sum_{i,j \in S \times S} x_{ijk} \leq |S| - 1 \quad S \in \{1, 2, \dots, L\} \quad \forall k \quad (6.21f)$$

$$x_{ijk} = 0 \text{ 或 } 1 \quad \forall i, j, k \quad (6.21g)$$

$$y_{ik} = 0 \text{ 或 } 1 \quad \forall i, k \quad (6.21h)$$

约束(6.21b)为每辆车的能力约束。约束(6.21c)保证每个客户都被服务。约束(6.21d)、(6.21e)保证客户是仅被一辆车访问。约束(6.21f)消除子回路。约束(6.21g)、(6.21h)表示变量的取值范围。

2. 编码与初始种群

对这类组合优化问题,编码方式、初始解的设置对问题的求解都有很大的影响。采用常用的自然数编码方式。对于 K 辆车和 L 个客户的问题,用从1到 L 的自然数随机排列来产生一组解 $X = (x_1, x_2, \dots, x_L)$ 。表6.4中,A-n32-k5表示32个用户,5辆配送车的实验例子,其余类似。

3. 实验结果

粒子群优化算法的各个参数设置如下:种群规模 $P=50$,迭代次数 $N=1\,000$, w_1 的初始值为1,随着迭代的进行,线性减小到0, $\varphi_1 = \varphi_2 = 1.4$, $|V_{\max}| \leq 100$ 。优化结果如表6.4所示。其中, $\text{dev}(\%)$ 为与最优解的偏差。

表 6.4 优化结果

实例	PSO	
	best	dev/%
A-n32-k5	829	5.73
A-n33-k5	705	6.65
A-n34-k5	832	6.94
A-n39-k6	872	6.08
A-n44-k6	1 016	8.49
A-n46-k7	977	6.89
A-n54-k7	1 205	3.26
A-n60-k9	1 476	9.01
A-n69-k9	1 275	10.00
A-n80-k10	1 992	12.98

6.7 蚁群算法及其应用

蚁群算法(ant colony optimization, ACO)是由意大利科学家 Marco Dorigo 等受蚂蚁觅食行为的启发,在 20 世纪 90 年代初提出来的。它是继模拟退火算法、遗传算法、禁忌搜索算法、人工神经网络算法等启发式搜索算法后的又一种应用于组合优化问题的启发式搜索算法。研究表明,蚁群优化算法在解决离散组合优化方面具有良好的性能,并在多方面得到应用。

M. Dorigo, V. Maniezzo 等人在观察蚂蚁觅食习性时发现,蚂蚁总能找到巢穴与食物之间的最短路径。经研究发现,蚁群觅食时总存在信息素(pheromone)跟踪和信息素遗留两种行为,即一方面蚂蚁会按照一定的概率沿着信息素较强的路径觅食,另一方面,蚂蚁会在走过的路上释放信息素,使得在一定范围内的其他蚂蚁能够觉察到并影响它们的行为。当一条路上的信息素越来越多,后来的蚂蚁选择这条路的概率也越来越大,从而进一步增加该路径的信息素强度,而其他路径上蚂蚁越来越少时,这条路径上的信息素会随着时间的推移逐渐减弱。这种选择过程称为蚂蚁的自催化过程,其原理是一种正反馈机制,所以蚂蚁系统也称为增强型学习系统。

20 世纪 90 年代后期,这种算法逐渐引起了很多研究者的注意,他们对算法做了各种改进并应用到其他领域。Dorigo 等提出了蚁群优化的算法框架,所有符合蚁群优化描述框架的蚂蚁算法都可称之为蚁群优化算法,或简称为蚁群算法。Gutjahr 首先证明了 ACO 类算法的收敛性。

6.7.1 基本蚁群算法模型

蚁群优化算法的第一个应用是著名的旅行商问题(TSP),M. Dorigo 等人充分利用了蚁群搜索食物的过程与旅行商问题之间的相似性,通过人工模拟蚂蚁搜索食物的过程,即通过个体之间的信息交流与相互协作最终找到从蚁穴到食物源的最短路径,来求解旅行商问题。下面用旅行商问题阐明蚁群系统的模型。

设 m 是蚁群中蚂蚁的数量,给定 n 个城市的集合, $d_{xy}(x, y=1, \dots, n)$ 表示元素(城市) x 和元素(城市) y 之间的距离。欧几里得空间中, $d_{xy} = \sqrt{(X_x - X_y)^2 + (Y_x - Y_y)^2}$ 。 η_{xy} 表示能见度,称为启发信息函数,等于距离的倒数,即 $\eta_{xy} = \frac{1}{d_{xy}}$ 。 $b_x(t)$ 表示时刻 t 位于城市 x 的蚂蚁的个数,

$m = \sum_{x=1}^n b_x(t)$ 。 $\tau_{xy}(t)$ 表示 t 时刻在 xy 连线上残留的信息素。各条路径上初始时刻的信息素相等,为一个小的正常数,即 $\tau_{xy}(0) = C(\text{const})$ 。蚂蚁 $k(k=1, \dots, m)$ 在运动过程中,根据各条路径上的信息素和启发信息决定转移方向。每只蚂蚁在 t 时刻选择下一个城市,并在 $t+1$ 时刻到达那里。 $P_{xy}^k(t)$ 表示在 t 时刻蚂蚁 k 选择从元素(城市) x 转移到元素(城市) y 的概率。

$P_{xy}^k(t)$ 由信息素 $\tau_{xy}(t)$ 和局部启发信息 η_{xy} 共同决定,也称为随机比例规则(random

proportional rule)。

$$P_{xy}^k(t) = \begin{cases} \frac{[\tau_{xy}(t)]^\alpha [\eta_{xy}]^\beta}{\sum_{y \in allowed_k(x)} [\tau_{xy}(t)]^\alpha [\eta_{xy}]^\beta} & y \in allowed_k(x) \\ 0 & \text{其他} \end{cases} \quad (6.22)$$

其中, $allowed_k(x) = \{1, \dots, n\} - tabu_k(x)$, 表示蚂蚁 k 下一步允许选择的城市; $tabu_k(x)$ ($k=1, 2, \dots, m$) 记录蚂蚁 k 当前所走过的城市; α 是信息素启发式因子, 表示轨迹的相对重要性, 反映了残留信息浓度 $\tau_{xy}(t)$ 在指导蚁群搜索中的相对重要程度; β 是期望启发式因子。

α 值越大, 该蚂蚁越倾向于选择其他蚂蚁经过的路径, 该状态转移概率越接近于贪婪规则。当 $\alpha=0$ 时, 就不再考虑信息素水平, 算法就成为有多重起点的随机贪婪算法。而当 $\beta=0$ 时, 算法就成为纯粹的正反馈的启发式算法。

随着时间的推移, 以前留下的信息素逐渐挥发, 用参数 $1-\rho$ 表示信息素挥发程度, 其中 ρ 是信息素残留常数 ($0 < \rho < 1$)。蚂蚁完成一次循环, 各路径上信息素浓度挥发规则为

$$\tau_{xy}(t+1) = \rho\tau_{xy}(t) + \Delta\tau_{xy}(t) \quad (6.23)$$

蚁群在 t 到 $t+1$ 时刻的信息素浓度更新规则为

$$\Delta\tau_{xy}(t) = \sum_{k=1}^m \Delta\tau_{xy}^k(t) \quad (6.24)$$

其中, $\Delta\tau_{xy}(t)$ 为路径 (x, y) 上 t 到 $t+1$ 时刻信息素的增量; $\Delta\tau_{xy}^k(t)$ 为第 k 只蚂蚁 t 到 $t+1$ 时刻留在路径 (x, y) 上信息素的增量。

M. Dorigo 给出 $\Delta\tau_{xy}^k(t)$ 的三种不同模型。

第一种称为蚂蚁圈系统 (ant-cycle system)。单只蚂蚁所访问路径上的信息素浓度更新规则为

$$\Delta\tau_{xy}^k(t) = \begin{cases} \frac{Q}{L_k} & \text{若第 } k \text{ 只蚂蚁在本次循环中从 } x \text{ 到 } y \\ 0 & \text{否则} \end{cases} \quad (6.25)$$

其中, Q 为常数; L_k 为优化问题的目标函数值, 表示第 k 只蚂蚁在本次循环中所走路径的长度。根据具体算法的不同, $\Delta\tau_{xy}^k(t)$ 、 $\Delta\tau_{xy}(t)$ 、 $\tau_{xy}(t)$ 及 $P_{xy}^k(t)$ 的表达形式可以不同, 要根据具体问题而定。

第二种称为蚂蚁数量系统 (ant-quantity system):

$$\Delta\tau_{xy}^k(t) = \begin{cases} \frac{Q}{d_{xy}} & \text{若第 } k \text{ 只蚂蚁在本次循环中从 } x \text{ 到 } y \\ 0 & \text{否则} \end{cases} \quad (6.26)$$

第三种称为蚂蚁密度系统 (ant-density system):

$$\Delta\tau_{xy}^k(t) = \begin{cases} Q & \text{若第 } k \text{ 只蚂蚁在本次循环中从 } x \text{ 到 } y \\ 0 & \text{否则} \end{cases} \quad (6.27)$$

第一种模型利用的是整体信息,即蚂蚁完成一个循环后,更新所有路径上的信息,通常作为蚁群优化算法的基本模型。后两种模型利用的是局部信息,每走一步都要更新残留信息素的浓度,而非等到所有蚂蚁完成对所有 n 个城市的访问以后。

比较上述三种方法,蚂蚁圈系统的效果最好,这是因为它利用的是全局信息 Q/L_k ,而其余两种算法用的是局部信息 Q/d_{xy} 和 Q 。全局信息更新方法很好地保证了残留信息素不会无限累积。如果路径没有被选中,那么上面的残留信息素会随时间的推移而逐渐减弱,这使算法能“忘记”不好的路径。即使路径经常被访问也不会因为 $\Delta\tau_{xy}^k(t)$ 的累积,而产生 $\Delta\tau_{xy}^k(t) \gg \eta_{xy}(t)$ 使期望值的作用无法体现。这充分体现了算法中全局范围内较短路径(较好解)的生存能力,加强了信息正反馈性能,提高了系统搜索收敛的速度。因而,在蚁群算法中,通常采用蚂蚁圈系统作为基本模型。

6.7.2 蚁群算法的参数选择

从蚁群搜索最短路径的机理不难看到,算法中有关参数的不同选择对蚁群算法的性能有至关重要的影响,但其选取的方法和原则,目前尚没有理论上的依据,通常都是根据经验而定。

信息素启发因子 α 的大小则反映了蚁群在路径搜索中随机性因素作用的强度,其值越大,蚂蚁选择以前走过的路径的可能性越大,搜索的随机性减弱。当 α 过大时会使蚁群的搜索过早陷入局部最优。

期望值启发式因子 β 的大小反映了蚁群在路径搜索中先验性、确定性因素作用的强度,其值越大,蚂蚁在某个局部点上选择局部最短路径的可能性越大。虽然搜索的收敛速度得以加快,但蚁群在最优路径的搜索过程中随机性减弱,易于陷入局部最优。蚁群算法的全局寻优性能,首先要求蚁群的搜索过程必须有很强的随机性;而蚁群算法的快速收敛性能,又要求蚁群的搜索过程必须要有较高的确定性。因此, α 和 β 对蚁群算法性能的影响和作用相互配合、密切相关的。

蚁群算法与遗传算法等各种模拟进化算法一样,也存在着收敛速度慢、易于陷入局部最优等缺陷。而信息素挥发度 $1-\rho$ 直接关系到蚁群算法的全局搜索能力及其收敛速度。由于信息素挥发度 $1-\rho$ 的存在,当要处理的问题规模比较大时,会使那些从来未被搜索到的路径(可行解)上的信息量减小到接近于0,因而降低了算法的全局搜索能力。当 $1-\rho$ 过大时,也会使那些从未被搜索到的路径上的信息量减少到接近0,所以,以前搜索过的路径被再次选择的可能性也会过大,影响算法的随机性能和全局搜索能力。反之,减小信息素挥发度 $1-\rho$,虽然可以提高算法的随机性能和全局搜索能力,但又会使算法的收敛速度降低。

对于旅行商问题,单个蚂蚁在一次循环中所经过的路径,表现为问题的可行解集中的一个解, k 个蚂蚁在一次循环中所经过的路径,则表现为问题的可行解集中的一个子集。显然,子集越大(即蚁群数量多),蚁群算法的全局搜索能力以及算法的稳定性越高。但蚂蚁数目增大后,会使大量曾被搜索过的解(路径)上的信息素量的变化趋于平均,信息素正反馈的作用不明显,搜索的随机性虽然得到了加强,但收敛速度减慢。反之,子集较小(即蚁群数量少),特别是当

要处理的问题规模比较大时,会使那些从来未被搜索到的解(路径)上的信息素量减小到接近于0,搜索的随机性减弱,虽然收敛速度加快,但会使算法的全局性能降低,算法的稳定性差,容易出现过早停滞现象。

在蚂蚁圈系统模型中,总信息素量 Q 为蚂蚁循环一周时释放在所经过的路径上的信息素总量。总信息素量 Q 越大,则在蚂蚁已经走过的路径上信息素的累积越快,可以加强蚁群搜索时的正反馈性能,有助于算法的快速收敛。由于在蚁群算法中各个算法参数的作用实际上是紧密结合的,其中对算法性能起着主要作用的应该是信息素启发式因子 α 、期望启发式因子 β 和信息素残留常数 ρ 等三个参数。总信息素量 Q 对算法性能的影响则有赖于上述三个参数的配置,以及算法模型的选取。例如,在蚂蚁圈系统模型和蚂蚁密度系统模型中,总信息素量 Q 对算法性能的影响情况显然有较大的差异。同样,信息素的初始值 τ_0 对算法性能的影响不是很大。

6.7.3 蚁群算法的应用

柔性作业车间调度问题:某加工系统有6台机床,要加工4个工件,每个工件有3道工序,如表6.5所示。比如工序 p_{11} 代表第一个工件的第一道工序,可由机床1用2个单元时间完成,或由机床2用3个单元时间完成,或由机床3用4个单元时间完成。

表 6.5 柔性作业车间调度事例

工序选择		加工机床及加工时间					
		1	2	3	4	5	6
J_1	p_{11}	2	3	4			
	p_{12}		3		2	4	
	p_{13}	1	4	5			
J_2	p_{21}	3		5		2	
	p_{22}	4	3		6		
	p_{23}			4		7	11
J_3	p_{31}	5	6				
	p_{32}		4		3	5	
	p_{33}			13		9	12
J_4	p_{41}	9		7	9		
	p_{42}		6		4		5
	p_{43}	1		3			3

经算法运行 300 代后,得到最优解为 17 个单元时间。最优解甘特图、历代最优解收敛图分别如图 6.12、图 6.13 所示。

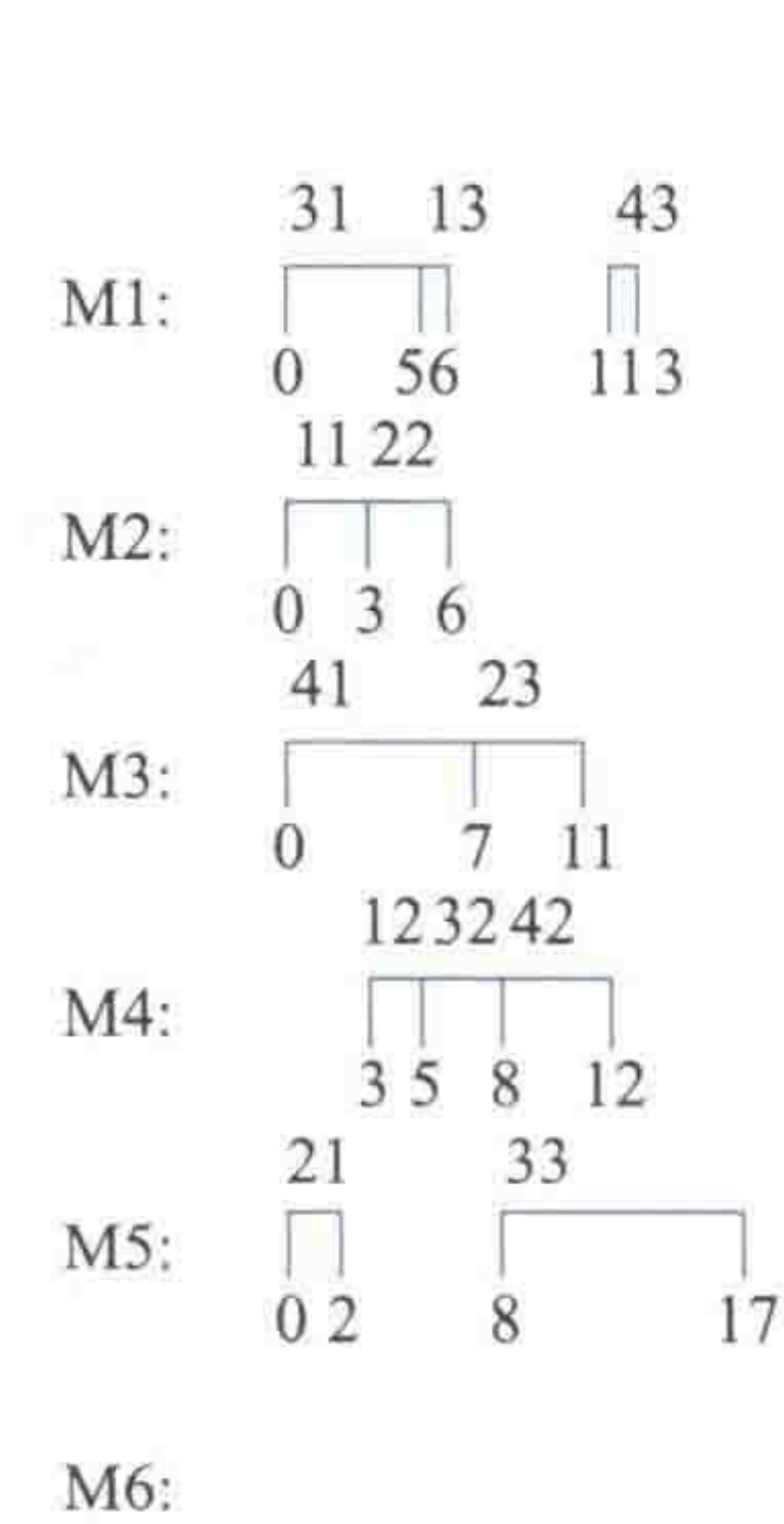


图 6.12 最优解甘特图

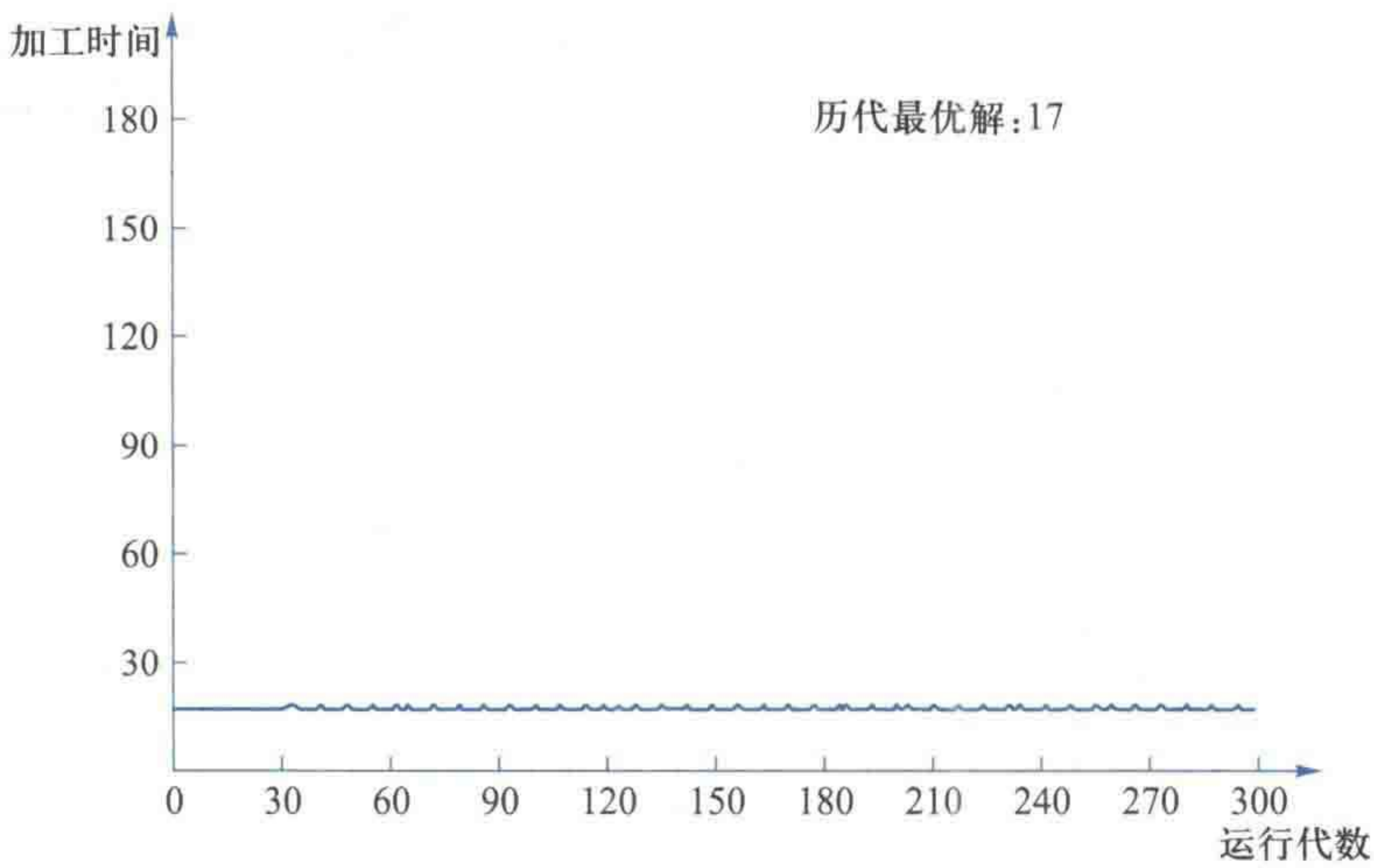


图 6.13 历代最优解收敛图

由图 6.12 可以看出机器 6 并没有加工任何工件。其原因因为它虽然可以加工工序 p_{23} 、 p_{33} 、 p_{42} 、 p_{43} ,但从表 6.5 可知机器 6 的加工时间大于其他可加工机器,特别是 p_{23} 、 p_{33} 的加工时间,因此机器 6 并未分到任何加工任务。

由图 6.13 可知,算法在大约 30 代以前就收敛到最优解,且各代最优解相差不大,可见算法较为稳定。

6.8 小 结

1. 基本遗传算法

遗传算法主要借用生物进化中“适者生存”的规律。

遗传算法的设计包括编码、适应度函数、选择、控制参数、交叉与变异等遗传算子。

遗传算法常用的编码方案有位串编码(二进制编码、Gray 编码)、实数编码等。

遗传算法中初始群体中的个体可以是随机产生的。群体规模太小时,遗传算法的优化性能一般不会太好,容易陷入局部最优解。而当群体规模太大时,则计算复杂。

遗传算法的适应度函数是用来区分群体中的个体好坏的标准。适应度函数一般是由目标函数变换得到的,但必须将目标函数转换为求最大值的形式,而且保证函数值非负。为了防止发生欺骗问题,对适应度函数值域的某种映射变换,称为适应度函数的尺度变换或者定标。

个体选择概率的常用分配方法有适应度比例方法、排序方法等。选择个体方法主要有轮

盘赌选择方法、锦标赛选择方法、最佳个体保存方法等。

遗传算法中起核心作用的是交叉算子。主要有一点交叉、二点交叉等基本的交叉算子,有部分匹配交叉等修正的交叉方法。

变异操作主要有位点变异、逆转变异、插入变异、互换变异等变异方法。

2. 改进遗传算法

双倍体遗传算法群体中的每个个体具有一个显性染色体,一个隐性染色体。每个染色体的编码/解码方式与基本的遗传算法相同。双倍体遗传算法采用显性遗传,即计算显性染色体的适应度,按照显性染色体的适应度进行选择、交叉、变异操作,隐性染色体也同时进行操作。当三个遗传算子都执行完成以后,将个体的染色体显隐性进行重新排定,个体中适应度较大的染色体设定为显性染色体,适应度较小的染色体设定为隐性染色体。

双种群遗传算法建立两个遗传算法群体,分别独立地运行复制、交叉、变异操作,同时当每一代运行结束以后,选择两个种群中的随机个体及最优个体分别交换。

自适应遗传算法的交叉和变异概率能够随适应度自动改变。当种群各个体适应度趋于一致或者趋于局部最优时,使交叉和变异概率增加,以跳出局部最优;而当群体适应度比较分散时,使交叉和变异概率减小,以利于优良个体的生存。

3. 粒子群优化算法

- ① 初始化每个粒子,即在允许范围内随机设置每个粒子的初始位置和速度。
- ② 评价每个粒子的适应度,计算每个粒子的目标函数。
- ③ 设置每个粒子的 p^i 。对每个粒子,将其适应度与其经历过的最好位置 p^i 进行比较,如果优于 p^i ,则将其作为该粒子的最好位置 p^i 。
- ④ 设置全局最优值 p^g 。对每个粒子,将其适应度与群体经历过的最好位置 p^g 进行比较,如果优于 p^g ,则将其作为当前群体的最好位置 p^g 。
- ⑤ 根据式(6.20)更新粒子的速度和位置。
- ⑥ 检查终止条件。如果未达到设定条件(预设误差或者迭代的次数),则返回第②步。

4. 蚁群算法

蚂蚁在运动过程中,根据各条路径上的信息素和启发信息按概率决定转移方向。

在 t 时刻蚂蚁 k 选择从元素(城市) x 转移到元素(城市) y 的概率:

$$P_{xy}^k(t) = \begin{cases} \frac{[\tau_{xy}(t)]^\alpha [\eta_{xy}]^\beta}{\sum_{y \in allowed_k(x)} [\tau_{xy}(t)]^\alpha [\eta_{xy}]^\beta} & y \in allowed_k(x) \\ 0 & \text{其他} \end{cases}$$

α 值越大,该蚂蚁越倾向于选择其他蚂蚁经过的路径,该状态转移概率越接近于贪婪规则。当 $\alpha=0$ 时,就不再考虑信息素水平,算法就成为有多重起点的随机贪婪算法。而当 $\beta=0$ 时,算法就成为纯粹的正反馈的启发式算法。

各路径上信息素浓度消散规则为: $\tau_{xy}(t+1) = \rho\tau_{xy}(t) + \Delta\tau_{xy}(t)$

蚁群的信息素浓度更新规则为: $\Delta\tau_{xy}(t) = \sum_{k=1}^m \Delta\tau_{xy}^k(t)$

蚂蚁圈系统: $\Delta\tau_{xy}^k(t) = \begin{cases} \frac{Q}{L_k} & \text{若第 } k \text{ 只蚂蚁在本次循环中从 } x \text{ 到 } y \\ 0 & \text{否则} \end{cases}$

蚂蚁数量系统: $\Delta\tau_{xy}^k(t) = \begin{cases} \frac{Q}{d_{xy}} & \text{若第 } k \text{ 只蚂蚁在本次循环中从 } x \text{ 到 } y \\ 0 & \text{否则} \end{cases}$

蚂蚁密度系统: $\Delta\tau_{xy}^k(t) = \begin{cases} Q & \text{若第 } k \text{ 只蚂蚁在本次循环中从 } x \text{ 到 } y \\ 0 & \text{否则} \end{cases}$

思考题

- 6.1 遗传算法的基本步骤和主要特点是什么?
- 6.2 适应度函数在遗传算法中的作用是什么? 试举例说明如何构造适应度函数。
- 6.3 选择的基本思想是什么?
- 6.4 简述多种群遗传算法与基本遗传算法的异同。
- 6.5 简述多倍体遗传算法与基本遗传算法的异同。
- 6.6 群智能算法的基本思想是什么?
- 6.7 群智能算法的主要特点是什么?
- 6.8 列举几种典型的群智能算法, 分析它们的主要优点、缺点。
- 6.9 简述群智能算法与进化算法的异同。
- 6.10 简述粒子群算法的流程。
- 6.11 简述粒子群算法位置更新方程中各部分的影响。
- 6.12 举例说明粒子群算法的搜索原理, 并简要叙述粒子群算法的特点。
- 6.13 粒子群算法的寻优过程包含哪几个阶段? 寻优的准则有哪些?
- 6.14 粒子群算法中的参数如何选择?
- 6.15 举例说明蚁群算法的搜索原理, 并简要叙述蚁群算法的特点。
- 6.16 蚁群算法的寻优过程包含哪几个阶段? 寻优的准则有哪些?
- 6.17 蚁群算法中的参数如何选择?

习题

6.1 已知 10 个个体的适应度如表 6.6 所示, 用幂函数变换法求出调整后的适应度值($f=2$), 然后采用适应度比例法分别求出调整前后各个个体的选择概率。

表 6.6 适 应 度 表

个体编号	原适应度	调整后的适应度	原选择概率	调整后的选择概率
1	2.5			
2	1.0			
3	3.0			
4	1.2			
5	2.1			
6	0.8			
7	2.3			
8	1.5			
9	0.9			
10	1.8			

6.2 用遗传算法求解下列函数的最大值

$f(x) = 0.4 + \text{sinc}(4x) + 1.1\text{sinc}(4x + 2) + 0.8\text{sinc}(x - 2) + 0.7\text{sinc}(6x - 4), x \in [-2, 2]$

其中

$$\text{sinc}(x) = \begin{cases} 1 & x = 0 \\ \frac{\sin(\pi x)}{\pi x} & x \neq 0 \end{cases}$$

提示：函数解用一个 16 位二进制数表示，种群规模 30， $P_c = 0.3$ ， $P_m = 0.01$ ，迭代次数 $N = 400$ ，最大值为 $f(-0.507\ 179) = 1.501\ 564$ 。

6.3 编制遗传算法的计算程序，具体求解一个优化问题。记录算法结束时的迭代次数，画出最优个体的适应度变化曲线和群体适应度的平均值变化曲线。

读一读

深度学习三巨头之一、卷积神经网络之父杨立昆

杨立昆(Yann LeCun)曾多次来中国进行交流,所以起了一个中文名字:杨立昆。杨立昆于1960年出生于法国巴黎附近,于1983年获得巴黎高等电子与电工技术工程师学院(ESIEE Paris)学士学位,随后获得皮埃尔和玛丽·居里大学计算机科学博士学位。1987年—1988年,杨立昆是多伦多大学Geoffrey Hinton实验室的博士后研究员。之后加入贝尔实验室,2003年加入纽约大学。现担任Facebook首席人工智能科学家和纽约大学教授。

杨立昆在人工神经网络方面作出了重要贡献,特别是在卷积神经网络方面,1998年提出的LeNet-5是卷积神经网络的基础模型。当时就在一些银行和公司的手写数字和字母识别方面获得成功应用。

杨立昆是美国国家科学院、美国国家工程院和法国科学院的院士。2019年3月27日,美国计算机协会(ACM)宣布把2018年的图灵奖颁给人工智能科学家Yoshua Bengio(约书亚·本吉奥)、Geoffrey Hinton(杰弗里·辛顿)和Yann LeCun(杨立昆),以表彰他们为当前人工智能的繁荣发展所奠定的基础。

2021年他出版图书《科学之路:人,机器与未来》介绍了他的学术生涯和研究成果,以及对人工智能未来的思考。